LinuxDesk

Product Analysis Document &
Software Requirements Specification

Version 1.0 — Pre-Development Analysis

Document ID: LD-SRS-001

24 July 2026

Draft for Stakeholder Review

Table of Contents

Design note (FR-DEP-056). Automatic rollback on health check failure is powerful and dangerous. If the health check is misconfigured, it will roll back healthy deployments. It must be opt-in, must be clearly explained at configuration time, and must be tested by the user in dry-run before being trusted in production.

Equally significant is the data-transmission question. Enterprise customers (Persona 7) will refuse a tool that sends server configuration to a third-party API. FR-AI-012 through FR-AI-017 are not optional privacy niceties; they are commercial prerequisites for the highest-value segment. AI should be positioned as a valuable optional layer, never as a core dependency — and the product must remain fully functional with every AI feature disabled.

6. Non-Functional Requirements

7. UI/UX Design

Multi-window. Any tab may be torn off into a separate window, supporting multi-monitor workflows — a common requirement for administrators monitoring one host while working on another.

Illustrative failure this prevents: a directory containing a file named foo; rm -rf ~ — legal on Linux — would, under naive string concatenation, cause a delete operation on that file to execute an arbitrary second command. This is not hypothetical; it is the single most likely path to a catastrophic defect in this class of application, and the CommandBuilder abstraction exists specifically to make it structurally impossible rather than merely unlikely.

Document Type: Product Analysis Document / Software Requirements Specification (SRS) **Product Working Name:** LinuxDesk (placeholder) **Version:** 1.0 (Pre-Development Analysis) **Status:** Draft for Stakeholder Review **Date:** 24 July 2026 **Classification:** Internal — Product & Engineering

---

Document Control

Field	Value
Document ID	LD-SRS-001
Revision	1.0
Authors	Product Architecture Group
Reviewers	Engineering Lead, Security Lead, UX Lead
Approval Required From	Product Owner, CTO
Next Review	Prior to Phase 1 kickoff

Revision History

Ver	Date	Author	Change Summary
0.1	—	Architecture	Initial skeleton, scope capture
0.5	—	Architecture	Market analysis, functional requirements
1.0	2026-07-24	Architecture	Full draft: all 15 sections, roadmap, risk register

Intended Audience

Audience	Sections of Primary Interest
Executive sponsors	1, 2, 12, 15
Product management	1–5, 12, 14, 15
Software architects	5, 6, 8, 9, 10, 11
UX designers	3, 4, 7
Security engineers	6, 10, 11, 13
QA / test engineers	5, 6, 10
Technical writers	All

Glossary of Terms

Term	Definition
Agentless	Operating on a remote host without installing a persistent daemon or binary on it
Capability probe	Startup routine that detects which tools, init system, and package manager a host provides
Command Abstraction Layer (CAL)	The layer translating GUI intents into shell command strings
Intent	A user-level task (e.g., "restart nginx") independent of the command that implements it
Adapter	A pluggable implementation of an intent for a specific distro/tool family
Session	A live authenticated connection to one remote host, including its channel pool
Profile	Stored connection configuration for a host (address, auth, preferences)
Preflight	A dry-run validation executed before a mutating operation
Sudo elevation	Executing a command with escalated privileges via `sudo`
Drift	Divergence between an expected local state and observed remote state
PTY	Pseudo-terminal; required for interactive prompts such as password entry

---

1. Executive Summary

1.1 Product Vision

LinuxDesk is a Windows desktop application, built in Java and JavaFX, that gives an operator a complete graphical operating environment for headless Linux servers. It communicates exclusively over SSH and SFTP, installs nothing on the target host, and presents every routine administrative task as a task-oriented graphical workflow rather than as a command to be recalled and typed.

The design premise is deliberately narrow and deliberately ambitious. Narrow, because the product does not attempt to be a terminal, an SSH client, a configuration-management system, or an orchestration platform. Ambitious, because it aims to cover the *breadth* of day-to-day server administration — files, services, processes, packages, users, networking, containers, databases, scheduled jobs, certificates, logs, backups, and deployments — behind a single coherent interface that a competent Windows user can operate on day one.

The mental model is explicit: **Windows Explorer for the filesystem, Task Manager for processes and resources, Services console for systemd, Event Viewer for logs, and Add/Remove Programs for packages** — all pointed at a remote Linux machine, all over a single encrypted channel.

A guiding constraint runs through the entire specification: *the product hides the terminal, it does not hide the truth.* Every action shows the command it will run before it runs it, reports what actually happened, and records the result. This distinction matters more than any single feature and is revisited throughout this document.

1.2 Problem Statement

Managing a headless Linux server today requires fluency in an interface that is powerful, terse, and unforgiving. That interface — the shell — imposes several distinct costs.

Cost 1: Recall burden. The shell is a recall interface, not a recognition interface. To change a file's group you must remember `chgrp`, or `chown :group`. To see which process holds port 8080 you must remember `ss -lntp`, or `lsof -i`, and know that both need elevation to show the process name. There is no menu to browse. A GUI converts recall to recognition, which is the single largest reduction in cognitive load available.

Cost 2: Fragmentation across distributions and eras. The "same" task has different commands depending on the host. Service management is `systemctl` on modern systems, `service` or `/etc/init.d` on older ones. Packages are `apt`, `dnf`, `yum`, `zypper`, `apk`, `pacman`, or `snap`. Firewalls are `ufw`, `firewalld`, `nftables`, or raw `iptables`. Network configuration is `netplan`, `NetworkManager`, `systemd-networkd`, or `/etc/network/interfaces`. An administrator must carry a mental matrix of tool-by-distro. Software can carry that matrix instead.

Cost 3: Destructive operations lack a confirmation surface. `rm -rf` has no undo and no preview. `chmod -R 777 /` executes as readily as any other command. `systemctl stop` on a database in production does exactly what it is told. The shell's greatest strength — it does precisely what you type, immediately — is also its greatest hazard in the hands of a tired or hurried operator.

Cost 4: Synthesis is manual. Answering "why is this server slow?" means running `top`, then `free -h`, then `df -h`, then `iostat`, then `journalctl`, then correlating the outputs by eye and by memory across separate screens. No single view assembles them.

Cost 5: The learning curve blocks otherwise-capable people. Developers who write excellent application code frequently cannot confidently deploy it. Students learning backend development hit an operations wall unrelated to their subject. Freelancers managing a handful of VPS instances for clients pay a recurring tax in lookup time. Small companies without a dedicated administrator either overpay for managed hosting or run under-maintained servers.

Cost 6: Knowledge is uncaptured and unauditable. Shell history is per-user, per-host, unstructured, and lost on rotation. When something breaks at 3 a.m., there is no structured record of who changed what and when.

The problem statement in one sentence: *routine Linux server administration demands memorized syntax, distribution-specific knowledge, and unguarded execution of irreversible commands — and the tools that could remove that demand each solve only a fraction of the surface.*

1.3 Why Current Tools Are Insufficient

The existing landscape splits into five families, none of which covers the target surface.

Family A — File transfer clients (WinSCP, FileZilla, Cyberduck). Excellent at moving bytes. WinSCP in particular is a mature, trusted, genuinely good Windows application for its scope. But the scope is files. There is no service management, no process management, no package management, no monitoring, no user administration. WinSCP's answer to any non-file task is to open a terminal window. The product is a file manager that happens to speak SSH.

Family B — Terminal multiplexers and SSH clients (PuTTY, MobaXterm, Termius, SecureCRT). These improve *access* to the shell — session management, tabs, key handling, saved profiles, sometimes an embedded SFTP pane. MobaXterm is a particularly rich example. But improving access to the shell is the opposite of the goal here. The user still types every command. A better terminal is still a terminal.

Family C — Web-based server control panels (Cockpit, Webmin, Virtualmin, cPanel, Plesk). These are the closest conceptual relatives and deserve the most careful analysis. Cockpit in particular is well-designed, officially supported on Red Hat and Debian families, and covers services, storage, networking, accounts, logs, containers, and updates. Webmin covers even more surface, if less elegantly.

Their limitation is architectural rather than functional: **they require server-side installation.** Cockpit needs the `cockpit` package and an open port. Webmin needs a Perl daemon and port 10000. cPanel needs a licensed platform-level install. This is disqualifying in three common situations — hardened hosts where installing an administrative daemon is prohibited by policy; hosts where only port 22 is reachable through the firewall or bastion; and fleets where the operator lacks authority to install anything. It is also a security consideration in its own right: a web control panel is an additional network-exposed, privileged attack surface, and both Webmin and cPanel have histories of serious remote vulnerabilities.

Secondary limitations: Cockpit is one-host-at-a-time by default (multi-host requires additional setup), browser-bound, and offers no local↔remote deployment workflow. Webmin's interface is dated and its module quality uneven.

Family D — Configuration management and orchestration (Ansible, Puppet, Chef, Salt, Rundeck, Terraform). These solve a genuinely different problem: *repeatable, declarative, fleet-scale state management.* Ansible is agentless over SSH — architecturally the closest to this product — but it is a YAML-authoring tool for engineers who already know both Linux and Ansible. It is superb for "configure 200 servers identically" and poor for "why is this one server's disk full right now." It has no interactive browse-and-inspect mode. It raises rather than lowers the required expertise.

Family E — Remote development environments (VS Code Remote SSH, JetBrains Gateway, GitHub Codespaces). Outstanding

for editing code on a remote host. VS Code Remote SSH is the best remote editing experience available. But it is developer-facing, requires installing a server component on the host, and treats administration as out of scope — its answer to "restart the service" is the integrated terminal.

Container-specific tools (Portainer, Rancher, Lens) are excellent within their domain and irrelevant outside it. Portainer manages Docker beautifully and cannot tell you why the host is out of memory.

The gap. No product currently offers: *agentless (SSH-only, zero server-side install), broad-surface (beyond files, beyond containers, beyond editing), task-oriented (GUI-first, not command-first), native-desktop (not browser-bound), and multi-host*. Each competitor holds three or four of these five properties. None holds all five.

1.4 Target Users

Detailed personas appear in Section 4. In summary:

Segment	Core Need	Willingness to Pay
Application developers	Deploy and troubleshoot without Linux fluency	Moderate — often expensed
Junior / generalist sysadmins	Guardrails, discoverability, safety on unfamiliar tasks	Moderate
Senior sysadmins & DevOps	Fast triage, fleet visibility, auditability	Low individually, high at team scale
Students & career changers	Learn by seeing commands behind actions	Low — free tier essential
Freelancers & agencies	Manage many small client servers efficiently	High relative to income
Small businesses without IT staff	Operate a server without hiring for it	High
Enterprise IT teams	Standardized, audited, least-privilege operations	Highest — but longest sales cycle

1.5 Unique Selling Proposition

Primary USP: *Full graphical control of a Linux server with nothing installed on the server.* SSH alone is the entire dependency. If you can SSH in, you can manage the machine graphically — including hosts where installing Cockpit is impossible or forbidden.

Supporting differentiators, in order of defensibility:

1. **Transparent abstraction.** Every action displays the exact command before execution, streams live output, and logs the result. Users learn the shell by using the GUI rather than being permanently insulated from it. This is a design commitment that competitors have generally not made — Cockpit and Webmin hide the command entirely.
2. **Breadth under one roof.** Files, services, processes, packages, users, network, Docker, databases, cron, certificates, logs, backups, and deployment in one application with one connection and one mental model.
3. **Deployment as a first-class workflow.** Local↔remote diff, selective sync, automatic pre-deploy backup, one-click rollback, post-deploy health check. This does not exist as an integrated workflow in any competitor at this level of the stack.
4. **Safety architecture.** Preflight validation, blast-radius warnings on recursive and system-path operations, typed confirmation for destructive actions, and an operation journal with rollback where technically feasible.
5. **Distribution neutrality via adapters.** One UI over a pluggable adapter layer that resolves systemd vs. SysV, apt vs. dnf vs. yum vs. zypper, ufw vs. firewalld vs. nft — detected automatically at connect time.
6. **Native desktop performance.** Real virtualized tables for 100k-row directories and process lists, OS-native drag-and-drop, multi-window, offline-capable local state.

Explicit non-goals (stated here because scope discipline is the primary execution risk):

- Not a better terminal emulator. An escape-hatch terminal exists; it is not the product.
- Not a configuration-management replacement. LinuxDesk is imperative and interactive; Ansible is declarative and repeatable. They coexist.
- Not a hosting control panel. No mail server, DNS zone hosting, or virtual-host provisioning as a product category.
- Not a Kubernetes platform in v1. Deferred to Phase 5 and scoped narrowly even then.

1.6 Market Opportunity

Market sizing (order-of-magnitude estimates; validate before committing spend). Linux runs the large majority of public-facing web servers and cloud instances. The population of people who administer at least one Linux host — professionally or otherwise — is plausibly in the tens of millions worldwide. The subset that is (a) Windows-desktop-based, (b) not fluent in the shell or not interested in using it for routine work, and (c) able to pay for tooling is far smaller, but a realistic serviceable obtainable market in the low hundreds of thousands of seats is defensible.

Two structural trends favor the product:

1. **Widening operator base.** Cloud VPS provisioning has become trivial. Many people now own servers who never trained as administrators. The supply of servers has outgrown the supply of administrators.
2. **Persistent Windows desktop share in enterprise.** The administrator's *desktop* is very often Windows even when every managed host is Linux. Tools that are Windows-native have a real distribution advantage in that segment.

Monetization model (recommended):

Tier	Price Point (indicative)	Includes
Free	$0	3 hosts, files, editor, services, processes, monitoring, logs

Pro	~$8–12 / user / month	Unlimited hosts, deployment, Docker, databases, backups, AI assist
Team	~$20–25 / user / month	Shared profiles, RBAC, centralized audit log, SSO
Enterprise	Custom	Self-hosted licence server, air-gap, compliance reporting, support SLA

The free tier is strategically important: it drives the bottom-up adoption motion that Cockpit's zero-cost bundling would otherwise dominate.

Go-to-market: developer-community-led (technical content, YouTube demonstrations, Reddit/HN presence), with a self-serve Pro conversion and a later outbound motion into IT teams. Bundling with VPS providers as a value-added tool is a plausible secondary channel.

Principal market risks: Cockpit is free and improving; "GUI for Linux" carries cultural stigma among senior administrators; support burden across the distribution matrix is substantial. These are quantified in Section 13.

---

2. Existing Market Analysis

Each product below is assessed on purpose, strengths, weaknesses, missing capabilities relative to this specification, and the specific delta LinuxDesk offers. Assessments reflect the products as generally understood at the time of writing; competitive analysis should be refreshed before each major release.

2.1 WinSCP

Purpose. Free, open-source Windows client for file transfer over SFTP, SCP, FTP, and WebDAV. The de facto default for Windows users moving files to Linux hosts.

Strengths. Extremely mature and stable. Dual-pane Norton Commander layout familiar to Windows power users. Excellent transfer resume, queue management, and synchronization of directory trees. Integrated text editor. Scripting and .NET assembly for automation. Portable mode. Free with a permissive licence, and genuinely trusted.

Weaknesses. Scope is files and only files. Its "remote command" facility is a raw prompt with no output UI worth the name. No service, process, package, user, or network management. No monitoring. Interface, while efficient, is visually dated. No multi-host dashboard.

Missing relative to this spec. Services; processes; packages; monitoring; users; firewall; Docker; databases; cron; certificates; deployment workflow with rollback; log viewer; AI assistance.

How LinuxDesk differs. LinuxDesk treats file management as approximately one-fifteenth of the product rather than the whole of it. WinSCP is nonetheless the benchmark for transfer-engine quality and its synchronization UX should be studied closely — matching it is a hard requirement, not a nice-to-have. WinSCP is also the most likely source of user comparison ("is the transfer as good as WinSCP?"), so the file module must not be visibly weaker.

2.2 MobaXterm

Purpose. All-in-one Windows terminal application: SSH, telnet, RDP, VNC, X11 server, embedded SFTP browser, network tools, and a Cygwin-based local Unix environment.

Strengths. Remarkable feature density. The automatic SFTP pane alongside each SSH session is genuinely useful. Built-in X11 server for remote GUI applications is unique among Windows tools. Session organization, macros, and a plugin system. Multi-protocol coverage in one binary.

Weaknesses. Fundamentally terminal-centric — the user still types every command. The interface is cluttered and inconsistent, having accreted features over many years. Free edition limits saved sessions and tunnels. Windows-only. No task-oriented administrative workflows: no service dashboard, no process table with actions, no package UI.

Missing relative to this spec. Every abstraction layer. MobaXterm provides access to Linux, not abstraction of it.

How LinuxDesk differs. Directly inverted philosophy. MobaXterm optimizes typing commands; LinuxDesk eliminates it. MobaXterm's session-organization model and its SFTP-alongside-shell pattern are worth borrowing; its interface density is a cautionary example.

2.3 PuTTY

Purpose. Minimal, free SSH and telnet client for Windows. The historical baseline.

Strengths. Tiny, fast, single-executable, no installation. Utterly dependable. Universally available and universally understood. PSCP/PSFTP/Plink companions for transfer and scripting. Puttygen for key generation.

Weaknesses. Bare terminal only. Session management is primitive. No tabs. No file browser. No SFTP GUI. Dated interface. No administrative functionality of any kind.

Missing relative to this spec. Everything above the raw connection.

How LinuxDesk differs. Different category entirely. PuTTY is relevant chiefly as the tool most users will be migrating *from*, and as a reminder that reliability and startup speed earn deep loyalty. It also sets the expectation that key handling must be flawless — PuTTY users will arrive with `.ppk` files, so importing PuTTY-format keys is a concrete compatibility requirement.

2.4 Termius

Purpose. Modern cross-platform SSH client (Windows, macOS, Linux, iOS, Android, web) with cloud-synchronized credentials and teams features.

Strengths. Best-in-class visual design among SSH clients. Genuine cross-platform including mobile. End-to-end encrypted vault syncing hosts and keys across devices. Snippets library. Port forwarding UI. Team credential sharing. Strong onboarding.

Weaknesses. Still terminal-first. Subscription pricing with meaningful features behind the paywall. Cloud sync is a trust and compliance consideration for regulated environments. SFTP browser is adequate but not a WinSCP substitute. No service, process, or package management.

Missing relative to this spec. All administrative abstraction; monitoring dashboards; deployment; Docker; databases.

How LinuxDesk differs. Termius modernized the terminal; LinuxDesk replaces the need for it. Termius's vault-sync design, team-sharing model, and pricing structure are the closest available template for LinuxDesk's own Team tier — worth studying in detail. Its polish sets the visual bar.

2.5 VS Code Remote SSH

Purpose. Extension enabling VS Code to edit, build, and debug on a remote host as though local.

Strengths. Exceptional remote editing. Full language services, IntelliSense, and debugging execute on the remote host. Integrated terminal, source control, and the entire extension ecosystem. Free. Enormous existing user base.

Weaknesses. Installs a server component (VS Code Server) on the remote host, consuming memory and disk and requiring a compatible environment — this fails on hardened, minimal, or resource-constrained hosts. Developer-focused; administration is out of scope by design. Requires a modern remote environment (glibc version constraints have historically excluded older hosts). Heavy for casual server inspection.

Missing relative to this spec. Services; processes; packages; users; firewall; monitoring dashboards; deployment with rollback; Docker UI; database UI.

How LinuxDesk differs. Complementary rather than competing. The realistic positioning is "VS Code for code, LinuxDesk for the machine," and the two should interoperate — an "Open in VS Code" action from a LinuxDesk directory is a sensible integration. LinuxDesk's editor should be competent but must not attempt to compete with VS Code; that is a losing battle and a scope trap.

2.6 Cockpit

Purpose. Web-based server administration console, developed under Red Hat sponsorship, packaged for most major distributions.

Strengths. The strongest functional overlap with this specification. Covers services, storage and LVM, networking, firewall, user accounts, logs (journal), software updates, containers (Podman), performance metrics, and includes an embedded terminal. Free and open source. Uses the host's own PAM authentication and respects existing privileges. Well-designed and genuinely pleasant to use. Officially supported and actively maintained. Modular and extensible.

Weaknesses. Requires server-side installation — the `cockpit` package plus an open port (9090). This is the decisive architectural difference. Multi-host management requires additional configuration and remains awkward. Browser-bound, with the performance ceiling and interaction limits that implies for large tables. No local↔remote deployment workflow. No database management. Docker support is Podman-oriented. Adds a privileged, network-exposed service to every managed host.

Missing relative to this spec. Agentless operation; native desktop UX; local↔remote deployment with rollback; database management; unified cross-host dashboard; local file drag-and-drop; AI assistance; offline history.

How LinuxDesk differs. This is the principal competitor and the honest comparison must be made repeatedly during development. Cockpit is free, good, and pre-packaged. LinuxDesk wins on: hosts where installation is impossible or prohibited; environments where only port 22 traverses the firewall; multi-host fleet work; local-to-remote deployment; native desktop responsiveness; and desire to avoid an extra privileged service per host. **If a team can install Cockpit everywhere and manages hosts one at a time, Cockpit may well be the better choice — and the product's positioning should be honest about that.** Overstating differentiation here will damage credibility with exactly the technical audience the product needs.

2.7 Webmin

Purpose. Long-established web-based system administration interface for Unix-like systems, with an unusually broad module catalogue.

Strengths. Very wide surface area — users, packages, cron, filesystems, Apache, BIND, Samba, MySQL, firewall, backups, and hundreds of third-party modules. Extremely mature (in development since the 1990s). Free. Highly extensible. Virtualmin/Cloudmin extend it into hosting control panel territory.

Weaknesses. Requires server-side installation and a daemon on port 10000. Interface is dated and inconsistent between modules. Module quality varies considerably. Security history includes serious remote code execution vulnerabilities, including a supply-chain compromise of official packages. Perl-based architecture is showing its age. Single-host orientation.

Missing relative to this spec. Agentless operation; native desktop client; modern UX; deployment workflow; container management of current quality; unified fleet view.

How LinuxDesk differs. Webmin demonstrates both the demand for broad graphical Linux administration and the risks of unbounded scope with uneven quality. It is a useful *feature checklist* and an equally useful warning: breadth without consistency produces a tool people tolerate rather than choose. LinuxDesk should aim for narrower initial breadth at consistently higher quality.

2.8 Portainer

Purpose. Web UI for Docker, Docker Swarm, and Kubernetes container management.

Strengths. Excellent within its domain. Clear container, image, volume, network, and stack management. Multi-environment support. Good RBAC in the Business edition. Widely adopted and well-regarded. Fast to deploy (itself a container).

Weaknesses. Containers only — no host-level administration. Runs as a container with Docker socket access, which is a significant privilege grant. Advanced features are commercially licensed. Web-based.

Missing relative to this spec. All host-level management — files, services, processes, packages, users, network, monitoring.

How LinuxDesk differs. LinuxDesk's Docker module will not match Portainer's depth and should not try in v1. The value is contextual integration: seeing the container list *alongside* host CPU, the systemd unit that starts the stack, and the compose file in the file browser. Portainer answers "what are my containers doing"; LinuxDesk answers "what is my server doing, including its containers."

2.9 Rundeck

Purpose. Operations runbook automation — defining, scheduling, and delegating job execution across nodes with access control and audit.

Strengths. Strong job definition, scheduling, and execution history. Fine-grained ACLs. Self-service delegation to non-experts (a genuinely overlapping goal). Node filtering across large fleets. Good audit trail. Integrates with existing tooling.

Weaknesses. Requires server installation and non-trivial configuration. Job-centric, not exploration-centric — someone must author the jobs first. No file browser, no interactive inspection, no monitoring dashboards. Steep initial setup. Java web application with real operational overhead.

Missing relative to this spec. Interactive administration; file management; monitoring; desktop client.

How LinuxDesk differs. Rundeck delegates *predefined* operations to non-experts; LinuxDesk enables *ad hoc* operations by non-experts. Rundeck's audit and ACL model is a strong reference for LinuxDesk's Team tier. The saved-operations concept in LinuxDesk's roadmap is deliberately a lightweight echo of Rundeck's jobs.

2.10 Ansible

Purpose. Agentless configuration management and orchestration over SSH using declarative YAML playbooks.

Strengths. Agentless over SSH — architecturally the closest relative to this product. Idempotent and declarative. Enormous module ecosystem. Scales to thousands of hosts. Version-controllable infrastructure-as-code. Free core, with AAP/AWX for enterprise. Industry standard.

Weaknesses. Steep learning curve requiring both Linux knowledge and Ansible knowledge — it *raises* the expertise floor rather than lowering it. No interactive exploration; no browsing, no live inspection. Slow for one-off tasks. Debugging playbooks is frustrating. Requires Python on the target for most modules. No GUI in the open-source core (AWX/AAP add one, but it is a playbook-execution UI, not an administration UI).

Missing relative to this spec. Interactive administration; graphical exploration; monitoring; file browsing; approachability for non-experts.

How LinuxDesk differs. Different problem, complementary tool. Ansible is for repeatable fleet-wide state; LinuxDesk is for interactive single-host and small-fleet work. The clearest positioning: "*Ansible makes 200 servers identical. LinuxDesk tells you why one of them is broken.*" A future integration — generating an Ansible task snippet from a sequence of LinuxDesk actions — would be a strong differentiator and is proposed in Section 14. Ansible's agentless SSH architecture also validates LinuxDesk's core technical bet: the SSH-only approach is proven at scale.

2.11 Visual Studio Code (Local)

Purpose. General-purpose extensible code editor.

Strengths. Best-in-class editing, extension ecosystem, free, ubiquitous.

Weaknesses. Not a server administration tool. Remote capability requires extensions and a server component.

How LinuxDesk differs. Sets the quality bar for the embedded editor's *feel* — users will expect find-and-replace, multi-cursor, and syntax highlighting to behave like VS Code. Matching that bar approximately is realistic; matching it fully is not, and attempting to is a scope trap.

2.12 GitHub Codespaces

Purpose. Cloud-hosted development environments accessed via browser or VS Code.

Strengths. Zero local setup; consistent, disposable, pre-configured environments; deep GitHub integration.

Weaknesses. Ephemeral development environments, not managed servers. Consumption-based pricing. Requires internet and GitHub. Not applicable to existing production infrastructure.

How LinuxDesk differs. Codespaces provisions new development environments; LinuxDesk manages existing servers. Minimal overlap. Relevant only as evidence that developers will accept remote-first workflows when the experience is good enough.

2.13 Additional Relevant Products

Product	Category	Relevance
FileZilla	FTP/SFTP client	Free alternative to WinSCP; same narrow scope; bundled-adware history has damaged trust
Bitvise SSH Client	Windows SSH/SFTP	Strong tunneling and SFTP; still terminal-centric
SecureCRT / SecureFX	Commercial SSH suite	Enterprise-grade, scriptable, expensive; terminal-centric
Royal TS	Multi-protocol connection manager	Excellent credential and connection organization across RDP/SSH/VNC; no Linux abstraction. Strong reference for LinuxDesk's profile management
Xshell / Xftp	Commercial Windows SSH suite	Popular in Asian markets; terminal-centric
Tabby (formerly Terminus)	Open-source modern	Attractive, extensible, terminal-centric

	terminal	
Windows Terminal + OpenSSH	Built-in Windows tooling	Free, native, improving; raises the baseline every LinuxDesk feature must beat
cPanel / Plesk	Hosting control panels	Broad and polished but hosting-oriented, licensed, and heavyweight; require full platform install
ISPConfig / CyberPanel / aaPanel	Open-source hosting panels	Free alternatives; server-installed; hosting-focused
Ajenti	Lightweight web admin panel	Server-installed; smaller community
Netdata	Real-time monitoring	Superb monitoring depth; agent-based; monitoring only. Sets the visual bar for LinuxDesk's metrics views
Zabbix / Nagios / Prometheus+Grafana	Monitoring platforms	Fleet monitoring and alerting; agent/exporter based; no administration
Lens	Kubernetes IDE	Excellent desktop Kubernetes client; strong reference for Phase 5
Rancher	Kubernetes management	Platform-scale; out of scope
Puppet / Chef / SaltStack	Configuration management	Agent-based (mostly); same category as Ansible
Teleport	Access proxy / bastion	Access control and audit for SSH/K8s; complementary — LinuxDesk should be able to connect *through* it
Guacamole	Clientless remote gateway	Browser-based RDP/VNC/SSH access; access not administration
Foreman	Provisioning & lifecycle	Provisioning-oriented; enterprise-scale
Proxmox VE	Virtualization platform	Manages the hypervisor, not the guest OS
DirectAdmin	Hosting panel	Lightweight commercial panel; hosting-focused
Termius Teams / SSH key managers (HashiCorp Vault, 1Password SSH agent)	Credential management	Complementary; LinuxDesk should integrate rather than reimplement

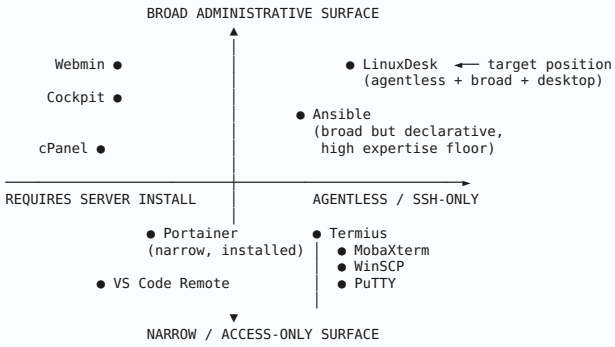
2.14 Comparison Matrix

Legend: ● Full · ◐ Partial · ○ None · ▲ Planned (LinuxDesk)

Capability	WinSCP	MobaXterm	PuTTY	Termius	VSCode Remote	Cockpit	Webmin	Portainer	Rundeck	Ansible	LinuxDesk
Agentless (SSH only)	●	●	●	●	○	○	○	○	◐	●	●
No server install required	●	●	●	●	○	○	○	○	○	●	●
Native desktop app	●	●	●	●	●	○	○	○	○	○	●
Windows-first UX	●	●	●	◐	◐	○	○	○	○	○	●
GUI file management	●	◐	○	◐	◐	◐	◐	○	○	○	●
Drag & drop transfer	●	◐	○	◐	○	○	○	○	○	○	●
Remote file editor	◐	◐	○	◐	●	◐	◐	○	○	○	●
Service management (systemd)	○	○	○	○	○	●	●	○	◐	●	●
Process management	○	○	○	○	○	◐	●	◐	○	◐	●
Resource monitoring	○	○	○	○	○	●	◐	◐	○	○	●
Package management	○	○	○	○	○	●	●	○	○	●	●
User & group management	○	○	○	○	○	●	●	◐	○	●	●
Network & firewall config	○	○	○	○	○	●	●	◐	○	●	●
Docker management	○	○	○	○	◐	◐	◐	●	○	◐	●
Kubernetes management	○	○	○	○	◐	○	○	●	○	◐	▲
Database management	○	○	○	○	◐	○	●	○	○	◐	●
Cron / scheduler UI	○	○	○	○	○	◐	●	○	●	◐	●
Log viewer (live)	○	◐	○	○	◐	●	◐	◐	◐	○	●
Backup & restore	◐	○	○	○	○	◐	●	◐	○	◐	●
Certificate / SSL management	○	○	○	○	○	◐	●	○	○	◐	●
Deployment with rollback	◐	○	○	○	○	○	○	◐	●	●	●
Local↔remote diff	●	◐	○	○	◐	○	○	○	○	○	●
Multi-host dashboard	◐	◐	○	●	◐	◐	○	●	●	●	●
Shows command before running	◐	●	●	●	●	○	○	○	◐	◐	●
Destructive-action guardrails	◐	○	○	○	○	◐	◐	◐	●	◐	●
Audit log of actions	◐	○	○	○	○	◐	◐	◐	●	●	●

Cross-distro adapters	n/a	n/a	n/a	n/a	n/a	●	●	n/a	◐	●	●
AI assistance	○	○	○	◐	●	○	○	○	○	○	●
Plugin architecture	◐	●	○	◐	●	●	●	◐	●	●	▲
Free tier	●	◐	●	◐	●	●	●	◐	●	●	●

2.15 Positioning Map



The upper-right quadrant — broad administrative surface *and* agentless — is currently occupied only by Ansible, which sits there with a very high expertise requirement. That quadrant, at a low expertise requirement, is empty. That is the opportunity.

---

3. Product Vision

3.1 Vision Statement

> ***LinuxDesk is a graphical desktop environment for managing Linux servers.***

Not a client for reaching Linux servers. An environment for *operating* them — where the server's filesystem, services, processes, resources, users, packages, network, containers, and logs are all directly manipulable objects in a coherent desktop interface, and where the SSH connection underneath is an implementation detail rather than the interaction model.

Supporting statement:

> *Manage a Linux server as easily as you manage Windows — without installing anything on the server, and without ever needing to remember a command.*

3.2 The Philosophy of Hiding Complexity

3.2.1 The Core Distinction: Hiding Complexity vs. Hiding Reality

The single most important design principle in this document:

> ***LinuxDesk hides the complexity of Linux. It does not hide the reality of Linux.***

Complexity is the syntax, the flag combinations, the distribution-specific tool names, the argument ordering, the memorization. That should be removed entirely — it is accidental difficulty that serves no user.

Reality is what actually happens on the machine: which command ran, what it returned, what changed, what broke, and what it cannot do. That must remain visible at all times.

The failure mode this principle guards against is the "magic button" — an interface element that performs an opaque action and reports only success or failure. Magic buttons work until they do not, and when they fail the user has less information than if they had used the shell, and no path to recovery. Every graphical Linux tool that has been abandoned by technical users was abandoned for this reason. **Trust is the product's primary currency, and opacity spends it.**

The practical expression of this principle is the **Command Transparency Panel**: a collapsible panel present in every module showing the exact command about to execute, and after execution, the exit code, stdout, stderr, and elapsed time. Users may collapse it permanently. It is never removed, never abbreviated, and never paraphrased.

3.2.2 Design Principles

P1 — Recognition over recall. The user should never need to produce a command from memory. Every capability is discoverable by browsing menus, ribbons, and context menus. The measure of success: a user who has never seen the product can locate any core function within thirty seconds without documentation.

P2 — Task-oriented, not command-oriented. Interface elements are named for user intent, not for the tool that implements it. "Install a program" rather than "apt-get." "Give this user administrator rights" rather than "add to sudoers." "Open port 443 for web traffic" rather than "ufw allow." The command is shown; the *label* is the task.

P3 — Show the command, always. See 3.2.1. Every action shows what it will run before running it, and what happened after. Non-negotiable.

P4 — Safe by default, powerful on request. Defaults are conservative. Destructive operations require confirmation proportionate to blast radius: a simple confirm for a single file, an explicit typed confirmation for a recursive delete, a hard block with an override for operations on system-critical paths. Advanced capability is never removed — it is placed behind deliberate action.

P5 — Progressive disclosure. The default view shows what most users need most of the time. Depth is one click away, not zero clicks away. A novice sees "Restart"; an expert expands to see the unit file, dependency tree, and last fifty journal lines.

P6 — Explain, don't just execute. When something fails, interpret the error. "Permission denied" becomes "You need administrator rights to modify this file. Retry with elevated privileges?" with a button. Errors are teaching moments and recovery points, not dead ends.

P7 — Never silently mutate. No background action changes remote state without the user's knowledge. Read-only polling for monitoring is acceptable and disclosed; writes are always user-initiated and always logged.

P8 — Familiarity over novelty. Where a Windows convention exists, use it. F2 renames. Delete deletes. Ctrl+C/Ctrl+V copies and pastes files. Right-click opens a context menu. The user's existing knowledge is an asset to be spent, not an obstacle to be retrained.

P9 — Fail loudly, recover gracefully. Errors are surfaced clearly with actionable next steps. Partial failures in batch operations are itemized — never "operation failed" for a 500-file transfer where 3 files failed.

P10 — The escape hatch is always available. A real terminal is one keystroke away (Ctrl+`). This is not an admission of defeat; it is a trust signal. Users who know a tool cannot trap them use it more freely. The terminal shares the same session and working directory as the current view.

3.2.3 The Abstraction Ladder

Users occupy different rungs, and the interface must serve all of them simultaneously without forcing a mode choice.

Rung	User State	What They See	Design Response
0	No Linux knowledge	Task labels only ("Restart the web server")	Plain-language actions, no jargon in primary labels
1	Recognizes concepts	Task labels + entity names ("Restart nginx.service")	Real identifiers shown alongside plain language
2	Learning by observation	Command preview visible	Transparency panel expanded by default for this cohort
3	Competent, prefers GUI speed	Command preview collapsed, keyboard shortcuts	Full keyboard operability, command palette
4	Expert, GUI for specific tasks	GUI where faster, terminal where not	Seamless terminal handoff preserving context

A user typically starts at rung 0 or 1 and drifts upward. **The product should make that drift comfortable and even pleasurable — a user who learns Linux through LinuxDesk becomes an advocate rather than a churned customer.** This reframes what would otherwise be a retention risk (users graduating to the shell) into an acquisition channel.

3.2.4 The Windows Metaphor Mapping

Windows Concept	LinuxDesk Equivalent	Linux Reality Underneath
File Explorer	File Manager module	`ls`, `stat`, SFTP protocol operations
Task Manager → Processes	Process Manager	`ps`, `top`, `/proc`, `kill`, `renice`
Task Manager → Performance	Dashboard / Monitoring	`/proc/stat`, `/proc/meminfo`, `df`, `ss`, `vmstat`
Services console (`services.msc`)	Service Manager	`systemctl`, `journalctl`, unit files
Add/Remove Programs	Package Manager	`apt`, `dnf`, `yum`, `zypper`, `snap`
Event Viewer	Log Viewer	`journalctl`, `/var/log/*`, `tail -F`
Computer Management → Users	User Manager	`useradd`, `usermod`, `passwd`, `/etc/passwd`, `/etc/group`
Task Scheduler	Cron Manager	`crontab`, systemd timers
Windows Defender Firewall	Firewall module	`ufw`, `firewalld`, `nft`, `iptables`
Disk Management	Storage module	`lsblk`, `df`, `mount`, `fstab`, LVM tools
Network Connections	Network module	`ip`, `nmcli`, `netplan`, `resolvectl`
Certificate Manager	Certificate module	OpenSSL, certbot, `/etc/ssl`
Backup and Restore	Backup module	`tar`, `rsync`, `mysqldump`, `pg_dump`
Registry / System Properties	System Configuration	`sysctl`, `/etc/*`, environment files

This mapping is a **UX scaffold, not a literal translation.** Linux concepts without Windows analogues — SELinux contexts, systemd unit dependency graphs, cgroups, namespaces — are presented on their own terms rather than distorted into a false Windows equivalent. Forcing a bad metaphor is worse than teaching a new concept.

3.2.5 What the Vision Explicitly Excludes

Excluded	Rationale
Being a better terminal	Opposite of the product thesis
Replacing Ansible/Puppet	Different problem class (declarative fleet state)
Hosting control panel features (mail, DNS zones, vhost provisioning)	Different market, heavy support burden
Managing hypervisors (Proxmox, ESXi, VM lifecycle)	Guest OS is the scope; hypervisor is not
Full IDE capability	VS Code exists and wins; interoperate instead
Agent-based deep monitoring (Netdata-class metric depth)	Violates the agentless principle; integrate instead
Windows Server management	Different OS, different product
Mobile-first design	Desktop-first; mobile is a Phase 5 companion at most

3.3 Product Personality

LinuxDesk should feel **competent, calm, and honest.**

- **Competent** — fast, precise, correct. It does not lie about what happened. Latency is visible, not hidden behind fake progress.
- **Calm** — no alarm fatigue, no gratuitous animation, no colored badges competing for attention. Serious tools are quiet. Red means something is actually wrong.
- **Honest** — it shows the command, admits when it cannot do something, and never claims success it did not verify.

It should *not* feel playful, chatty, or clever. It is a tool that people use when production is down at 3 a.m. It should behave accordingly.

---

4. Target Users

Seven primary personas, each with context, goals, pain points, product usage, success criteria, and monetization notes.

4.1 Persona Overview

#	Persona	Linux Skill	Hosts	Frequency	Tier	Priority
1	Application Developer	Low–Medium	1–5	Weekly	Pro	**P0**
2	System Administrator	Medium–High	10–100	Daily	Pro/Team	**P0**
3	DevOps Engineer	High	50–500	Daily	Team	P1
4	Student / Learner	None–Low	1–2	Weekly	Free	P1
5	Freelancer / Agency	Medium	5–50	Daily	Pro	**P0**
6	Small Business Operator	None–Low	1–3	Monthly	Pro	P2
7	Enterprise IT Team	Mixed	100+	Daily	Enterprise	P2 (high value, long cycle)

P0 personas drive Phase 1–2 feature prioritization. Personas 1, 2, and 5 share a common core need — files, deployment, services, logs, monitoring — which is exactly the Phase 1 scope defined in Section 12. This alignment is deliberate.

4.2 Persona 1 — The Application Developer

Profile. Priya, 29, backend developer at a 40-person SaaS company. Windows laptop, WSL used reluctantly. Writes Java and Python confidently. Deploys to three Ubuntu VPS instances. Considers Linux administration a tax on her actual job.

Goals. Deploy code changes without ceremony. Read application logs when something breaks. Restart the service after a config change. Confirm the server has not run out of disk. Return to writing code.

Pain points. Cannot remember `rsync` flags and re-reads the same Stack Overflow answer monthly. Deploys by dragging files in WinSCP and hoping nothing was missed. Restarting a service means opening PuTTY and recalling `sudo systemctl restart`. Reading logs means `tail -f` on a path she has to look up. Once ran `rm -rf` in the wrong directory and lost an afternoon.

Usage. Deployment module (compare local↔remote, deploy changed files, rollback), log viewer with error highlighting, service restart, file editor for config, disk usage check.

Success criteria. Deploys in under two minutes without opening a terminal. Locates the relevant error in logs in under thirty seconds. Never fears an irreversible mistake.

Value. Time reclaimed and anxiety removed. **Willingness to pay: moderate-to-high, and frequently expensed to the employer.** Priya is the highest-conversion persona per unit of engineering effort.

Design implications. Deployment must be the most polished module in Phase 1. Log viewing must be genuinely fast. Rollback must be one click and must actually work.

4.3 Persona 2 — The System Administrator

Profile. Marcus, 41, sole administrator for a 120-person manufacturing firm. Manages roughly 60 servers — mixed Ubuntu, RHEL, and a few remaining CentOS 7 hosts. Twelve years of experience. Comfortable in the shell but chronically time-poor.

Goals. Triage incidents quickly. Perform routine maintenance across many hosts without repetition. Onboard a junior colleague without hand-holding every task. Maintain an audit trail for the annual compliance review.

Pain points. Context-switching between six terminal windows during an incident. Distribution differences (RHEL vs. Ubuntu) cause errors under time pressure. Bulk operations mean writing throwaway scripts. Junior staff cannot be trusted with root and must interrupt him constantly. No record of who changed what.

Usage. Multi-host dashboard, process and service management, package updates across hosts, user management, firewall configuration, audit log, saved operations.

Success criteria. Diagnoses "server is slow" in under a minute from one screen. Applies security updates to twenty hosts in one workflow. Delegates safe tasks to a junior without granting root.

Value. Time multiplication and delegation capability. **Willingness to pay: moderate individually, high at team scale** — Marcus is the buyer for Team tier.

Design implications. Multi-host operations and the audit log are what convert Marcus. He will also be the harshest critic of any abstraction that lies — he will test the command preview against his own expectations and abandon the product if they diverge.

4.4 Persona 3 — The DevOps Engineer

Profile. Chen, 34, platform engineer at a 300-person technology company. Manages several hundred hosts, largely through Terraform and Ansible. Deeply comfortable in the shell. Skeptical of GUIs on principle.

Goals. Fast incident triage on hosts that automation cannot reach or has not yet fixed. Visual confirmation that a playbook produced the intended result. Onboarding new team members faster.

Pain points. Ansible is poor for exploration and one-off investigation. Debugging a single misbehaving host means SSH plus a sequence

of diagnostic commands. Explaining infrastructure to new hires is slow without visuals.

Usage. Monitoring dashboards, process inspection, container management, log analysis, configuration diff. **Rarely uses file management.** Will use the embedded terminal frequently and will judge the product partly on how well the terminal integrates.

Success criteria. Triages faster than SSH plus manual commands. Never obstructed by the abstraction. Can always drop to a shell instantly.

Value. Speed for a narrow set of tasks. **Willingness to pay: low individually.** Chen matters less as a customer and more as an *influencer* — his public opinion shapes whether the technical community treats the product as credible or as a toy.

Design implications. Chen is why P3 (show the command) and P10 (escape hatch) are non-negotiable. A single "magic button" that does something he did not expect will produce a dismissive public review. Design for his skepticism explicitly.

4.5 Persona 4 — The Student / Career Changer

Profile. Aisha, 22, computer science student. Windows machine. Rented a $5/month VPS to learn deployment. Overwhelmed by the shell.

Goals. Deploy a personal project and see it work. Understand what commands actually do. Build confidence without breaking things irrecoverably.

Pain points. Tutorials assume knowledge she lacks. Error messages are opaque. No mental model for the filesystem hierarchy or the permission model. Terrified of destroying the server.

Usage. File browser (to build a spatial model of the filesystem), command transparency panel (**the primary learning mechanism**), service management, package installation, error explanations.

Success criteria. Deploys a working application. Can explain what `systemctl restart nginx` does. Graduates to comfortable shell use within months.

Value. Learning acceleration. **Willingness to pay: very low.** Free tier is mandatory for this persona.

Design implications. Aisha justifies the free tier and the educational framing of the transparency panel. Students become professionals who bring tools with them; the acquisition cost is near zero and the lifetime value is realized years later. An optional "Learning Mode" — expanded command explanations, a "why does this work?" link on every action — is a low-cost, high-goodwill feature.

4.6 Persona 5 — The Freelancer / Small Agency

Profile. Diego, 36, freelance web developer with eleven active clients. Manages roughly 25 servers across DigitalOcean, Hetzner, and various shared hosts. Moderate Linux skill. Bills hourly; every minute of server administration is a minute not billable to development.

Goals. Switch between client servers instantly with correct credentials. Perform identical maintenance across many hosts. Never confuse one client's server with another's. Deploy quickly.

Pain points. Credential sprawl across a password manager, `.ssh/config`, and scattered notes. Wasted time re-establishing context on each server. Genuine fear of running the right command on the wrong host.

Usage. Profile management with colored per-client grouping, deployment, backups, multi-host updates, quick monitoring checks.

Success criteria. Connects to any client server in under five seconds. Never runs a command on the wrong host. Reduces administration time by half.

Value. Direct revenue impact — administration time converts to billable time. **Willingness to pay: high relative to income.** Diego is likely the **best value-per-seat persona** in the entire set.

Design implications. Profile organization, visual host differentiation (color coding, environment badges, distinct window chrome for production), and a prominent "you are on PRODUCTION-CLIENT-A" indicator are disproportionately valuable. A wrong-host safeguard — requiring confirmation when a destructive action targets a host tagged production — is a small feature with outsized appeal.

4.7 Persona 6 — The Small Business Operator

Profile. Sandra, 48, operations manager at a 15-person logistics company. No IT staff. One Linux server runs their inventory application, installed years ago by a contractor who has since disappeared.

Goals. Keep the server running. Know when something is wrong before customers notice. Perform backups. Apply security updates. Avoid paying a consultant for trivial tasks.

Pain points. No Linux knowledge whatsoever. Terminal is genuinely intimidating. Consultant charges a minimum call-out fee for a five-minute restart. Backups may not have run in months and she has no way to check.

Usage. Dashboard health check, restart service, backup and verify, apply updates, disk space monitoring.

Success criteria. Answers "is the server healthy?" in ten seconds. Restarts the application safely. Confirms backups completed.

Value. Reduced consultant dependency and reduced downtime risk. **Willingness to pay: high** — the alternative is a consultant retainer costing multiples more.

Design implications. Sandra needs the simplest possible surface: a health dashboard with plain-language status, a small number of large, safe actions, and aggressive guardrails. She is served by a "Simple Mode" that hides most modules. She is **not** a Phase 1 priority — serving her well requires the rest of the product to be mature first — but she represents meaningful long-tail revenue and should not be designed out.

4.8 Persona 7 — The Enterprise IT Team

Profile. A 12-person infrastructure team at a regulated financial services firm. Mixed skill levels. Several hundred Linux hosts. Strict change control, mandatory audit trails, least-privilege policy, and a security team that must approve every tool.

Goals. Standardize administrative procedures. Enforce least privilege. Produce audit evidence for compliance review. Enable junior staff without granting broad root access. Reduce onboarding time.

Pain points. Shell access is difficult to audit meaningfully. Junior staff either receive excessive privilege or are blocked constantly. Compliance evidence is assembled manually. Tool approval takes months.

Usage. Centralized profile management, RBAC, complete audit logging, approval workflows for high-risk operations, SSO integration, on-premises licence server.

Success criteria. Passes security review. Produces exportable audit evidence. Reduces junior onboarding from weeks to days. No credential material leaves the corporate boundary.

Value. Compliance capability and risk reduction. **Willingness to pay: highest per seat — but with the longest sales cycle** (6–18 months) and the most demanding security requirements (SOC 2, penetration test results, SBOM, air-gapped operation).

Design implications. Enterprise requirements shape architecture from day one even though the segment is a Phase 4 target: the audit log must be tamper-evident and structured from the first release, credential storage must support enterprise key stores, and telemetry must be fully disableable. **Retrofitting these is far more expensive than building them in.** This is a specific instance of a general principle — architectural decisions that serve the latest-arriving persona must be made earliest.

4.9 Persona Priority and Feature Mapping

Feature Area	P1 Dev	P2 SysAdmin	P3 DevOps	P4 Student	P5 Freelance	P6 SMB	P7 Enterprise
File management	High	Medium	Low	High	High	Low	Medium
File editor	High	Medium	Low	Medium	High	Low	Medium
Deployment	**Critical**	Low	Low	Medium	**Critical**	Low	Medium
Monitoring	Medium	**Critical**	High	Low	Medium	**Critical**	High
Process management	Medium	**Critical**	High	Low	Medium	Low	High
Service management	High	**Critical**	Medium	Medium	High	Medium	High
Log viewer	**Critical**	**Critical**	High	Medium	High	Low	High
Package management	Medium	**Critical**	Low	Medium	High	Medium	High
User management	Low	**Critical**	Medium	Low	Low	Low	**Critical**
Network / firewall	Low	High	Medium	Low	Medium	Low	High
Docker	Medium	Medium	High	Low	Medium	Low	Medium
Database	Medium	Medium	Low	Low	High	Low	Medium
Cron	Low	High	Medium	Low	Medium	Low	Medium
Backup	Low	High	Low	Low	High	**Critical**	High
Certificates	Low	Medium	Low	Low	High	Low	High
Multi-host	Low	**Critical**	High	Low	**Critical**	Low	**Critical**
Audit log	Low	Medium	Low	Low	Low	Low	**Critical**
AI assistance	Medium	Low	Low	High	Medium	High	Low

Reading the matrix: the features marked Critical for the three P0 personas (Developer, SysAdmin, Freelancer) — deployment, monitoring, services, logs, packages, multi-host — constitute the Phase 1 and Phase 2 scope. This is not a coincidence; the roadmap in Section 12 is derived from this matrix.

5. Functional Requirements

5.0 Requirement Conventions

Identifier format: `FR-<MODULE>-<NNN>`

Priority levels:

Level	Meaning	Release
M	Must have — product is not viable without it	Phase 1–2
S	Should have — significant value, deferrable	Phase 2–3
C	Could have — desirable if capacity permits	Phase 3–4
W	Won't have this release — recorded for future	Phase 5+

Complexity: T-shirt sizing of engineering effort (XS ≤ 1 day, S ≤ 3 days, M ≤ 2 weeks, L ≤ 6 weeks, XL > 6 weeks) for one engineer.

Cross-cutting requirements applying to every module (stated once, not repeated per module):

ID	Requirement
FR-GEN-001	Every mutating operation SHALL display the exact command in the transparency panel before execution
FR-GEN-002	Every mutating operation SHALL be recorded in the audit log with timestamp, host, user, command, exit code, and duration
FR-GEN-003	Every long-running operation SHALL be cancellable and SHALL report progress
FR-GEN-004	Every operation SHALL surface stderr on failure, with plain-language interpretation where a known pattern matches
FR-GEN-005	Every module SHALL degrade gracefully when a required remote tool is absent, stating which tool is missing and how to install it
FR-GEN-006	Every module SHALL be fully operable by keyboard
FR-GEN-007	No module SHALL block the UI thread; all remote I/O SHALL be asynchronous
FR-GEN-008	Every destructive operation SHALL require confirmation proportionate to blast radius (see FR-SEC-020)

5.1 Connection Management

Purpose. Establish, authenticate, maintain, and organize SSH/SFTP connections to remote hosts. This module is the foundation of the entire product; every other module depends on it. Its reliability determines the perceived reliability of everything else.

5.1.1 SSH Connectivity

ID	Requirement	Pri	Cx
FR-CON-001	Establish SSH2 connections to a host by hostname or IP, IPv4 and IPv6	M	S
FR-CON-002	Support configurable port (default 22)	M	XS
FR-CON-003	Support connection timeout, configurable, default 15s	M	XS
FR-CON-004	Support keep-alive with configurable interval (default 30s) to survive NAT/idle timeouts	M	S
FR-CON-005	Support automatic reconnection with exponential backoff on transient failure	M	M
FR-CON-006	Support jump hosts / bastion chaining (`ProxyJump` semantics), minimum depth 3	S	M
FR-CON-007	Support SOCKS and HTTP proxy traversal	C	M
FR-CON-008	Support local, remote, and dynamic port forwarding with a management UI	S	M
FR-CON-009	Import existing `~/.ssh/config` including `Host`, `HostName`, `User`, `Port`, `IdentityFile`, `ProxyJump`	S	M
FR-CON-010	Multiplex multiple logical channels over a single TCP connection (see §10.3)	M	M
FR-CON-011	Support algorithm negotiation preferences with a documented, secure default set	S	S
FR-CON-012	Display negotiated cipher, MAC, KEX, and host key algorithm in connection details	C	XS
FR-CON-013	Support connecting through Teleport, Boundary, or similar access proxies via standard SSH semantics	C	M

5.1.2 Authentication

ID	Requirement	Pri	Cx
FR-CON-020	Password authentication with optional secure storage	M	S
FR-CON-021	Public key authentication: RSA, ECDSA, Ed25519	M	M
FR-CON-022	Encrypted private keys with passphrase prompt and optional session caching	M	M
FR-CON-023	Keyboard-interactive authentication (required for many MFA configurations)	M	M
FR-CON-024	Multi-factor authentication (TOTP prompt, push-based challenge)	S	M
FR-CON-025	Pageant / OpenSSH agent integration on Windows	S	M
FR-CON-026	Agent forwarding, disabled by default with an explicit security warning when enabled	S	S
FR-CON-027	Certificate-based authentication (OpenSSH certificates)	C	M

FR-CON-028	Sudo password prompt with optional per-session caching, cleared on disconnect	M	M
FR-CON-029	Support passwordless sudo detection to avoid unnecessary prompting	S	S
FR-CON-030	Support `su` as an alternative elevation path where sudo is unavailable	C	M

5.1.3 Host Key Verification

ID	Requirement	Pri	Cx
FR-CON-040	Verify host keys against a local known-hosts store on every connection	M	M
FR-CON-041	Present the fingerprint (SHA-256 and MD5) on first connection for explicit user acceptance	M	S
FR-CON-042	Block connection with a prominent, non-dismissible-by-default warning on host key mismatch	M	S
FR-CON-043	Import and optionally write back to OpenSSH `known_hosts`	S	S
FR-CON-044	Manage known hosts: view, search, remove entries	S	S
FR-CON-045	Support strict mode (organization policy) preventing acceptance of unknown host keys	C	S

Design note. FR-CON-042 must not be a routine "click OK" dialog. A host key change is either an infrastructure event the user knows about or an active attack. The dialog must require deliberate action and explain both possibilities in plain language. **The temptation to soften this for usability must be resisted** — this is the one place where friction is the feature.

5.1.4 Server Profiles

ID	Requirement	Pri	Cx
FR-CON-050	Create, edit, duplicate, and delete named server profiles	M	S
FR-CON-051	Profile fields: name, host, port, username, auth method, key path, initial directory, colour tag, environment tag, notes	M	S
FR-CON-052	Organize profiles into nested folders/groups	M	M
FR-CON-053	Tag profiles with free-form labels and filter by tag	S	S
FR-CON-054	Colour-code profiles; colour applies to window chrome and tab headers	M	S
FR-CON-055	Mark a profile as Production; apply distinct visual treatment and extra confirmation on destructive actions	M	M
FR-CON-056	Search profiles by any field, incrementally	M	S
FR-CON-057	Import profiles from PuTTY registry, WinSCP INI/registry, `~/.ssh/config`, and Termius export	S	L
FR-CON-058	Export profiles to an encrypted portable file, with a documented option to exclude secrets	S	M
FR-CON-059	Per-profile default settings: terminal encoding, sudo behaviour, editor preferences, deployment profile	S	M
FR-CON-060	Per-profile connection health indicator in the profile list (reachable / unreachable / unknown)	C	M

Design note (FR-CON-055). The production safeguard directly serves Persona 5 (Diego) and Persona 2 (Marcus). Implementation: production-tagged hosts render with a red-tinted title bar, a persistent banner, and require typed hostname confirmation for any recursive delete, service stop, or package removal. This is low engineering cost and disproportionately high perceived value.

5.1.5 Bookmarks and History

ID	Requirement	Pri	Cx
FR-CON-070	Bookmark remote directory paths per profile	M	S
FR-CON-071	Global bookmarks applying across profiles (e.g. `/var/log`)	S	S
FR-CON-072	Maintain connection history: host, user, timestamp, duration, disconnect reason	M	S
FR-CON-073	Reconnect from history with one action	M	XS
FR-CON-074	Maintain recently visited directories per host, capped and persisted	M	S
FR-CON-075	Maintain recently opened files per host with quick reopen	M	S
FR-CON-076	Clear history and bookmarks selectively or entirely	M	XS

5.1.6 Session Management

ID	Requirement	Pri	Cx
FR-CON-080	Maintain multiple concurrent sessions to different hosts	M	M
FR-CON-081	Maintain multiple concurrent sessions to the same host	S	S
FR-CON-082	Session switcher with keyboard navigation (Ctrl+1..9, Ctrl+Tab)	M	S
FR-CON-083	Detect and display session state: connecting, connected, degraded, reconnecting, disconnected	M	M
FR-CON-084	Restore the previous session set on application launch (opt-in)	S	M
FR-CON-085	Graceful disconnect with warning if operations are in flight	M	S
FR-CON-086	Per-session resource statistics: bytes transferred, commands executed, uptime	C	S
FR-CON-087	Broadcast a single operation to multiple selected sessions (multi-host execution)	S	L

5.1.7 Capability Detection

ID	Requirement	Pri	Cx

FR-CON-090	On connect, probe and cache: distribution and version, kernel version, architecture, init system, package manager, firewall tool, shell, sudo availability, SELinux/AppArmor state	M	M
FR-CON-091	Detect presence of optional tools: docker, podman, systemctl, journalctl, ss/netstat, lsof, tar, gzip, xz, zip, rsync, mysql, psql, sqlite3, openssl, certbot, crontab, git	M	M
FR-CON-092	Disable or annotate UI affordances for absent tools rather than failing at execution time	M	M
FR-CON-093	Complete the capability probe within 3 seconds on a typical connection	M	M
FR-CON-094	Cache the probe result per host with a configurable TTL; allow manual refresh	M	S
FR-CON-095	Display a Host Capabilities panel showing detected environment and available features	S	S

Design note. The capability probe is the mechanism that makes distribution neutrality possible and is therefore among the most architecturally significant components in the product. It runs as a single batched command sequence to minimize round trips. Getting this wrong produces the worst possible failure mode: an enabled button that fails at execution. FR-CON-092 is the requirement that prevents it.

5.2 File Management

Purpose. Complete graphical filesystem management, matching or exceeding Windows Explorer conventions while exposing Linux-specific concepts (permissions, ownership, symbolic links, special files) that Explorer has no equivalent for.

5.2.1 Browsing and Navigation

ID	Requirement	Pri	Cx
FR-FIL-001	Display directory contents in a virtualized table supporting ≥100,000 entries without UI degradation	M	L
FR-FIL-002	Columns: name, size, type, modified, permissions, owner, group, link target	M	S
FR-FIL-003	Configurable, reorderable, resizable columns persisted per user	M	S
FR-FIL-004	Sort by any column, ascending/descending, directories-first option	M	S
FR-FIL-005	View modes: details, list, large icons, tiles	S	M
FR-FIL-006	Breadcrumb path bar with click-to-navigate on each segment	M	S
FR-FIL-007	Editable path field accepting typed paths with tab-completion	M	M
FR-FIL-008	Tree view sidebar with lazy-loaded expansion	M	M
FR-FIL-009	Back, forward, up navigation with history stack	M	S
FR-FIL-010	Show/hide dotfiles, toggleable, persisted	M	XS
FR-FIL-011	Dual-pane mode: remote↔remote and local↔remote	M	L
FR-FIL-012	Tabbed browsing within a session	M	M
FR-FIL-013	File type icons derived from extension and MIME detection	M	M
FR-FIL-014	Inline preview pane: text, images, PDF metadata, archive contents	S	L
FR-FIL-015	Directory size calculation on demand (`du`), asynchronous with progress	M	M
FR-FIL-016	Free-space indicator for the current mount point	M	S
FR-FIL-017	Refresh: manual (F5) and optional automatic polling	M	S
FR-FIL-018	Display and navigate symbolic links, distinguishing them visually and indicating broken links	M	M
FR-FIL-019	Identify special file types: block/char devices, sockets, FIFOs	S	S
FR-FIL-020	Filter the current view by name pattern without a remote round trip	M	S

5.2.2 File Operations

ID	Requirement	Pri	Cx
FR-FIL-030	Create directory, with recursive parent creation option	M	XS
FR-FIL-031	Create empty file; create from template	M	S
FR-FIL-032	Rename in place (F2), with conflict detection	M	S
FR-FIL-033	Copy within remote host (server-side, no local round trip)	M	M
FR-FIL-034	Move/rename within remote host	M	S
FR-FIL-035	Delete with confirmation; recursive delete with typed confirmation	M	M
FR-FIL-036	Optional trash/recycle behaviour (move to a designated directory rather than unlink)	S	M
FR-FIL-037	Batch operations on multi-selection with per-item progress and per-item error reporting	M	L
FR-FIL-038	Duplicate file/directory in place	S	XS
FR-FIL-039	Create symbolic and hard links via dialog	S	S
FR-FIL-040	Touch (update timestamps)	C	XS

FR-FIL-041	Compute and display checksums: MD5, SHA-1, SHA-256	S	S
FR-FIL-042	Compare two remote files (checksum and content diff)	S	M
FR-FIL-043	Copy path, filename, or URL to clipboard	M	XS
FR-FIL-044	Open file with associated local application (download to temp, launch, watch for changes, re-upload)	M	L
FR-FIL-045	Conflict resolution dialog on overwrite: overwrite, skip, rename, compare, apply-to-all	M	M

5.2.3 Transfer (Upload / Download)

ID	Requirement	Pri	Cx
FR-FIL-050	Upload files and directories recursively via SFTP	M	M
FR-FIL-051	Download files and directories recursively via SFTP	M	M
FR-FIL-052	OS-native drag and drop: Explorer→app, app→Explorer, pane→pane	M	L
FR-FIL-053	Transfer queue with pause, resume, cancel, reorder, and priority	M	L
FR-FIL-054	Per-file and aggregate progress: percentage, rate, ETA, bytes transferred	M	M
FR-FIL-055	Configurable parallel transfer streams (default 4, max 16)	M	L
FR-FIL-056	Resume interrupted transfers where the server supports it	M	L
FR-FIL-057	Preserve timestamps and permissions on transfer, optionally	M	S
FR-FIL-058	Bandwidth throttling, global and per-transfer	S	M
FR-FIL-059	Transfer integrity verification via checksum comparison, optional	S	M
FR-FIL-060	Automatic retry on transient failure, configurable attempts and backoff	M	M
FR-FIL-061	Directory synchronization: local→remote, remote→local, bidirectional, with preview before execution	M	XL
FR-FIL-062	Transfer history with re-run capability	S	S
FR-FIL-063	Handle filename encoding differences (UTF-8, Latin-1) and illegal-on-Windows characters (`:` , `?` , `*` , `<` , `>` , `|` , `"`) with a documented substitution scheme	M	M
FR-FIL-064	Warn and handle case-sensitivity collisions when downloading to Windows	M	M
FR-FIL-065	Warn on Windows path length limits (>260 chars) and offer long-path handling	S	S

Design note. FR-FIL-063 through FR-FIL-065 address a class of defect that is invisible in testing on well-behaved filesystems and highly visible in production. A Linux directory may legitimately contain `report:2024.txt` and both `README` and `readme`. Windows accepts neither. These must be handled explicitly and transparently rather than failing opaquely mid-transfer — a partial directory download that silently skipped files is worse than one that failed loudly.

5.2.4 Search

ID	Requirement	Pri	Cx
FR-FIL-070	Search by filename, glob pattern, and regular expression	M	M
FR-FIL-071	Search file contents (grep) with regex support	M	M
FR-FIL-072	Filter by size range, modification date range, owner, group, permissions, file type	M	M
FR-FIL-073	Configurable search depth and directory exclusions	M	S
FR-FIL-074	Stream results progressively as found; do not block on completion	M	M
FR-FIL-075	Cancel a running search cleanly, terminating the remote process	M	M
FR-FIL-076	Act on search results directly (open, delete, download, reveal in browser)	M	S
FR-FIL-077	Save named searches for reuse	C	S
FR-FIL-078	Export search results to CSV	C	XS

5.2.5 Permissions and Ownership

ID	Requirement	Pri	Cx
FR-FIL-080	Display permissions in both symbolic (`rwxr-xr-x`) and octal (`755`) form	M	S
FR-FIL-081	Permission editor with a checkbox matrix (user/group/other × read/write/execute)	M	M

FR-FIL-082	Direct octal entry with live symbolic preview and validation	M	S
FR-FIL-083	Special bits: setuid, setgid, sticky — with explanatory tooltips	M	S
FR-FIL-084	Recursive permission application with separate file and directory masks	M	M
FR-FIL-085	Warn prominently on world-writable and setuid-on-executable changes	M	S
FR-FIL-086	Change owner and group, with user/group pickers populated from the remote host	M	M
FR-FIL-087	Recursive ownership change	M	S
FR-FIL-088	Display and edit POSIX ACLs where supported (`getfacl` / `setfacl`)	C	L
FR-FIL-089	Display SELinux context where present; editing deferred	C	M
FR-FIL-090	Display extended attributes	C	M
FR-FIL-091	Preview the effect of a recursive permission change (affected file count) before applying	S	M

Design note. FR-FIL-091 is a direct expression of Principle P4. A recursive `chmod` on a large tree is irreversible and commonly destructive when done wrong. Showing "this will modify 14,203 files across 892 directories" before execution converts a dangerous operation into an informed one at trivial engineering cost.

5.2.6 Compression and Archives

ID	Requirement	Pri	Cx
FR-FIL-100	Create archives: tar, tar.gz, tar.bz2, tar.xz, zip	M	M
FR-FIL-101	Extract archives with destination selection and overwrite policy	M	M
FR-FIL-102	Browse archive contents without extracting	S	L
FR-FIL-103	Extract selected entries from an archive	S	M
FR-FIL-104	Add files to an existing archive where the format permits	C	M
FR-FIL-105	Compression level selection	S	XS
FR-FIL-106	Password-protected zip creation and extraction	C	M
FR-FIL-107	Progress reporting for large archive operations	M	M
FR-FIL-108	Warn on archive extraction paths containing `..` or absolute paths (path traversal / zip-slip)	M	S

5.2.7 Drag and Drop

ID	Requirement	Pri	Cx
FR-FIL-110	Drag from Windows Explorer into a remote directory → upload	M	L
FR-FIL-111	Drag from remote pane to Windows Explorer → download	M	XL
FR-FIL-112	Drag between remote panes → move (same host) or transfer (different hosts)	M	L
FR-FIL-113	Modifier keys: Ctrl = copy, Shift = move, Alt = create link	M	M
FR-FIL-114	Visual drop-target feedback and invalid-target indication	M	M
FR-FIL-115	Drop onto a directory row targets that directory, not the current view	M	S
FR-FIL-116	Drag files into the editor to open them	S	S

Implementation note (FR-FIL-111). Dragging *out* to Explorer requires the file to exist locally at drop time, but the drop target is unknown until the drop occurs. The practical approach is a virtual file promise with deferred materialization via `CFSTR_FILEDESCRIPTOR` / `CFSTR_FILECONTENTS`, requiring native Windows shell interop (JNA or a small JNI shim). **This is the single hardest UI feature in the product and should be scheduled with generous contingency.** A viable fallback is to materialize into a temporary directory and provide standard file paths, accepting a copy on drop.

5.3 File Editor

Purpose. Edit remote text files — primarily configuration files and scripts — without a download/edit/upload cycle. The editor must be competent, not comprehensive; VS Code integration handles serious code editing.

5.3.1 Core Editing

ID	Requirement	Pri	Cx
FR-EDT-001	Open remote text files with automatic encoding detection (UTF-8, UTF-16, Latin-1, Windows-1252)	M	M
FR-EDT-002	Detect and preserve line endings (LF, CRLF, CR); warn on mixed	M	S
FR-EDT-003	Multiple files open in tabs	M	S
FR-EDT-004	Modified-state indicator per tab; warn on close with unsaved changes	M	S
FR-EDT-005	Save to remote with atomic write (temp file + rename) to prevent truncation on failure	M	M
FR-EDT-006	Save As to a different remote path	M	XS
FR-EDT-007	Read-only mode when write permission is absent, clearly indicated	M	S
FR-EDT-008	Offer sudo-elevated save when permission is denied	M	M

FR-EDT-009	Large file handling: warn above 10 MB, stream or refuse above 100 MB with a clear explanation	M	M
FR-EDT-010	Binary file detection with a refusal to open in text mode	M	S
FR-EDT-011	Preserve original file permissions and ownership on save	M	M
FR-EDT-012	Preserve SELinux context on save where applicable	C	M

5.3.2 Editing Features

ID	Requirement	Pri	Cx
FR-EDT-020	Syntax highlighting: bash, Python, JavaScript, JSON, YAML, XML, HTML, CSS, SQL, Java, Go, Rust, PHP, Ruby, Dockerfile, nginx.conf, Apache conf, systemd unit, INI, TOML, Markdown, Properties, Makefile	M	L
FR-EDT-021	Language detection by extension, shebang, and filename convention	M	M
FR-EDT-022	Line numbers, current-line highlight, and column ruler	M	S
FR-EDT-023	Code folding for brace-, indent-, and tag-delimited structures	S	M
FR-EDT-024	Bracket matching and auto-closing	S	S
FR-EDT-025	Auto-indent, configurable tab width, tabs-vs-spaces	M	S
FR-EDT-026	Multi-cursor and column/block selection	S	L
FR-EDT-027	Unlimited undo/redo within a session	M	M
FR-EDT-028	Find with regex, case sensitivity, whole word, and match count	M	M
FR-EDT-029	Replace and replace-all with preview of affected lines	M	M
FR-EDT-030	Find across open tabs	S	M
FR-EDT-031	Go to line	M	XS
FR-EDT-032	Word wrap toggle	M	XS
FR-EDT-033	Whitespace and EOL character visualization	S	S
FR-EDT-034	Convert line endings, encoding, and indentation style	S	S
FR-EDT-035	Zoom / font size adjustment	M	XS
FR-EDT-036	Configurable colour themes, dark and light	M	M
FR-EDT-037	Syntax validation for JSON, YAML, XML with inline error markers	S	M
FR-EDT-038	Configuration file validation via native tools where available (`nginx -t`, `apachectl configtest`, `sshd -t`, `visudo -c`)	S	M

Design note (FR-EDT-038). This is a high-value, low-cost feature and a strong differentiator. Editing `sshd_config` incorrectly and restarting the service can lock the operator out of the host permanently. Validating before saving — and refusing to restart SSH with an invalid config without explicit override — prevents a genuinely catastrophic and genuinely common failure. The same applies to `sudoers`, where a syntax error can render the system unadministrable.

5.3.3 Auto-Save, History, and Comparison

ID	Requirement	Pri	Cx
FR-EDT-040	Local draft auto-save at a configurable interval, surviving application crash	M	M
FR-EDT-041	Optional remote auto-save, off by default, with explicit warning	S	S
FR-EDT-042	Automatic backup of the remote file before first save in a session, retained locally	M	M
FR-EDT-043	Local version history per file with timestamps	M	L
FR-EDT-044	Restore any historical version	M	M
FR-EDT-045	Side-by-side diff: current vs. any historical version	M	L
FR-EDT-046	Diff local file vs. remote file	M	L
FR-EDT-047	Diff two arbitrary remote files	S	M
FR-EDT-048	Detect external modification of the open file and prompt for reload/merge/overwrite	M	M
FR-EDT-049	Three-way merge on conflict	C	XL
FR-EDT-050	Configurable history retention (count and age)	S	S

5.3.4 Integration

ID	Requirement	Pri	Cx
FR-EDT-060	Open the current file in an external local editor with change watching and automatic re-upload	M	L
FR-EDT-061	"Open folder in VS Code Remote SSH" action from the file browser	C	S
FR-EDT-062	Insert snippets from a user-managed library	C	M
FR-EDT-063	Templates for common configuration files (systemd unit, nginx server block, Dockerfile, cron entry)	S	M

5.4 Deployment

Purpose. Move an application from a local working directory to a remote server safely and repeatably, with verification and a reliable path back. This is the module most likely to drive purchase decisions for Personas 1 and 5 and should receive disproportionate design attention.

5.4.1 Comparison and Analysis

ID	Requirement	Pri	Cx
FR-DEP-001	Compare a local directory tree against a remote directory tree	M	L
FR-DEP-002	Classify each entry: identical, local-only, remote-only, modified, type-conflict	M	M
FR-DEP-003	Comparison criteria: size, modification time, checksum (configurable, checksum optional for speed)	M	M
FR-DEP-004	Present differences in a reviewable tree with per-file diff on demand	M	L
FR-DEP-005	Estimate transfer size and duration before execution	M	S
FR-DEP-006	Complete comparison of a 10,000-file tree in under 30 seconds	M	L
FR-DEP-007	Cache comparison state to accelerate repeat comparisons	S	M

5.4.2 Deployment Execution

ID	Requirement	Pri	Cx
FR-DEP-010	Deploy only changed files; never transfer unchanged content	M	M
FR-DEP-011	Per-file selection: include or exclude any file from the deployment set	M	M
FR-DEP-012	Ignore patterns (`.gitignore` syntax) with import from an existing `.gitignore`	M	M
FR-DEP-013	Default ignore set: `.git`, `node_modules`, `__pycache__`, `.env`, `*.log`, `.DS_Store`, `target/`, `build/`, `dist/`, `.idea`, `.vscode`	M	XS
FR-DEP-014	Delete remote files absent locally, opt-in with explicit confirmation and file listing	M	M
FR-DEP-015	Preserve or set specific permissions on deployed files	M	S
FR-DEP-016	Pre-deployment hooks: arbitrary remote commands executed in order, halting on failure	M	M
FR-DEP-017	Post-deployment hooks	M	M
FR-DEP-018	Deploy to a staging directory then atomically swap via symlink (zero-downtime pattern)	S	L
FR-DEP-019	Halt deployment on first error with a clear report of completed and pending operations	M	M
FR-DEP-020	Dry-run mode producing a full action report without mutating the remote host	M	M

5.4.3 Backup and Rollback

ID	Requirement	Pri	Cx
FR-DEP-030	Automatic backup of affected remote files before deployment	M	L
FR-DEP-031	Backup strategies: archive to remote path, copy to timestamped directory, download locally	M	M
FR-DEP-032	Configurable backup retention (count and age) with automatic pruning	M	M
FR-DEP-033	One-click rollback to the immediately previous deployment	M	L
FR-DEP-034	Rollback to any retained prior deployment	M	M
FR-DEP-035	Rollback executes post-rollback hooks (typically a service restart)	M	S
FR-DEP-036	Verify backup integrity before proceeding with deployment	M	M
FR-DEP-037	Warn and require explicit confirmation if backup is disabled	M	XS
FR-DEP-038	Estimate and display backup storage consumption	S	S

Design note. FR-DEP-030 and FR-DEP-033 together constitute the product's most compelling single feature for the developer persona. A deployment that cannot be undone is a deployment made nervously. Rollback must be genuinely one click, must complete in seconds, and must be tested exhaustively — **a rollback that fails is worse than no rollback feature at all**, because the user will have relied on it.

5.4.4 Deployment Profiles and History

ID	Requirement	Pri	Cx
FR-DEP-040	Named deployment profiles: local path, remote path, target host, ignore rules, hooks, backup policy, health check	M	M

FR-DEP-041	Multiple profiles per project (development, staging, production)	M	S
FR-DEP-042	Profile-level confirmation requirements (e.g. production requires typed confirmation)	M	S
FR-DEP-043	Export and import profiles as version-controllable files	S	M
FR-DEP-044	Deployment history: timestamp, profile, operator, file count, bytes, duration, outcome, hook results	M	M
FR-DEP-045	Full file manifest per historical deployment	M	M
FR-DEP-046	Compare any two historical deployments	C	M
FR-DEP-047	Repeat a historical deployment	S	S
FR-DEP-048	Export deployment history as CSV or JSON for reporting	C	XS

5.4.5 Health Check and Service Control

ID	Requirement	Pri	Cx
FR-DEP-050	Post-deployment health check: HTTP request with expected status code and optional body match	M	M
FR-DEP-051	Health check: TCP port reachability	M	S
FR-DEP-052	Health check: process presence	M	S
FR-DEP-053	Health check: systemd unit active state	M	S
FR-DEP-054	Health check: arbitrary command with expected exit code	M	S
FR-DEP-055	Configurable retry count, interval, and overall timeout	M	S
FR-DEP-056	Automatic rollback on health check failure, opt-in	S	M
FR-DEP-057	Restart, reload, stop, or start a service as a deployment step	M	S
FR-DEP-058	Prefer reload over restart where the unit supports it, to avoid downtime	S	S
FR-DEP-059	Display service status and recent journal entries after the deployment step	M	M
FR-DEP-060	Deployment summary report: what changed, what ran, what the health check returned	M	M

Design note (FR-DEP-056). Automatic rollback on health check failure is powerful and dangerous. If the health check is misconfigured, it will roll back healthy deployments. It must be opt-in, must be clearly explained at configuration time, and must be tested by the user in dry-run before being trusted in production.

5.5 Monitoring

Purpose. Answer "is this server healthy, and if not, why?" at a glance, using only data obtainable over SSH without an installed agent.

5.5.1 Metric Collection

ID	Requirement	Pri	Cx
FR-MON-001	Collect metrics via a single batched command per poll cycle to minimize round trips	M	L
FR-MON-002	Configurable poll interval (1s–5min, default 5s), per host	M	S
FR-MON-003	Pause polling when the module is not visible, to conserve bandwidth and remote CPU	M	M
FR-MON-004	Poll cycle overhead must not exceed 1% CPU on the remote host	M	L
FR-MON-005	Retain in-memory time series for the session, configurable window (default 1 hour)	M	M
FR-MON-006	Optional local persistence of metric history for longer-term comparison	S	L
FR-MON-007	Degrade gracefully when a metric source is unavailable rather than failing the whole poll	M	M

5.5.2 CPU

ID	Requirement	Pri	Cx
FR-MON-010	Overall CPU utilization percentage, real-time	M	M
FR-MON-011	Per-core utilization with individual visualization	M	M
FR-MON-012	Breakdown: user, system, nice, iowait, irq, softirq, steal, idle	M	M
FR-MON-013	Highlight elevated iowait and steal with plain-language interpretation	S	S
FR-MON-014	Historical CPU chart over the retention window	M	M
FR-MON-015	CPU model, core count, thread count, base frequency, current frequency	M	S
FR-MON-016	Load average (1, 5, 15 min) with core-count-relative interpretation	M	S
FR-MON-017	Top CPU-consuming processes shown alongside the CPU view	M	S
FR-MON-018	CPU temperature where sensors are available	C	M
FR-MON-019	Configurable threshold alerts	S	M

Design note (FR-MON-013, FR-MON-016). Raw numbers are not insight. A load average of 8.0 is alarming on a 2-core machine and

unremarkable on a 32-core one; the UI must present it relative to core count. High `steal` on a virtualized host indicates a noisy neighbour, not a local problem, and telling the user so saves hours of misdirected investigation. This interpretive layer is a direct expression of Principle P6 and is where the product earns its value over `top`.

5.5.3 Memory

ID	Requirement	Pri	Cx
FR-MON-020	Total, used, free, available, buffers, cached, shared	M	S
FR-MON-021	Distinguish "used" from "available" and explain the difference in the UI	M	S
FR-MON-022	Swap total, used, free, and swap activity rate	M	S
FR-MON-023	Historical memory chart	M	M
FR-MON-024	Top memory-consuming processes	M	S
FR-MON-025	Detect and surface OOM-killer events from the kernel log	S	M
FR-MON-026	Threshold alerts	S	M

Design note (FR-MON-021). "Linux ate my RAM" is among the most common misunderstandings among newcomers. The cached-memory-counted-as-used confusion causes unnecessary alarm and misguided remediation. Presenting *available* memory as the primary figure, with buffers/cache shown separately and explained, is a small UI decision that eliminates a whole class of user confusion.

5.5.4 Storage

ID	Requirement	Pri	Cx
FR-MON-030	Filesystem usage per mount point: size, used, available, percentage, type	M	S
FR-MON-031	Inode usage per mount point	M	S
FR-MON-032	Visual capacity indicators with warning and critical thresholds	M	S
FR-MON-033	Historical usage trend with time-to-full projection	S	M
FR-MON-034	Directory size analysis (largest consumers) with drill-down	M	L
FR-MON-035	Identify large files above a size threshold	M	M
FR-MON-036	Identify recently grown directories	C	M
FR-MON-037	Block device listing: devices, partitions, mount points, filesystem types	M	M
FR-MON-038	LVM information where present: physical volumes, volume groups, logical volumes	C	M
FR-MON-039	RAID status where present (`/proc/mdstat`)	C	M
FR-MON-040	SMART health status where `smartctl` is available	C	M

Design note (FR-MON-031). Inode exhaustion produces "No space left on device" while `df -h` shows free space — a confusing and reasonably common failure, particularly on mail spools and session directories. Surfacing inode usage alongside block usage resolves it immediately.

5.5.5 Network

ID	Requirement	Pri	Cx
FR-MON-050	Interface list with state, addresses, MAC, MTU, speed	M	M
FR-MON-051	Per-interface throughput (bytes/sec in and out), real-time	M	M
FR-MON-052	Packet counts, errors, drops, collisions	M	S
FR-MON-053	Historical bandwidth chart	M	M
FR-MON-054	Active connection list: protocol, local, remote, state, owning process	M	M
FR-MON-055	Listening port list with owning process and service identification	M	M
FR-MON-056	Connection count by state (ESTABLISHED, TIME_WAIT, etc.)	S	S
FR-MON-057	Reverse DNS resolution for remote addresses, optional	C	S
FR-MON-058	Bandwidth by process where `nethogs` or equivalent is available	C	M

5.5.6 Disk I/O, GPU, and System Information

ID	Requirement	Pri	Cx
FR-MON-060	Per-device read/write throughput and IOPS	M	M
FR-MON-061	I/O wait percentage and device utilization	M	M
FR-MON-062	Per-process I/O where permissions allow	S	M
FR-MON-063	Historical I/O chart	M	M
FR-MON-064	GPU utilization, memory, and temperature via `nvidia-smi` where present	C	M
FR-MON-065	AMD and Intel GPU support where tooling permits	W	L
FR-MON-066	System uptime and boot time	M	XS
FR-MON-067	Hostname, FQDN, distribution, kernel version, architecture, virtualization type	M	S

FR-MON-068	Logged-in users and their sessions	M	S
FR-MON-069	Hardware summary: CPU, memory modules, disks, network adapters	S	M
FR-MON-070	Timezone, NTP synchronization status, and clock drift	S	S
FR-MON-071	Pending reboot indicator (`/var/run/reboot-required` , kernel version mismatch)	S	S

5.5.7 Dashboard

ID	Requirement	Pri	Cx
FR-MON-080	Single-screen health overview: CPU, memory, disk, network, load, uptime, service status	M	L
FR-MON-081	Overall health indicator (healthy / degraded / critical) derived from configurable rules	M	M
FR-MON-082	Customizable widget layout with drag-to-arrange	S	L
FR-MON-083	Multi-host overview showing all connected hosts in one view	M	L
FR-MON-084	Alert banner for threshold breaches with a link to the relevant module	M	M
FR-MON-085	Export current metric snapshot as a report	C	S
FR-MON-086	"What changed recently" panel: recent deployments, service restarts, package installs	S	M

5.6 Process Management

Purpose. A Windows Task Manager equivalent: see what is running, what it is consuming, and act on it.

ID	Requirement	Pri	Cx
FR-PRC-001	List all processes: PID, PPID, user, command, %CPU, %MEM, RSS, VSZ, state, start time, elapsed, TTY, nice	M	M
FR-PRC-002	Refresh at a configurable interval with a pause control	M	S
FR-PRC-003	Sort by any column	M	S
FR-PRC-004	Incremental search by name, command line, PID, or user	M	S
FR-PRC-005	Filter by user, state, or resource threshold	M	S
FR-PRC-006	Tree view showing the parent-child hierarchy	M	M
FR-PRC-007	Toggle between flat and tree views	M	S
FR-PRC-008	Process detail panel: full command line, working directory, environment, open files, threads, limits	M	L
FR-PRC-009	Display open file descriptors and network sockets per process	S	M
FR-PRC-010	Display thread count and per-thread detail	C	M
FR-PRC-011	Terminate a process (SIGTERM) with confirmation	M	S
FR-PRC-012	Force kill (SIGKILL) with elevated confirmation	M	S
FR-PRC-013	Send an arbitrary signal from a documented list	S	S
FR-PRC-014	Terminate a process tree	S	M
FR-PRC-015	Warn prominently before terminating critical processes (PID 1, sshd, the current session, kernel threads)	M	M
FR-PRC-016	Change process priority (renice), including negative values with elevation	M	S
FR-PRC-017	Change I/O priority (ionice) where available	C	S
FR-PRC-018	Restart a process by identifying and restarting its owning systemd unit	S	M
FR-PRC-019	Highlight processes exceeding configurable resource thresholds	S	S
FR-PRC-020	Show total process and thread counts, and count by state	S	XS
FR-PRC-021	Identify zombie processes and explain their significance	S	S
FR-PRC-022	Multi-select and batch signal	S	S
FR-PRC-023	Copy process details to clipboard	M	XS
FR-PRC-024	Correlate a process with its systemd unit, container, or cgroup where applicable	S	M

Design note (FR-PRC-015). Killing `sshd` disconnects the operator and may render the host unreachable. Killing PID 1 is catastrophic. Killing the session's own shell terminates the connection. These require a hard warning explaining the specific consequence — not a generic "are you sure?". This is a case where the GUI can be materially *safer* than the shell, which offers no such protection.

5.7 Service Management

Purpose. Manage system services — primarily systemd, with SysV fallback — as a Windows Services console equivalent.

5.7.1 Service Listing and Control

ID	Requirement	Pri	Cx
FR-SVC-001	List units: name, description, load state, active state, sub-state, enabled state	M	M
FR-SVC-002	Filter by unit type (service, socket, timer, target, mount, path)	M	S

FR-SVC-003	Filter by state: running, stopped, failed, enabled, disabled, masked	M	S
FR-SVC-004	Incremental search by name and description	M	S
FR-SVC-005	Prominently surface failed units	M	S
FR-SVC-006	Start a service	M	XS
FR-SVC-007	Stop a service, with warning for critical units	M	S
FR-SVC-008	Restart a service	M	XS
FR-SVC-009	Reload configuration without restart, where supported	M	S
FR-SVC-010	Enable / disable at boot	M	XS
FR-SVC-011	Mask / unmask, with explanation of the distinction from disable	S	S
FR-SVC-012	Verify state after any control action and report the actual outcome	M	M
FR-SVC-013	Batch operations on multi-selection	S	S
FR-SVC-014	Guard against stopping the SSH service, requiring explicit typed confirmation	M	S

5.7.2 Service Detail and Configuration

ID	Requirement	Pri	Cx
FR-SVC-020	Detail view: description, unit file path, main PID, memory, CPU time, tasks, cgroup	M	M
FR-SVC-021	Display recent journal entries for the unit	M	M
FR-SVC-022	Display dependencies (Requires, Wants, After, Before)	M	M
FR-SVC-023	Visualize the dependency graph	C	L
FR-SVC-024	Display reverse dependencies (what depends on this unit)	S	M
FR-SVC-025	View the unit file contents	M	S
FR-SVC-026	Edit the unit file with validation and automatic `daemon-reload`	S	M
FR-SVC-027	Create a drop-in override rather than editing the packaged unit	S	M
FR-SVC-028	Create a new service unit from a guided template	S	M
FR-SVC-029	Display and edit unit environment variables	S	M
FR-SVC-030	Show restart count and last failure reason	S	S
FR-SVC-031	Systemd timer listing with next and last trigger times	S	M
FR-SVC-032	Fall back to SysV init (`service` , `chkconfig`) on hosts without systemd	S	L

Design note (FR-SVC-027). Editing a distribution-packaged unit file directly means the change is lost on package upgrade. Drop-in overrides in `/etc/systemd/system/<unit>.d/` survive upgrades. Defaulting to the correct pattern — and explaining why — is exactly the kind of embedded expertise that justifies the product's existence.

5.8 Log Viewer

Purpose. Read, search, and follow logs from journald and plain files, with the responsiveness of a native application.

ID	Requirement	Pri	Cx
FR-LOG-001	Browse and open journald logs via `journalctl`	M	M
FR-LOG-002	Browse and open plain log files under `/var/log` and arbitrary paths	M	S
FR-LOG-003	Live tail with automatic scroll and a scroll-lock toggle	M	M
FR-LOG-004	Handle log rotation transparently during a live tail	M	M
FR-LOG-005	Load large files efficiently: tail-first, page backwards on demand, never load whole file	M	L
FR-LOG-006	Search within loaded content with regex support and match navigation	M	M
FR-LOG-007	Server-side search across full file history (grep) for files too large to load	M	M
FR-LOG-008	Filter by severity for structured sources (journald priority)	M	M
FR-LOG-009	Filter by time range with a date-time picker	M	M
FR-LOG-010	Filter by unit, process, or user for journald	M	M
FR-LOG-011	Automatic severity highlighting: error, warning, critical, and stack traces	M	M
FR-LOG-012	Custom user-defined highlight rules with colours	S	M
FR-LOG-013	Detect and collapse repeated lines with a repeat count	S	M
FR-LOG-014	Multi-line entry handling (stack traces, multi-line JSON)	M	M
FR-LOG-015	Parse and pretty-print structured JSON log lines	S	M
FR-LOG-016	Multiple log tabs, from multiple hosts, side by side	M	M
FR-LOG-017	Merged view of multiple sources sorted by timestamp	C	L
FR-LOG-018	Download the current log, filtered selection, or full file	M	S
FR-LOG-019	Copy selected lines to clipboard	M	XS
FR-LOG-020	Bookmark log positions	C	S

FR-LOG-021	Jump from a log entry to the related service or process	S	M
FR-LOG-022	Display log file size and rotation configuration	S	S
FR-LOG-023	Manage journald disk usage (`journalctl --vacuum`)	C	S
FR-LOG-024	Error frequency histogram over the visible time range	C	M

Design note (FR-LOG-005). Log files reaching gigabytes are routine. Loading one entirely into memory is the most likely cause of an out-of-memory crash in the product. The viewer must be built as a windowed, streaming reader from the outset — retrofitting this is a rewrite of the module. This constraint should be treated as an architectural requirement, not an optimization.

5.9 Docker and Container Management

Purpose. Manage Docker on the remote host by wrapping the Docker CLI over SSH. Depth comparable to Portainer is not the goal; contextual integration with host management is.

5.9.1 Containers

ID	Requirement	Pri	Cx
FR-DOC-001	List containers: name, image, status, ports, created, CPU, memory	M	M
FR-DOC-002	Filter running / stopped / all	M	XS
FR-DOC-003	Start, stop, restart, pause, unpause	M	S
FR-DOC-004	Remove container, with force option and confirmation	M	S
FR-DOC-005	View container logs with live follow and search	M	M
FR-DOC-006	Inspect container (formatted JSON with a readable summary view)	M	M
FR-DOC-007	Real-time container resource statistics	M	M
FR-DOC-008	Execute a command in a container	S	M
FR-DOC-009	Interactive shell into a container	S	L
FR-DOC-010	Copy files to and from a container	S	M
FR-DOC-011	Create a container via a guided form (image, ports, volumes, environment, restart policy, network)	S	L
FR-DOC-012	Show and edit container port mappings via recreation	C	M
FR-DOC-013	Rename a container	C	XS
FR-DOC-014	Container health status where a healthcheck is defined	S	S

5.9.2 Images, Volumes, Networks, Compose

ID	Requirement	Pri	Cx
FR-DOC-020	List images: repository, tag, ID, created, size	M	S
FR-DOC-021	Pull an image with progress reporting	M	M
FR-DOC-022	Remove images with dangling-image identification	M	S
FR-DOC-023	Display image layer history	C	S
FR-DOC-024	Build an image from a remote Dockerfile with streamed output	C	M
FR-DOC-025	Prune unused images, containers, volumes, networks with a preview of reclaimed space	M	M
FR-DOC-026	List volumes with size and mount points	M	S
FR-DOC-027	Create and remove volumes	S	S
FR-DOC-028	Browse volume contents via the host filesystem path	S	M
FR-DOC-029	List networks with driver, scope, and connected containers	M	S
FR-DOC-030	Create and remove networks	S	S
FR-DOC-031	Connect and disconnect containers from networks	C	S
FR-DOC-032	Detect and list Docker Compose projects	S	M
FR-DOC-033	Compose up, down, restart, and pull with streamed output	S	M
FR-DOC-034	View and edit `docker-compose.yml` in the integrated editor with validation	S	M
FR-DOC-035	Show per-service status within a Compose project	S	M
FR-DOC-036	Display Docker daemon information and disk usage	S	S
FR-DOC-037	Podman support via CLI compatibility	C	M

5.10 Database Management

Purpose. Basic administration and querying of databases resident on the managed host, accessed through their command-line clients over SSH. Not a replacement for a dedicated database IDE.

ID	Requirement	Pri	Cx
FR-DBM-001	Detect installed database engines: MySQL/MariaDB, PostgreSQL, SQLite, MongoDB, Redis	M	M
FR-DBM-002	Connect via local socket or TCP using the host's CLI client	M	M
FR-DBM-003	Store database credentials in the profile's encrypted credential store	M	M
FR-DBM-004	List databases with size and encoding	M	M
FR-DBM-005	List tables with row count estimate, size, and engine	M	M
FR-DBM-006	View table schema: columns, types, keys, indexes, constraints	M	M
FR-DBM-007	Browse table data with pagination and sorting	M	L
FR-DBM-008	Execute arbitrary SQL with a results grid	M	L
FR-DBM-009	SQL editor with syntax highlighting and history	M	M
FR-DBM-010	Distinguish read from write statements; require confirmation for writes	M	M
FR-DBM-011	Warn on `UPDATE` / `DELETE` without a `WHERE` clause	M	S
FR-DBM-012	Display affected row count and execution time	M	S
FR-DBM-013	Export results as CSV, JSON, or SQL insert statements	M	M
FR-DBM-014	Export a database dump (`mysqldump` , `pg_dump`) with options	M	M
FR-DBM-015	Import a dump file with progress and error reporting	M	M
FR-DBM-016	Create and drop databases	S	S
FR-DBM-017	Manage database users and grants	S	M
FR-DBM-018	Display active connections and running queries	S	M
FR-DBM-019	Terminate a running query or connection	S	S
FR-DBM-020	Display server status variables and configuration	C	M
FR-DBM-021	Basic query plan display (`EXPLAIN`)	C	M
FR-DBM-022	SQLite file browsing directly from the file manager	S	M
FR-DBM-023	Redis key browsing and basic operations	C	M

Design note (FR-DBM-011). `DELETE FROM users;` executes without complaint in every database CLI. A warning dialog identifying a missing `WHERE` clause costs almost nothing to implement and prevents a category of incident that ends careers. This is the clearest possible illustration of the product's safety thesis.

5.11 User and Permission Management

ID	Requirement	Pri	Cx
FR-USR-001	List users: username, UID, GID, home, shell, comment, last login, lock status	M	M
FR-USR-002	Distinguish system users from regular users, with filtering	M	S
FR-USR-003	Create a user: username, UID, home, shell, primary and supplementary groups, comment	M	M
FR-USR-004	Modify user attributes	M	M
FR-USR-005	Delete a user with options to remove or retain the home directory	M	S
FR-USR-006	Lock and unlock accounts	M	S
FR-USR-007	Set and reset passwords	M	M
FR-USR-008	Password policy: expiry, minimum age, warning period	S	M
FR-USR-009	Force password change at next login	S	S
FR-USR-010	List groups: name, GID, members	M	S
FR-USR-011	Create, modify, and delete groups	M	S
FR-USR-012	Add and remove group members	M	S
FR-USR-013	Display effective group membership per user	M	S
FR-USR-014	Grant and revoke sudo privileges via group membership or sudoers	M	M
FR-USR-015	View sudoers configuration with syntax awareness	S	M
FR-USR-016	Edit sudoers safely with `visudo -c` validation before commit	S	M
FR-USR-017	List authorized SSH keys per user with fingerprint, type, and comment	M	M
FR-USR-018	Add an authorized key by paste or file selection	M	M
FR-USR-019	Remove an authorized key	M	S
FR-USR-020	Generate a new key pair locally and install the public key remotely	M	M
FR-USR-021	Validate `authorized_keys` file permissions and warn if incorrect	M	S
FR-USR-022	Display currently logged-in users and active sessions	M	S
FR-USR-023	Terminate a user session	S	S
FR-USR-024	Display login history (`last`) and failed attempts (`lastb`)	S	S
FR-USR-025	Warn when modifying or deleting the account used by the current session	M	S

FR-USR-026	Warn when removing the last account with sudo access	M	M

Design note (FR-USR-026). Removing the last sudo-capable account renders a host permanently unadministrable without console or rescue access. The check is inexpensive; the failure it prevents is unrecoverable remotely.

5.12 Cron and Scheduler Management

ID	Requirement	Pri	Cx
FR-CRN-001	List cron jobs for the current user	M	S
FR-CRN-002	List cron jobs for all users, with elevation	M	M
FR-CRN-003	List system cron entries (`/etc/crontab`, `/etc/cron.d`, `cron.daily` etc.)	M	M
FR-CRN-004	Parse and display schedule expressions in plain language ("every day at 3:00 AM")	M	M
FR-CRN-005	Display next scheduled run times	M	M
FR-CRN-006	Create a job with a guided schedule builder (presets plus advanced expression entry)	M	L
FR-CRN-007	Validate cron expressions with live plain-language feedback	M	M
FR-CRN-008	Edit and delete jobs	M	S
FR-CRN-009	Enable and disable jobs by commenting rather than deleting	M	S
FR-CRN-010	Display job environment variables (`PATH`, `SHELL`, `MAILTO`)	S	S
FR-CRN-011	Test-run a job immediately and capture output	S	M
FR-CRN-012	Show recent execution evidence from logs where determinable	S	M
FR-CRN-013	List systemd timers with schedule, next run, and last run	M	M
FR-CRN-014	Create systemd timers via guided form	C	L
FR-CRN-015	Warn about common cron pitfalls: absolute paths, minimal environment, output redirection	S	S
FR-CRN-016	List `at` jobs where available	C	S

Design note (FR-CRN-004, FR-CRN-015). `*/15 9-17 * * 1-5` is unreadable to most users and even experienced administrators misread it under time pressure. Rendering it as "every 15 minutes, 9 AM–5 PM, Monday to Friday" is straightforward and directly serves the product's core thesis. The pitfall warnings address the single most common cause of "my cron job doesn't run" — cron's minimal `PATH` differing from an interactive shell's.

5.13 Package Management

ID	Requirement	Pri	Cx
FR-PKG-001	Detect the host package manager: apt, dnf, yum, zypper, pacman, apk, snap, flatpak	M	M
FR-PKG-002	Present a unified UI abstracting manager differences	M	L
FR-PKG-003	List installed packages: name, version, architecture, size, repository, install date	M	M
FR-PKG-004	Search repositories by name and description	M	M
FR-PKG-005	Display package detail: description, dependencies, reverse dependencies, files, changelog	M	M
FR-PKG-006	Install a package with a dependency preview and download size	M	M
FR-PKG-007	Remove a package with a reverse-dependency impact preview	M	M
FR-PKG-008	Distinguish remove from purge (configuration retention)	M	S
FR-PKG-009	List available updates with current and candidate versions	M	M
FR-PKG-010	Identify security updates distinctly where the metadata permits	M	M
FR-PKG-011	Update selected packages	M	M
FR-PKG-012	Full system upgrade with explicit confirmation	M	M
FR-PKG-013	Stream operation output live with progress	M	M
FR-PKG-014	Handle interactive prompts (configuration file conflicts, licence acceptance) via PTY	M	L
FR-PKG-015	Warn when an operation requires a reboot; surface pending-reboot state	M	S
FR-PKG-016	Warn when an operation will remove a large number of packages or a critical package	M	M
FR-PKG-017	Manage repositories: list, add, remove, enable, disable	S	M
FR-PKG-018	Manage repository GPG keys	C	M
FR-PKG-019	Refresh package metadata	M	XS
FR-PKG-020	Clean the package cache with reclaimed-space reporting	S	S
FR-PKG-021	Hold and unhold package versions	S	S
FR-PKG-022	Display package operation history	S	M
FR-PKG-023	Snap and Flatpak management where present	S	M
FR-PKG-024	Multi-host package operations (update N hosts)	S	L

FR-PKG-025	Verify package integrity where the manager supports it	C	M

Design note (FR-PKG-014). `apt` and `dnf` frequently prompt interactively — most notoriously the `dpkg` configuration-file conflict prompt. A non-PTY execution either hangs indefinitely or silently accepts a default that may be wrong. Package management therefore *requires* PTY-based execution with prompt detection and a graphical response mechanism. This is one of the strongest arguments for the PTY execution strategy described in §10.4, and it should be validated in a technical spike before Phase 2 commits to the module.

5.14 Network Management

ID	Requirement	Pri	Cx
FR-NET-001	List interfaces with state, addresses, MAC, MTU, and statistics	M	M
FR-NET-002	Bring interfaces up and down	S	S
FR-NET-003	Configure static and DHCP addressing via the host's mechanism (netplan, NetworkManager, systemd-networkd, ifupdown)	S	XL
FR-NET-004	Display and edit the routing table	S	M
FR-NET-005	Display and configure DNS resolvers	S	M
FR-NET-006	Display and edit `/etc/hosts`	S	S
FR-NET-007	Display and set hostname	S	S
FR-NET-008	Detect the active firewall (ufw, firewalld, nftables, iptables)	M	M
FR-NET-009	List firewall rules in a readable, unified form	M	L
FR-NET-010	Add rules via guided form (port, protocol, source, action, comment)	M	L
FR-NET-011	Delete and reorder rules	M	M
FR-NET-012	Enable and disable the firewall with an explicit lockout warning	M	M
FR-NET-013	Common-service presets (HTTP, HTTPS, SSH, MySQL)	S	S
FR-NET-014	**Prevent rule changes that would sever the current SSH session, with a hard block and override**	M	L
FR-NET-015	Display listening ports with owning processes	M	M
FR-NET-016	Display and edit `sshd_config` with `sshd -t` validation before applying	M	M
FR-NET-017	Warn before applying SSH configuration changes that could cause lockout	M	M
FR-NET-018	Diagnostic tools: ping, traceroute, DNS lookup, port check, from the remote host	S	M
FR-NET-019	Bandwidth test between local and remote	C	M
FR-NET-020	Display active connections with process attribution	M	M

Design note (FR-NET-014, FR-NET-017). Firewall and SSH configuration are the two operations that can permanently lock an operator out of a remote host. The product must model the current session's connection parameters and refuse — not merely warn — any change that would break it, unless the user explicitly overrides after reading a specific explanation. A recommended additional safeguard is a **staged apply with automatic revert**: apply the change, require the user to confirm continued connectivity within 60 seconds, and automatically revert if confirmation does not arrive. This pattern is well established in network device management and is directly applicable here. It is a significant differentiator and worth the implementation cost.

5.15 Backup and Restore

ID	Requirement	Pri	Cx
FR-BAK-001	Back up selected directories to a remote archive	M	M
FR-BAK-002	Back up to a local destination via download	M	M
FR-BAK-003	Include and exclude patterns	M	S
FR-BAK-004	Compression format and level selection	M	S
FR-BAK-005	Incremental backup via rsync where available	S	L
FR-BAK-006	Database backup: mysqldump, pg_dump, SQLite copy	M	M
FR-BAK-007	Combined file and database backup as a single job	S	M
FR-BAK-008	Named backup jobs with saved configuration	M	M
FR-BAK-009	Schedule backup jobs via cron or systemd timer	S	M
FR-BAK-010	Backup catalogue: contents, timestamp, size, source	M	M
FR-BAK-011	Verify backup integrity (archive test, checksum)	M	M
FR-BAK-012	Browse backup contents without full extraction	S	L
FR-BAK-013	Restore a full backup with a destination selector	M	M
FR-BAK-014	Restore selected files from a backup	M	M
FR-BAK-015	Restore preview showing what will be overwritten	M	M
FR-BAK-016	Retention policy with automatic pruning	M	M
FR-BAK-017	Estimate backup size before execution	S	S

FR-BAK-018	Warn if the destination has insufficient free space	M	S
FR-BAK-019	Backup to S3-compatible object storage	C	L
FR-BAK-020	System configuration backup (`/etc` snapshot)	S	M

5.16 Security Management

ID	Requirement	Pri	Cx
FR-SEC-001	Firewall management (see §5.14)	M	L
FR-SEC-002	SSH server configuration with validation (see §5.14)	M	M
FR-SEC-003	Display SSH host keys and fingerprints	S	S
FR-SEC-004	List installed SSL/TLS certificates from common locations	M	M
FR-SEC-005	Display certificate detail: subject, issuer, validity, SANs, key size, signature algorithm	M	M
FR-SEC-006	Warn on certificates expiring within a configurable window	M	M
FR-SEC-007	Generate self-signed certificates and CSRs	S	M
FR-SEC-008	Install certificate files to standard locations	S	M
FR-SEC-009	Let's Encrypt / certbot integration: issue, renew, list	S	L
FR-SEC-010	Verify certificate chain validity	S	M
FR-SEC-011	Test the live TLS configuration of a served endpoint	C	M
FR-SEC-012	Display SELinux / AppArmor status and mode	S	S
FR-SEC-013	Display recent SELinux/AppArmor denials	C	M
FR-SEC-014	Display fail2ban status and banned addresses where present	C	M
FR-SEC-015	Security posture summary: SSH root login, password auth, firewall state, pending security updates, world-writable files in sensitive paths	S	L
FR-SEC-016	Audit log of all actions performed by the product (see §11.5)	M	M
FR-SEC-017	Display recent authentication failures	S	S
FR-SEC-018	Check for common misconfigurations with remediation guidance	C	L
FR-SEC-019	Display listening services and flag unexpected exposure	S	M
FR-SEC-020	Blast-radius classification for every destructive operation, driving confirmation level	M	M

Blast-radius confirmation tiers (FR-SEC-020):

Tier	Criteria	Confirmation
0	Read-only	None
1	Single non-system file, reversible	Standard dialog
2	Multiple files, service state change	Dialog with affected-item count
3	Recursive operation, package removal, user deletion	Dialog + checkbox acknowledgement
4	System path, firewall, SSH config, last sudo user, production host	Typed confirmation of hostname or resource name
5	Operation that would sever the session or render the host unadministrable	Hard block with explicit override and a written explanation of consequence

5.17 System Configuration, Environment, and Disk Management

5.17.1 System Configuration

ID	Requirement	Pri	Cx
FR-SYS-001	Display and edit kernel parameters (`sysctl`), transient and persistent	S	M
FR-SYS-002	Display and set hostname and FQDN	S	S
FR-SYS-003	Display and set timezone	S	S
FR-SYS-004	Display and configure NTP synchronization	S	M
FR-SYS-005	Display and set locale	C	S
FR-SYS-006	Display and set default target / runlevel	C	S
FR-SYS-007	Display kernel modules; load and unload	C	M
FR-SYS-008	Display and edit `limits.conf`	C	M
FR-SYS-009	Reboot and shutdown with scheduling and confirmation	M	S
FR-SYS-010	Display boot messages (`dmesg`)	S	S

5.17.2 Environment Variables

ID	Requirement	Pri	Cx

FR-ENV-001	Display the current session environment	M	S
FR-ENV-002	Display and edit system-wide environment (`/etc/environment`, `/etc/profile.d`)	S	M
FR-ENV-003	Display and edit per-user environment (`.bashrc`, `.profile`)	S	M
FR-ENV-004	Display and edit systemd unit environment	S	M
FR-ENV-005	Manage `.env` files in application directories	S	M
FR-ENV-006	Mask values matching secret-like key patterns by default	M	S
FR-ENV-007	Warn that environment changes require re-login or service restart to take effect	M	XS

5.17.3 Disk and Partition Management

ID	Requirement	Pri	Cx
FR-DSK-001	List block devices, partitions, and their attributes	M	M
FR-DSK-002	Display partition tables	S	M
FR-DSK-003	Display mounted filesystems and mount options	M	S
FR-DSK-004	Mount and unmount filesystems	S	M
FR-DSK-005	Display and edit `/etc/fstab` with validation	S	M
FR-DSK-006	Warn that an invalid fstab entry can prevent boot	M	XS
FR-DSK-007	Display LVM structure	C	M
FR-DSK-008	Extend logical volumes and filesystems	C	L
FR-DSK-009	Create and format partitions	C	L
FR-DSK-010	Display RAID status	C	M
FR-DSK-011	Display SMART health data	C	M
FR-DSK-012	Filesystem check operations	W	L

Design note (FR-DSK-009, FR-DSK-012). Destructive disk operations are deliberately ranked Could-have and Won't-have. Partitioning errors destroy data irrecoverably, and the risk-to-value ratio for a remote GUI performing them is poor. Read-only visibility is the correct initial scope; if partitioning is added later, it must be gated behind explicit expert-mode activation.

5.18 AI Features

Purpose. Reduce the remaining knowledge gap where a graphical interface alone cannot — interpreting unfamiliar errors, translating intent to action, and summarizing large volumes of log data.

Governing constraints. AI features are assistive and never autonomous. The AI proposes; the user disposes.

ID	Requirement	Pri	Cx
FR-AI-001	Natural language to command translation, presented as a proposal requiring explicit approval	S	L
FR-AI-002	**Never execute an AI-generated command automatically under any circumstance**	M	S
FR-AI-003	Display the proposed command with a plain-language explanation of what it does	M	M
FR-AI-004	Flag AI-proposed commands that are destructive, with the same blast-radius tiering as manual operations	M	M
FR-AI-005	Explain a Linux error message in plain language with likely causes	S	M
FR-AI-006	Suggest remediation steps for common failures, as proposals	S	M
FR-AI-007	Summarize a log excerpt, identifying error patterns and anomalies	S	L
FR-AI-008	Explain a configuration file's contents and the effect of a proposed change	C	M
FR-AI-009	Deployment assistant: analyze a local project and suggest a deployment configuration	C	L
FR-AI-010	Explain what a selected command in the transparency panel does (learning aid)	S	M
FR-AI-011	Diagnose a host from collected metrics and recent logs, producing a ranked hypothesis list	C	XL
FR-AI-012	**All AI features must be disableable entirely by the user and by administrative policy**	M	S
FR-AI-013	Explicit, granular disclosure of what data is transmitted, to which provider, before first use	M	M
FR-AI-014	Redact secrets, keys, passwords, and tokens from any data sent to an AI provider	M	L
FR-AI-015	Support a locally-hosted model endpoint for privacy-sensitive environments	C	L
FR-AI-016	Support bring-your-own API key	S	S
FR-AI-017	Never transmit file contents, credentials, or host identifiers without per-instance consent	M	M
FR-AI-018	Clearly label all AI-generated content as such	M	XS

Design note. AI features carry disproportionate risk relative to their engineering cost. An AI that generates a plausible but wrong command, executed automatically, would destroy the product's credibility permanently and irrecoverably. FR-AI-002 is therefore absolute and admits no exception — not for "safe" commands, not for user opt-in, not for convenience.

Equally significant is the data-transmission question. Enterprise customers (Persona 7)

will refuse a tool that sends server configuration to a third-party API. FR-AI-012 through FR-AI-017 are not optional privacy niceties; they are commercial prerequisites for the highest-value segment. AI should be positioned as a valuable optional layer, never as a core dependency — and the product must remain fully functional with every AI feature disabled.

---

6. Non-Functional Requirements

6.1 Performance

ID	Requirement	Target	Measurement
NFR-PRF-001	Application cold start to interactive	≤ 3 s	p95, mid-range hardware
NFR-PRF-002	Application warm start	≤ 1.5 s	p95
NFR-PRF-003	SSH connection establishment	≤ 2 s + network RTT	p95, LAN
NFR-PRF-004	Capability probe completion	≤ 3 s	p95
NFR-PRF-005	Directory listing render, 1,000 entries	≤ 500 ms	p95
NFR-PRF-006	Directory listing render, 100,000 entries	≤ 3 s	p95
NFR-PRF-007	Scroll performance in large tables	≥ 55 fps	Sustained
NFR-PRF-008	Process list refresh	≤ 800 ms	p95
NFR-PRF-009	Metric poll cycle round trip	≤ 500 ms	p95
NFR-PRF-010	Remote CPU consumed by polling	≤ 1%	Mean, 5 s interval
NFR-PRF-011	Log tail latency (line written → displayed)	≤ 1 s	p95
NFR-PRF-012	Editor open, 1 MB file	≤ 1 s	p95
NFR-PRF-013	Editor typing latency	≤ 16 ms	p99
NFR-PRF-014	Deployment comparison, 10,000 files	≤ 30 s	p95
NFR-PRF-015	File transfer throughput	≥ 80% of raw SCP	Same network
NFR-PRF-016	UI thread never blocked	> 100 ms	p99.9
NFR-PRF-017	Idle memory footprint	≤ 400 MB	1 session
NFR-PRF-018	Memory footprint, 10 sessions	≤ 1.2 GB	Steady state
NFR-PRF-019	No memory growth over 24 h idle session	≤ 5%	Soak test
NFR-PRF-020	Installer size	≤ 120 MB	Bundled JRE

Performance strategy. Three techniques carry most of the load: (a) **virtualization** of all large collections so render cost is proportional to visible rows rather than total rows; (b) **command batching** so a metric poll is one round trip rather than eight; (c) **strict asynchrony** so no remote I/O occurs on the JavaFX Application Thread. Violating (c) is the single most likely cause of a product that "feels slow" regardless of raw throughput, and it must be enforced by architecture (see §8.7) rather than by developer discipline.

6.2 Security

ID	Requirement	Pri
NFR-SEC-001	Credentials encrypted at rest with AES-256-GCM	M
NFR-SEC-002	Master key derived via Argon2id or PBKDF2-HMAC-SHA256 (≥ 600,000 iterations)	M
NFR-SEC-003	Windows DPAPI integration for machine-bound key protection	M
NFR-SEC-004	Private key passphrases never persisted unless explicitly opted into	M
NFR-SEC-005	Secrets held in `char[]` / `byte[]` and zeroed after use, never in `String`	M
NFR-SEC-006	Secrets never written to logs, crash dumps, or telemetry	M
NFR-SEC-007	Host key verification mandatory; no silent trust-on-first-use	M
NFR-SEC-008	Modern SSH algorithms only by default; legacy algorithms behind explicit opt-in	M
NFR-SEC-009	Command construction must prevent injection (see §11.7)	M
NFR-SEC-010	All executable code signed with an EV certificate	M
NFR-SEC-011	Update packages signed and signature-verified before application	M
NFR-SEC-012	Dependency vulnerability scanning in CI, build fails on High/Critical	M
NFR-SEC-013	SBOM published with each release	S
NFR-SEC-014	Third-party penetration test before 1.0 GA	M
NFR-SEC-015	No telemetry without opt-in; full disclosure of transmitted fields	M
NFR-SEC-016	Application functions fully offline except for updates and opt-in AI	M
NFR-SEC-017	Session credentials cleared from memory on disconnect	M
NFR-SEC-018	Idle lock requiring re-authentication, configurable	S
NFR-SEC-019	Audit log tamper-evident (hash chained)	S
NFR-SEC-020	Responsible disclosure policy and security contact published	M

6.3 Reliability and Availability

ID	Requirement
NFR-REL-001	Crash-free session rate ≥ 99.5%
NFR-REL-002	No data loss on crash — editor drafts and transfer queues persisted continuously
NFR-REL-003	Automatic recovery of the session set after abnormal termination
NFR-REL-004	Network interruption handled with automatic reconnect and operation resumption where possible
NFR-REL-005	Failure in one session must never affect another session
NFR-REL-006	Failure in one module must never crash the application
NFR-REL-007	Partial batch failures reported per item, never as an aggregate failure
NFR-REL-008	All remote operations idempotent or explicitly marked non-idempotent
NFR-REL-009	Graceful degradation when remote tools are missing
NFR-REL-010	Transfer integrity verifiable on demand

6.4 Scalability

Dimension	Target (v1)	Target (v2)
Concurrent sessions	10	50
Stored profiles	500	5,000
Directory entries rendered	100,000	1,000,000
Process list entries	10,000	50,000
Log file size navigable	5 GB	50 GB
Deployment file count	50,000	250,000
Concurrent transfers	16	32
Multi-host batch operation	20 hosts	200 hosts

6.5 Extensibility and Plugin Architecture

ID	Requirement	Pri
NFR-EXT-001	Modules communicate only through defined service interfaces, never direct references	M
NFR-EXT-002	New modules addable without modifying the core	M
NFR-EXT-003	Adapter interfaces for distribution-specific behaviour (package manager, init system, firewall)	M
NFR-EXT-004	New adapters addable without modifying calling code	M
NFR-EXT-005	Public plugin API with semantic versioning and a documented stability contract	S
NFR-EXT-006	Plugin isolation: a plugin failure must not crash the host application	S
NFR-EXT-007	Plugin permission model with user-visible capability declarations	S
NFR-EXT-008	Plugin signature verification	S
NFR-EXT-009	Theming API for visual customization	C
NFR-EXT-010	Scripting API for user automation	C

Design note. NFR-EXT-003 and NFR-EXT-004 are the architectural expression of distribution neutrality. Every distribution-specific decision must be resolved in an adapter, not in a module. If a module contains a conditional on distribution name, that is a design defect — the adapter layer has failed and the condition will proliferate. This rule should be enforced in code review and, where practical, by static analysis.

6.6 Usability and Accessibility

ID	Requirement	Pri
NFR-USA-001	Core tasks completable by a Windows-literate user without documentation	M
NFR-USA-002	Any function reachable within 3 clicks from the main window	M
NFR-USA-003	Full keyboard operability, no mouse-only functions	M
NFR-USA-004	Standard Windows keyboard conventions honoured	M
NFR-USA-005	Command palette (Ctrl+Shift+P) for keyboard-driven access to all actions	S
NFR-USA-006	Contextual help on every non-obvious control	M
NFR-USA-007	Error messages state what failed, why, and what to do next	M
NFR-USA-008	Screen reader compatibility (NVDA, JAWS, Narrator)	S
NFR-USA-009	WCAG 2.1 AA contrast ratios in both themes	M
NFR-USA-010	UI scaling 100%–300%, honouring Windows DPI settings	M
NFR-USA-011	No information conveyed by colour alone	M
NFR-USA-012	Configurable font sizes	M

NFR-USA-013	Reduced-motion option	S
NFR-USA-014	Undo available wherever technically feasible	S

6.7 Localization

ID	Requirement	Pri
NFR-LOC-001	All UI strings externalized from code	M
NFR-LOC-002	Launch languages: English; then German, French, Spanish, Portuguese, Japanese, Chinese (Simplified), Hindi, Russian	S
NFR-LOC-003	Locale-aware date, time, and number formatting	M
NFR-LOC-004	UTF-8 throughout; correct handling of non-Latin filenames	M
NFR-LOC-005	Layouts tolerant of 40% string expansion	S
NFR-LOC-006	RTL layout support	C
NFR-LOC-007	Linux command output is never translated — only UI chrome	M

Design note (NFR-LOC-007). Translating the interface is correct; translating command output would be a serious error. `Permission denied` must remain `Permission denied` because that string is what the user will search for and what documentation refers to. The *interpretation* alongside it may be localized.

6.8 Offline Support

ID	Requirement
NFR-OFF-001	Application launches and operates without internet connectivity
NFR-OFF-002	Profiles, history, bookmarks, settings, and audit log stored locally
NFR-OFF-003	Editor version history available offline
NFR-OFF-004	Deployment history and manifests available offline
NFR-OFF-005	Documentation bundled locally
NFR-OFF-006	Only update checks, licence validation, and opt-in AI require connectivity
NFR-OFF-007	Licence validation tolerates extended offline periods (≥ 30 days)

6.9 Cross-Platform

Position. Windows-first in design and priority; cross-platform-capable in architecture.

ID	Requirement	Phase
NFR-CRP-001	Windows 10 (1809+) and Windows 11, x64 and ARM64	1
NFR-CRP-002	Platform-specific code isolated behind interfaces	1
NFR-CRP-003	No platform-specific code in modules or domain logic	1
NFR-CRP-004	Linux desktop build (Ubuntu, Fedora)	3
NFR-CRP-005	macOS build (Intel and Apple Silicon)	4
NFR-CRP-006	Platform-idiomatic conventions per platform	3–4
NFR-CRP-007	Profiles portable across platforms	3

Design note. Windows-first is a deliberate strategic choice, not a limitation. The target user is defined partly by being a Windows desktop user managing Linux servers; that is precisely the underserved gap. Attempting simultaneous three-platform launch would dilute the Windows experience — native drag-and-drop, DPAPI credential integration, Explorer integration, PuTTY key import — that constitutes a meaningful part of the differentiation. Architecture must permit later expansion; the initial release must not compromise for it.

6.10 Maintainability, Observability, and Compliance

ID	Requirement	Target
NFR-MNT-001	Unit test coverage of domain and service layers	≥ 80%
NFR-MNT-002	Integration test coverage against containerized reference distributions	All P0 flows
NFR-MNT-003	Automated UI tests for critical paths	≥ 20 scenarios
NFR-MNT-004	Static analysis with zero Critical findings	Build gate
NFR-MNT-005	Cyclomatic complexity per method	≤ 15
NFR-MNT-006	Public API documented	100%
NFR-MNT-007	Structured local logging with configurable levels and rotation	—
NFR-MNT-008	One-click diagnostic bundle export (logs, environment, redacted config)	—
NFR-MNT-009	Crash reporting, opt-in, with a pre-submission preview of transmitted data	—
NFR-MNT-010	GDPR compliance: data minimization, export, deletion	—
NFR-MNT-011	Third-party licence compliance and attribution file	—

NFR-MNT-012	Accessibility conformance statement published	—

7. UI/UX Design

7.1 Design Language

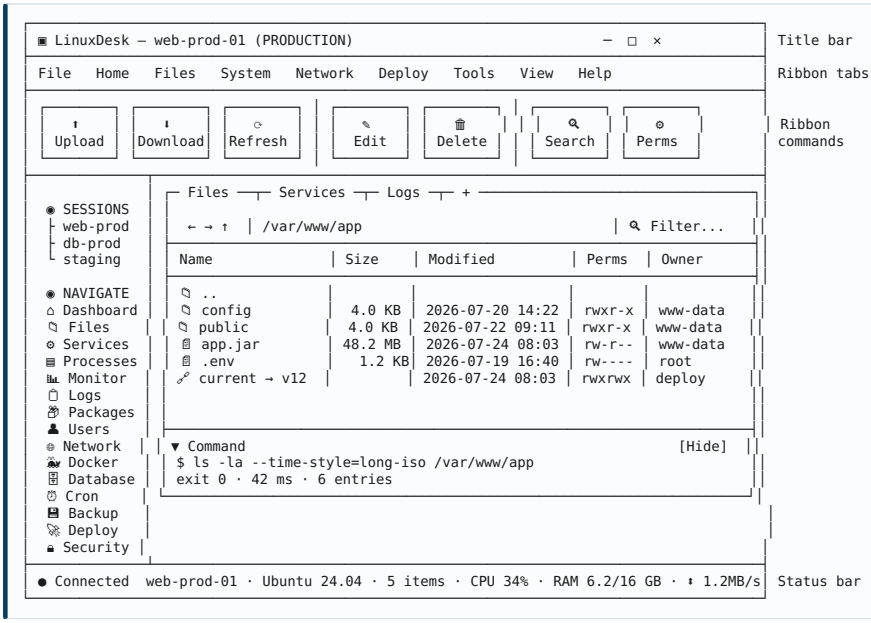
Foundations. Windows 11 Fluent-influenced: clear hierarchy, generous but not wasteful spacing, subtle depth, restrained motion. Information density tuned for professional use — denser than a consumer application, looser than a terminal.

Element	Specification
Primary UI font	Segoe UI Variable 9pt (Windows), system default elsewhere
Monospace font	Cascadia Code / JetBrains Mono 10pt
Base grid	4 px
Corner radius	4 px controls, 8 px panels
Row height (dense)	24 px
Row height (comfortable)	32 px
Icon set	Fluent-style outline, 16/20/24 px
Motion duration	120–180 ms, ease-out; disableable
Elevation	3 levels: base, raised, overlay

Semantic colour. Colour carries meaning consistently and is always paired with an icon or text (NFR-USA-011).

Semantic	Light	Dark	Usage
Success / running	`#107C10`	`#6CCB5F`	Active services, completed operations
Warning	`#F7630C`	`#FCA43C`	Thresholds approached, deprecations
Error / stopped	`#C42B1C`	`#FF6B5C`	Failures, stopped critical services
Info	`#0078D4`	`#4CC2FF`	Neutral notices, selection
Production	`#8B1A1A`	`#B33A3A`	Production host chrome
Neutral text	`#1B1B1B`	`#E8E8E8`	Body
Surface	`#FFFFFF`	`#1F1F1F`	Panels
Background	`#F3F3F3`	`#141414`	Window

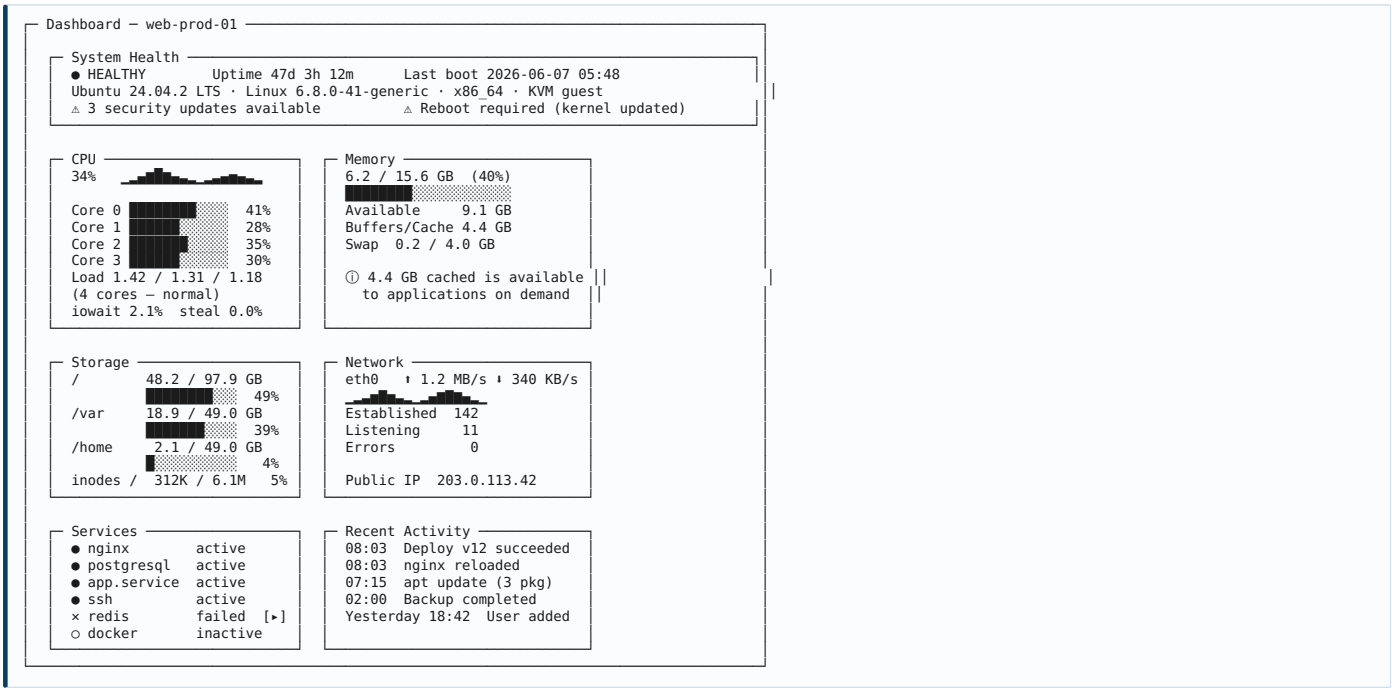
7.2 Application Shell



Layout rationale. The ribbon provides discoverability (P1: recognition over recall) and is the most familiar command surface for Windows users. The left sidebar provides persistent module navigation and session switching. The tabbed centre allows multiple views within one session. The transparency panel sits at the bottom of the work area — visible but subordinate. The status bar carries continuous ambient awareness of connection and load.

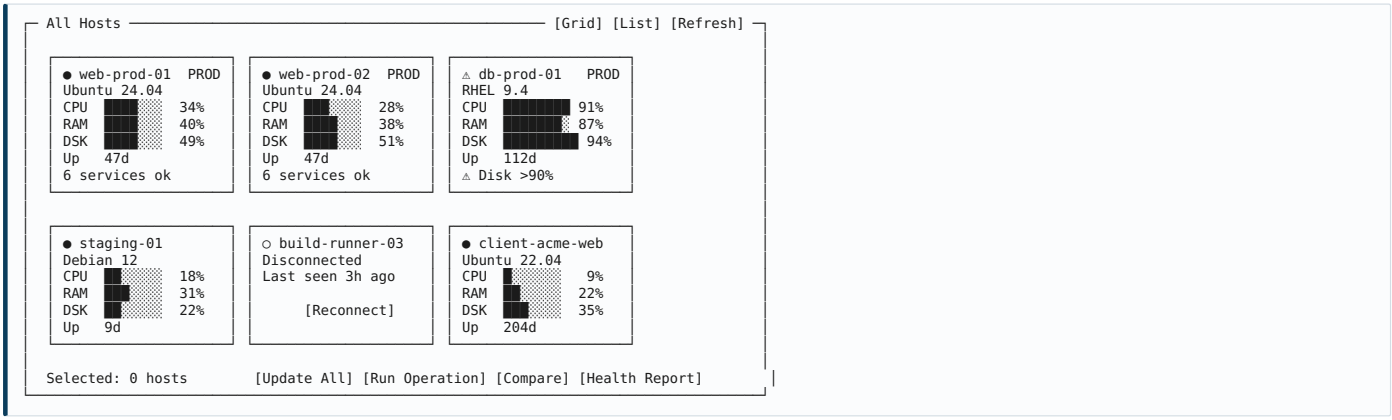
Production indication. When the active session targets a production-tagged host, the title bar and the tab strip render in the production colour and the hostname is suffixed with `(PRODUCTION)`. This is deliberately hard to miss.

7.3 Dashboard



Design rationale. This single screen answers the three questions Personas 2, 5, and 6 ask most often: is it up, is it healthy, and what changed recently. Note the memory card's inline explanation (FR-MON-021) and the load average's core-relative annotation (FR-MON-016) — interpretation, not just measurement.

7.4 Multi-Host Overview

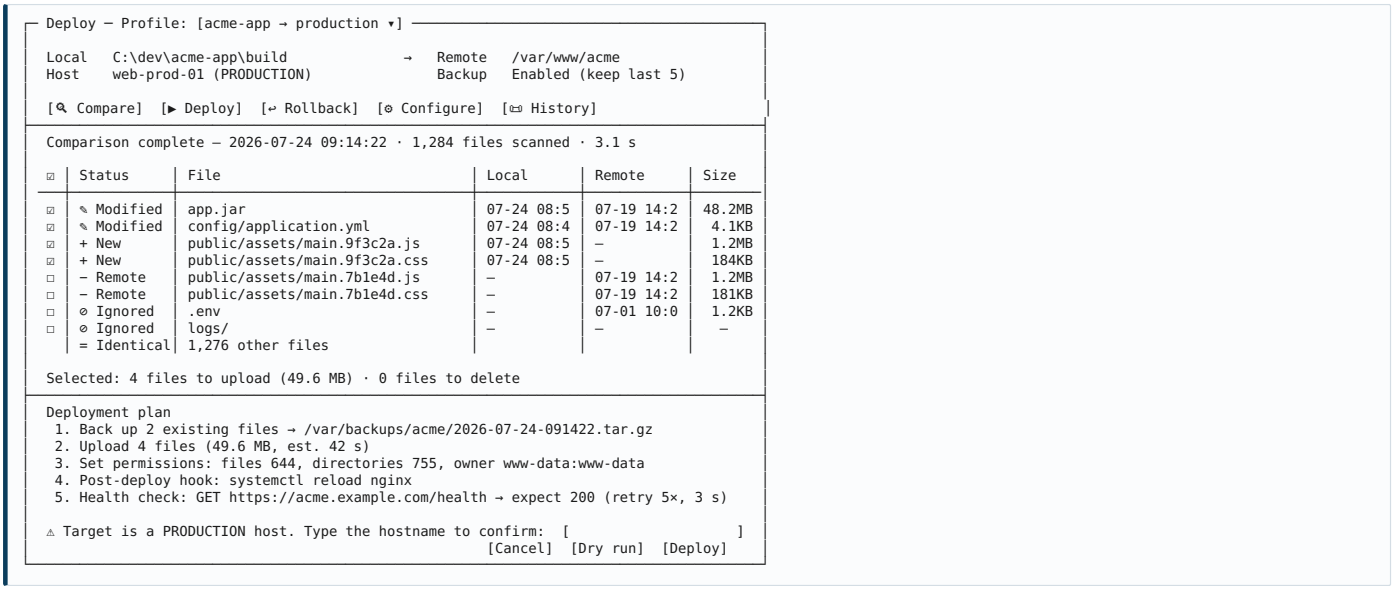


7.5 Service Manager

Design rationale. This screen demonstrates Principle P6 (explain, don't just execute) concretely. The raw journal output is shown *and* interpreted, with a proposed fix that the user may apply, inspect, or dismiss. The "Show me why" link expands into an explanation of Unix

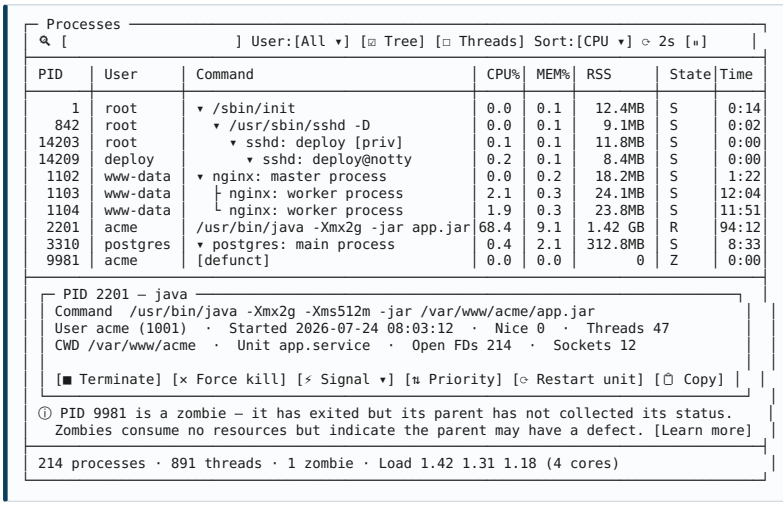
file ownership as it applies to this case — the learning path for Persona 4.

7.6 Deployment

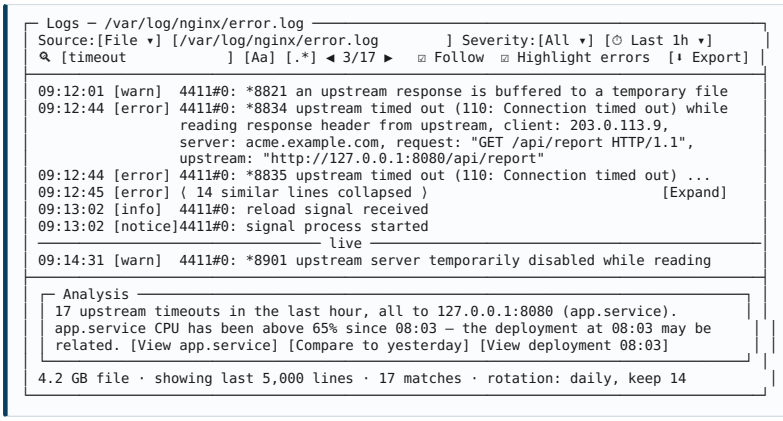


Design rationale. The full plan is stated before execution — nothing happens that was not described. The production host requires typed confirmation (blast-radius tier 4). Dry run sits adjacent to Deploy, making the safe option equally easy to reach.

7.7 Process Manager



7.8 Log Viewer



Design rationale. The correlation panel is where the product's integration advantage becomes visible: logs, service metrics, and deployment history are in the same application, so connecting them is possible. Neither WinSCP nor Cockpit nor Portainer can offer this because none of them holds all three data sources.

7.9 Permissions Dialog

```
┌─ Permissions — /var/www/acme ──────────────────────────────────────┐
│                                                                    │
│                    Read      Write     Execute                     │
│  Owner    (www-data)   ☑         ☑         ☑                       │
│  Group    (www-data)   ☑         ☐         ☑                       │
│  Others                ☑         ☐         ☑                       │
│                                                                    │
│  Special    ☐ Set UID    ☐ Set GID    ☐ Sticky bit                 │
│                                                                    │
│  Octal  [ 755 ]          Symbolic  drwxr-xr-x                      │
│                                                                    │
│  Owner  [www-data          ▾]     Group  [www-data          ▾]     │
│                                                                    │
│  ☑ Apply recursively                                               │
│     Files       [ 644 ]    Directories  [ 755 ]                    │
│                                                                    │
│  ⚠ This will modify 1,284 files in 96 directories.                 │
│                                                                    │
│  ┌─ Command ──────────────────────────────────────────────────┐    │
│  │ find /var/www/acme -type d -exec chmod 755 {} +            │    │
│  │ find /var/www/acme -type f -exec chmod 644 {} +            │    │
│  │ chown -R www-data:www-data /var/www/acme                   │    │
│  └────────────────────────────────────────────────────────────┘    │
│                                          [Cancel]  [Preview]  [Apply]
└────────────────────────────────────────────────────────────────────┘
```

Design rationale. Separate file and directory masks encode a real piece of expertise: applying `chmod -R 644` to a tree removes the execute bit from directories and makes them unenterable — a common and confusing self-inflicted failure. The GUI structurally prevents it while showing exactly what it will run.

7.10 Context Menus

```
Right-click on a file:                 Right-click on a service:

┌──────────────────────────┐           ┌──────────────────────────┐
│ Open                     │           │ Start                    │
│ Open with…            ▸  │           │ Stop                     │
│ Edit                     │           │ Restart                  │
│ Preview                  │           │ Reload configuration     │
├──────────────────────────┤           ├──────────────────────────┤
│ Download          Ctrl+D │           │ Enable at boot           │
│ Copy              Ctrl+C │           │ Disable at boot          │
│ Cut               Ctrl+X │           │ Mask                     │
│ Paste             Ctrl+V │           ├──────────────────────────┤
├──────────────────────────┤           │ View logs                │
│ Rename               F2  │           │ View unit file           │
│ Delete              Del  │           │ Edit override…           │
│ Duplicate                │           │ Dependencies          ▸  │
├──────────────────────────┤           ├──────────────────────────┤
│ Compress…             ▸  │           │ Copy unit name           │
│ Checksum              ▸  │           │ Properties               │
├──────────────────────────┤           └──────────────────────────┘
│ Permissions…             │
│ Ownership…               │
│ Create link…          ▸  │
├──────────────────────────┤
│ Copy path                │
│ Open terminal here  Ctrl+` │
│ Properties        Alt+Enter │
└──────────────────────────┘
```

7.11 Notifications and Status

```
Toast (transient, bottom-right):

┌──────────────────────────────┐
│ ✓ Deployment complete        │
│   4 files · 49.6 MB · 44 s   │
│   Health check passed        │
│                 [View report]│
└──────────────────────────────┘

Alert (persistent banner):

┌──────────────────────────────┐
│ ⚠ db-prod-01: disk usage 94% │
│   /var has 2.8 GB free       │
│        [Investigate] [Dismiss]│
└──────────────────────────────┘

Progress (transfer queue):

┌──────────────────────────────────────┐
│ Uploading 4 files                    │
│ ████████████████▒▒▒▒  68%  33.7/49.6 │
│ main.9f3c2a.js          1.2 MB  ✓    │
│ main.9f3c2a.css         184 KB  ✓    │
│ app.jar                48.2 MB  ⟳ 62%│
│ application.yml          4.1 KB  ⋯   │
│ 2.4 MB/s · 6 s remaining             │
│              [Pause] [Cancel] [Details]
└──────────────────────────────────────┘

Error (modal, actionable):

┌──────────────────────────────────────┐
│ × Cannot write to /etc/nginx/nginx.conf
│                                      │
│ Permission denied. This file is owned│
│ by root and you are connected as     │
│ 'deploy'.                            │
│                                      │
│ You can retry with administrator     │
│ rights (sudo).                       │
│                                      │
│  [Cancel]  [Show details]  [Retry as │
│                            administrator]
└──────────────────────────────────────┘
```

Design rationale. The error dialog demonstrates NFR-USA-007: what failed, why it failed, and what to do next, with the remedy as a button rather than an instruction. Compare with the shell's bare `Permission denied`.

7.12 Keyboard Model

Shortcut	Action	Shortcut	Action
`Ctrl+N`	New connection	`F2`	Rename

`Ctrl+T`	New tab	`F5`	Refresh
`Ctrl+W`	Close tab	`Del`	Delete
`Ctrl+Tab`	Next session	`Shift+Del`	Delete permanently
`Ctrl+1..9`	Switch to session N	`Alt+Enter`	Properties
`Ctrl+Shift+P`	Command palette	`Ctrl+C/X/V`	Copy / cut / paste
`Ctrl+F`	Find in view	`Ctrl+A`	Select all
`Ctrl+Shift+F`	Search remote host	`Backspace`	Up one level
`Ctrl+D`	Download selection	`Alt+←/→`	Back / forward
`Ctrl+U`	Upload	`Ctrl+L`	Focus path bar
`Ctrl+E`	Edit selection	`Ctrl+H`	Toggle hidden files
`Ctrl+``	Open terminal here	`Ctrl+,`	Settings
`Ctrl+S`	Save (editor)	`Ctrl+Shift+D`	Deploy
`Ctrl+Shift+L`	Open logs	`Ctrl+Shift+M`	Open monitor
`Esc`	Cancel operation	`F1`	Contextual help

Command palette. `Ctrl+Shift+P` opens a fuzzy-searchable list of every action in the application, scoped to the current context. This serves rung-3 and rung-4 users (§3.2.3) who want GUI safety with keyboard speed, and it doubles as a discoverability mechanism — typing "restart" surfaces every restart-capable action across all modules.

7.13 Theming and Responsive Behaviour

Themes. Light, Dark, and System-follow. Theme switching is instantaneous with no restart. Both themes meet WCAG 2.1 AA contrast. An optional high-contrast theme supports Windows high-contrast mode.

Density. Compact (24 px rows), Comfortable (32 px rows), Spacious (40 px rows) — user-selectable, defaulting to Comfortable.

Responsive breakpoints.

Window Width	Behaviour
≥ 1600 px	Full layout: sidebar expanded, dual-pane, detail panel visible
1200–1599 px	Sidebar expanded, single pane, detail panel collapsible
900–1199 px	Sidebar collapsed to icons, detail panel as overlay
< 900 px	Sidebar hidden behind a toggle, ribbon collapsed to a compact bar

Multi-window. Any tab may be torn off into a separate window, supporting multi-monitor workflows — a common requirement for administrators monitoring one host while working on another.

8. Software Architecture

8.1 Architectural Drivers

The architecture is shaped by six forces, in priority order:

1. **Responsiveness under high latency.** Every operation crosses a network with 10–300 ms RTT. The UI must never wait synchronously. This is the dominant constraint and it drives the asynchronous command bus, channel pooling, and command batching.
2. **Heterogeneity of targets.** The same UI must drive Ubuntu, RHEL, Debian, SUSE, Alpine, and Amazon Linux across multiple versions. This drives the adapter layer.
3. **Breadth of modules.** Fifteen-plus functional modules must be developed, tested, and released without coupling. This drives strict modularity and the event bus.
4. **Safety.** Destructive operations must be intercepted uniformly, not per-module. This drives a central command interception pipeline.
5. **Auditability.** Every action must be recorded consistently. This also drives central interception — the same pipeline serves both.
6. **Extensibility.** Third-party modules must be addable later without core changes. This drives service-interface isolation and, eventually, classloader isolation.

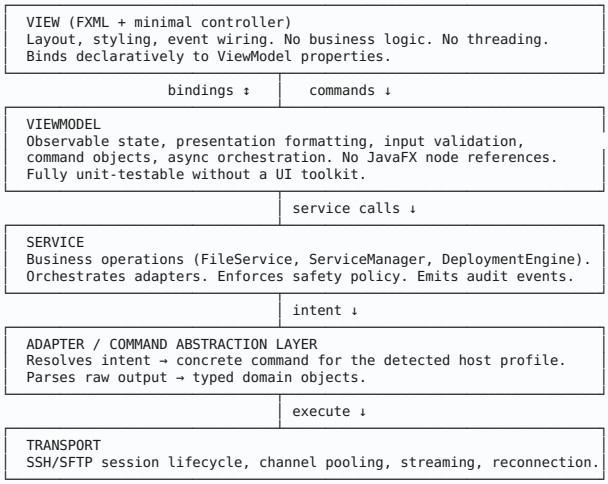
8.2 Pattern Selection: MVVM

Recommendation: MVVM (Model–View–ViewModel), not MVC.

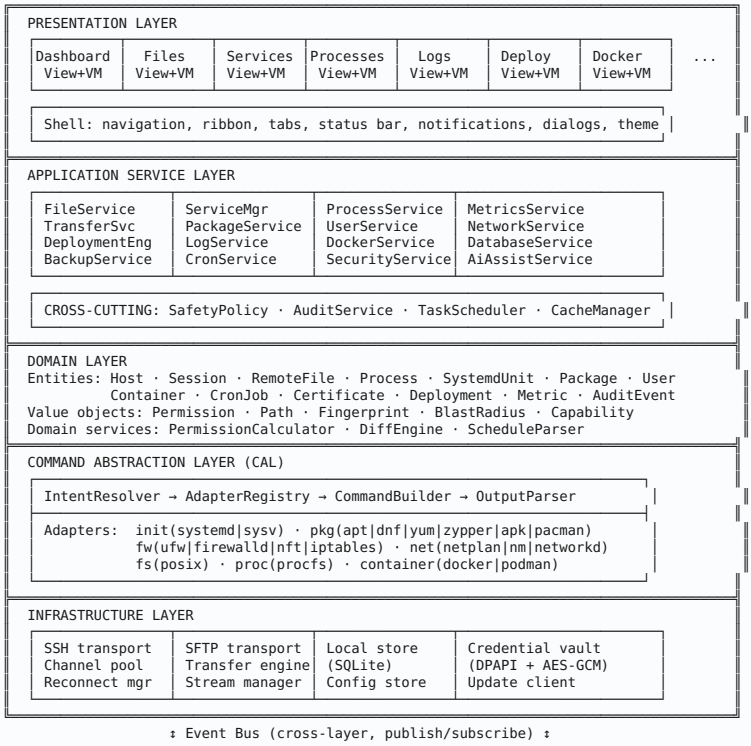
Criterion	MVC	MVVM	Assessment
Fit with JavaFX property/binding system	Poor — controller manually syncs state	Excellent — properties bind declaratively	**MVVM**
Testability of presentation logic	Controller often couples to view	ViewModel is a plain testable object	**MVVM**
Async operation handling	Awkward — controller manages threading	Natural — ViewModel exposes observable state updated off-thread	**MVVM**
Suitability for data-heavy tables	Manual refresh logic	`ObservableList` binds directly to `TableView`	**MVVM**
Team familiarity	Higher	Moderate	MVC
Boilerplate	Lower	Higher	MVC

JavaFX's `Property` / `Binding` infrastructure is an MVVM implementation in all but name; adopting MVC would mean fighting the framework. The additional boilerplate is real but is mitigated by a shared `BaseViewModel` providing async operation state (idle/running/succeeded/failed), error propagation, and cancellation.

Layer responsibilities:



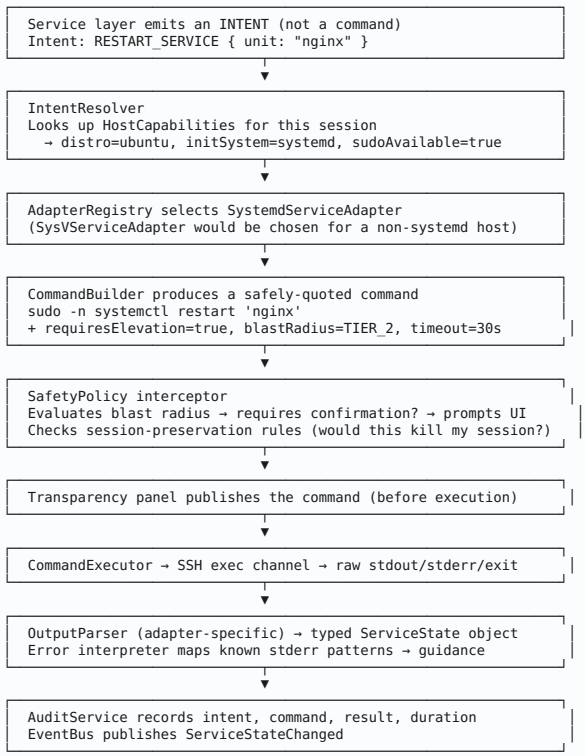
8.3 Layered Architecture



Dependency rule. Dependencies point downward only. The domain layer depends on nothing. Presentation never reaches past the service layer. This is enforced by module boundaries (JPMS) and verified in CI by an ArchUnit test suite — an architectural rule that is not automatically enforced will erode within months on a project of this size.

8.4 Command Abstraction Layer

The CAL is the component that makes distribution neutrality tractable, and it is the most architecturally distinctive part of the system.



Why this matters. Every module speaks intents. No module contains a distribution conditional. Adding Alpine support means writing an `ApkPackageAdapter` — the Package module is untouched. Adding a safety rule means changing `SafetyPolicy` once, and every module inherits it. Adding audit fields means changing `AuditService` once.

Parser strategy. Output parsing is the most defect-prone part of the system, because command output formats vary by tool version and locale. Three mitigations:

1. **Prefer machine-readable output.** `systemctl show --property=...`, `ps -o` with explicit fields, `ss -H`, `df -P`, `docker ... --format`

`'{{json .}}'`, `journalctl -o json`. Never parse human-formatted output where a stable machine format exists.

2. **Force a stable locale.** Every command executes with `LC_ALL=C` to prevent locale-dependent formatting and month names.
3. **Parser test corpus.** Real captured output from every supported distribution and version, stored as fixtures and run in CI. When a parser breaks on a new release, the test catches it before the user does.

8.5 Module Structure

```
linuxdesk/
├── app/                        Bootstrap, DI wiring, main class
├── core/
│   ├── domain/                 Entities, value objects, domain services
│   ├── events/                 Event bus, event definitions
│   ├── task/                   Task scheduler, progress, cancellation
│   ├── cache/                  Multi-tier cache
│   └── safety/                 SafetyPolicy, blast radius, confirmations
├── transport/
│   ├── ssh/                    Session, channel pool, reconnect, keepalive
│   ├── sftp/                   File operations, transfer engine
│   ├── auth/                   Auth providers, agent integration, host keys
│   └── stream/                 Long-lived stream management
├── cal/
│   ├── intent/                 Intent definitions
│   ├── registry/               Adapter registry, capability probe
│   ├── builder/                Command construction, quoting, elevation
│   ├── parser/                 Output parsers, error interpreter
│   └── adapters/
│       ├── init/               systemd, sysv
│       ├── pkg/                apt, dnf, yum, zypper, apk, pacman, snap
│       ├── firewall/           ufw, firewalld, nftables, iptables
│       ├── network/            netplan, networkmanager, networkd, ifupdown
│       ├── container/          docker, podman
│       └── fs/                 posix
├── services/
│   ├── file/  service/  process/  metrics/  logs/  package/
│   ├── user/  network/  docker/   database/ cron/  backup/
│   ├── security/  deployment/  audit/  ai/
├── ui/
│   ├── shell/                  Window, ribbon, sidebar, tabs, status bar
│   ├── common/                 Reusable controls, virtualized table, charts
│   ├── theme/                  CSS, tokens, theme switching
│   ├── dialogs/                Confirmation, error, progress, properties
│   └── modules/                One package per functional module (View+VM)
├── persistence/
│   ├── store/                  SQLite access, migrations
│   ├── vault/                  Credential encryption
│   └── settings/               Preferences
├── platform/
│   ├── windows/                DPAPI, shell integration, drag-out, notifications
│   └── api/                    Platform-neutral interfaces
├── plugin/
│   ├── api/                    Public plugin SPI
│   └── host/                   Loader, isolation, permissions
└── update/                     Update check, download, verify, apply
```

Module boundaries are declared with JPMS (`module-info.java`), which enforces at compile time what would otherwise be a convention. The `ui` module cannot reach `transport` directly; it must go through `services`.

8.6 Dependency Injection

Recommendation: a lightweight compile-time DI framework (Dagger 2 or Micronaut DI), not Spring, and not manual wiring.

Option	Startup cost	Reflection	Verdict
Spring Framework	High (reflection, context scan)	Heavy	**Rejected** — startup budget is 3 s (NFR-PRF-001); Spring's cost is disproportionate for a desktop app
Guice	Moderate	Runtime reflection	Viable but slower than needed
Dagger 2	Near-zero (compile-time codegen)	None	**Recommended** — no startup penalty, fails at compile time rather than runtime
Manual wiring	Zero	None	Viable at first, unmaintainable at 15+ modules

Compile-time DI also interacts well with GraalVM native-image should that be pursued later for startup optimization.

Scopes: `@Singleton` for application-wide services; `@SessionScope` for per-connection objects (session, capabilities, channel pool, caches); `@ModuleScope` for per-module state. Session scope is critical — a per-session object graph makes it structurally impossible to leak state between hosts, which is the defect class most likely to cause a "wrong server" incident.

8.7 Concurrency and Background Tasks

Threading model:

Pool	Purpose	Sizing
JavaFX Application Thread	UI rendering and event handling only	1 (framework)
Command executor (per session)	Short remote commands	4 threads
Transfer executor (per session)	File transfers	Configurable, default 4
Stream executor (per session)	Long-lived streams (log tail, live stats)	1 per active stream
Polling scheduler	Periodic metric collection	1 scheduled thread per session

Local worker pool	Diffing, checksums, parsing, compression	`min(cores, 8)`

The cardinal rule: no blocking I/O on the JavaFX Application Thread, ever. Enforced by (a) an assertion in the transport layer that throws if invoked on the FX thread in debug builds, and (b) an ArchUnit rule preventing UI classes from calling transport classes directly.

Async pattern. All service methods return `CompletableFuture<T>`. ViewModels attach continuations that marshal back to the FX thread via `Platform.runLater`. A shared `AsyncOperation<T>` wrapper provides observable `running` / `progress` / `result` / `error` properties, so views bind to operation state rather than managing it.

Cancellation. Every operation carries a cancellation token. Cancelling a remote command sends the appropriate signal and closes the channel; cancelling a transfer aborts cleanly and records resumable state; cancelling a search terminates the remote `find` / `grep` process rather than merely ignoring its output. Cancellation that does not actually stop remote work is a common and costly defect — it leaves load on the server.

Backpressure. Streaming sources (log tail, `docker stats`) can produce faster than the UI can render. Streams use a bounded buffer with a drop-oldest policy and a visible indicator when lines are dropped, rather than growing unboundedly until the heap exhausts.

8.8 Event Bus

A typed, in-process publish/subscribe bus decouples modules.

Event	Publisher	Typical Subscribers
`SessionConnected` / `SessionLost`	Transport	All modules, status bar, sidebar
`CapabilitiesDetected`	CAL	All modules (enable/disable features)
`CommandExecuted`	Executor	Transparency panel, audit service
`FileSystemChanged`	File service	File views, deployment, editor
`ServiceStateChanged`	Service manager	Dashboard, service view, processes
`MetricSampled`	Metrics service	Dashboard, charts, alert evaluator
`ThresholdBreached`	Alert evaluator	Notification centre, dashboard
`TransferProgress` / `TransferCompleted`	Transfer engine	Queue view, status bar, notifications
`DeploymentCompleted`	Deployment engine	History, dashboard activity, notifications
`AuditEventRecorded`	Audit service	Audit view, export

Discipline required. Event buses degrade into untraceable spaghetti when overused. Rules: events are past-tense facts, never commands; events are immutable; a subscriber must never publish synchronously in response to an event it received (preventing cascades); the full event catalogue is documented in one file. Direct service calls remain the default; the bus is for genuine cross-cutting notification only.

8.9 Caching

Tier	Contents	TTL	Invalidation
L1 in-memory hot	Current directory listing, visible process page	5 s	Explicit refresh, FileSystemChanged event
L2 in-memory session	Capabilities, user list, group list, package index, unit list	5–15 min	Manual refresh, related mutation
L3 local persistent	Profiles, history, editor versions, deployment manifests, audit log	Indefinite	Explicit deletion, retention policy
L4 negative cache	Absent tools, permission-denied paths	Session	Reconnect

Cache correctness principle: never serve a cached value for data the user is about to act on destructively. A delete operation re-stats the target immediately before acting. Showing a stale listing is a minor annoyance; deleting based on one is a serious defect.

8.10 Plugin Framework

Phased approach. Phase 1–2 use internal module registration only. A public plugin API arrives in Phase 4, once the internal interfaces have stabilized under real use — publishing an API before it has stabilized commits the project to supporting design mistakes indefinitely.

Extension points (planned): new sidebar module; new adapter (distro/tool support); context menu contribution; ribbon command; file previewer; log parser/highlighter; health check type; deployment hook; theme.

Isolation: each plugin loads in its own `ModuleLayer` with its own classloader. Plugins declare required capabilities (`filesystem.read`, `command.execute`, `network.outbound`) in a manifest; the user sees and approves these at install time. A plugin that throws is disabled with a notification, not permitted to crash the host.

---

9. Technology Stack

9.1 Summary

Layer	Selection	Rationale
Language	**Java 21 LTS**	Virtual threads, pattern matching, records, sealed types; LTS support horizon
UI toolkit	**JavaFX 21+**	Specified; mature, property/binding model, CSS styling, hardware-accelerated
Build	**Gradle 8+ (Kotlin DSL)**	Better incremental builds and multi-module ergonomics than Maven for this shape
SSH/SFTP	**Apache MINA SSHD** (primary), JSch fork (fallback)	Actively maintained, permissive licence, full protocol coverage
DI	**Dagger 2**	Compile-time, zero startup cost
Editor	**RichTextFX**	Mature JavaFX rich text with syntax highlighting support
Local store	**SQLite via JDBC**	Embedded, zero-admin, transactional, portable
ORM/access	**JOOQ** or plain JDBC	Type-safe SQL without ORM weight
Logging	**SLF4J + Logback**	Standard, structured output, rotation
Charts	**JavaFX Charts** + custom canvas	Built-in for simple charts, canvas for high-frequency
JSON	**Jackson**	Standard, streaming, well-understood
YAML	**SnakeYAML**	Compose files, netplan, unit parsing
HTTP	**Java 11+ HttpClient**	Built in, async, no dependency
Diff	**java-diff-utils**	Myers diff for editor and deployment comparison
Crypto	**Bouncy Castle** + JCA	Key parsing, formats JCA lacks
Native interop	**JNA**	DPAPI, shell integration, drag-out
Testing	**JUnit 5, Mockito, AssertJ, TestFX, Testcontainers, ArchUnit**	Full pyramid including containerized distro testing
Packaging	**jpackage** (JDK)	Native MSI with bundled JRE
Installer	**WiX Toolset** via jpackage	Per-user and per-machine install, upgrade handling
Update	**Custom + signature verification**	Full control over verification and rollout
Code quality	**SpotBugs, PMD, Checkstyle, OWASP Dependency-Check**	CI gates

9.2 Key Selection Rationale

9.2.1 SSH Library

Library	Licence	Maintenance	SFTP	PTY	Agent	Verdict
Apache MINA SSHD	Apache 2.0	Active	Full	Yes	Yes	**Selected**
JSch (original)	BSD	Abandoned	Full	Yes	Partial	Rejected — unmaintained
JSch forks (mwiede)	BSD	Active	Full	Yes	Yes	Viable fallback
SSHJ	Apache 2.0	Moderate	Full	Yes	Yes	Strong alternative
Ganymed	BSD	Abandoned	Limited	Yes	No	Rejected

MINA SSHD is selected for licence compatibility, active maintenance, complete protocol support including modern algorithms, and a client API that exposes the channel-level control the channel pool requires. **Risk mitigation:** the transport layer wraps the library behind an internal interface so a substitution is contained to one module. Given that the SSH library is the deepest external dependency in the system, this insulation is not optional.

9.2.2 Java 21 and Virtual Threads

Java 21's virtual threads are unusually well-matched to this workload. The application is overwhelmingly I/O-bound — waiting on network round trips — which is precisely what virtual threads optimize. A blocking-style programming model becomes viable for transport code without the thread-count cost of platform threads, simplifying the transfer engine and per-stream handling considerably. Structured concurrency (preview) further simplifies fan-out operations such as multi-host commands.

9.2.3 Packaging

`jpackage` produces a native Windows MSI with a bundled JRE, eliminating the "install Java first" barrier that has historically damaged Java desktop adoption. Combined with `jlink` to strip unused JDK modules, the installer target of ≤120 MB (NFR-PRF-020) is achievable.

GraalVM native-image is attractive for startup time but is not recommended for v1: JavaFX native-image support requires substantial reflection configuration, and the build complexity is a poor trade against a 3-second startup budget that jlink+jpackage can meet. Revisit at Phase 4.

9.2.4 Rejected Alternatives

Rejected	Reason
Electron / web stack	Memory footprint, table performance at 100k rows, no native drag-out, contradicts native-desktop differentiation

Swing	Legacy; JavaFX specified; weaker styling and binding
Compose Multiplatform	Promising but less mature for data-dense desktop; smaller talent pool
Spring Boot	Startup cost; server-oriented
Hibernate	Weight disproportionate to a local SQLite store
Maven	Gradle's incremental build and multi-module handling are materially better here

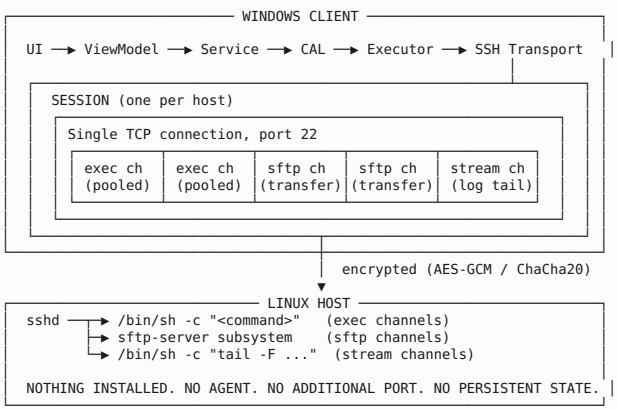
9.3 Development Infrastructure

Concern	Tooling
Source control	Git, trunk-based with short-lived branches
CI/CD	GitHub Actions — build, test, static analysis, dependency scan, sign, package
Test infrastructure	Testcontainers running Ubuntu 22.04/24.04, Debian 12, RHEL 9, Rocky 9, Alpine 3.20, SUSE 15
Code review	Mandatory, minimum one approver, architecture rules enforced by ArchUnit in CI
Documentation	Markdown in-repo; API docs from Javadoc; user docs from the same source
Telemetry	Opt-in, self-hosted, minimal field set
Crash reporting	Opt-in with pre-submission preview

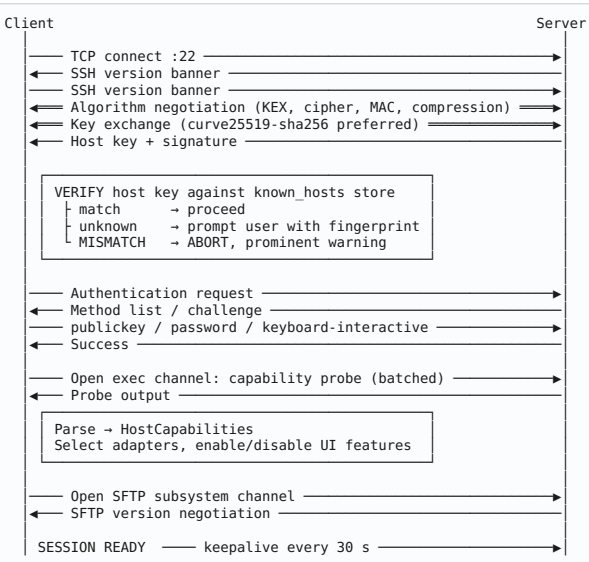
Testcontainers-based integration testing is not optional. With a target matrix of six-plus distributions, manual verification does not scale and regressions in output parsing will otherwise reach users. Every adapter must have integration tests running against real distribution images in CI.

10. Communication Flow

10.1 Transport Overview



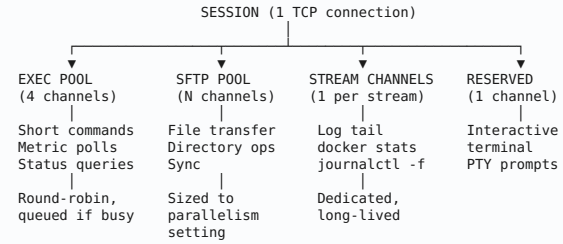
10.2 Connection Establishment



The capability probe as a single batched command — a compound shell invocation writing delimited sections — reduces what would be twenty round trips to one. On a 200 ms RTT link this is the difference between 4 seconds and 0.3 seconds, and it is the reason NFR-PRF-004 is achievable.

10.3 Channel Multiplexing

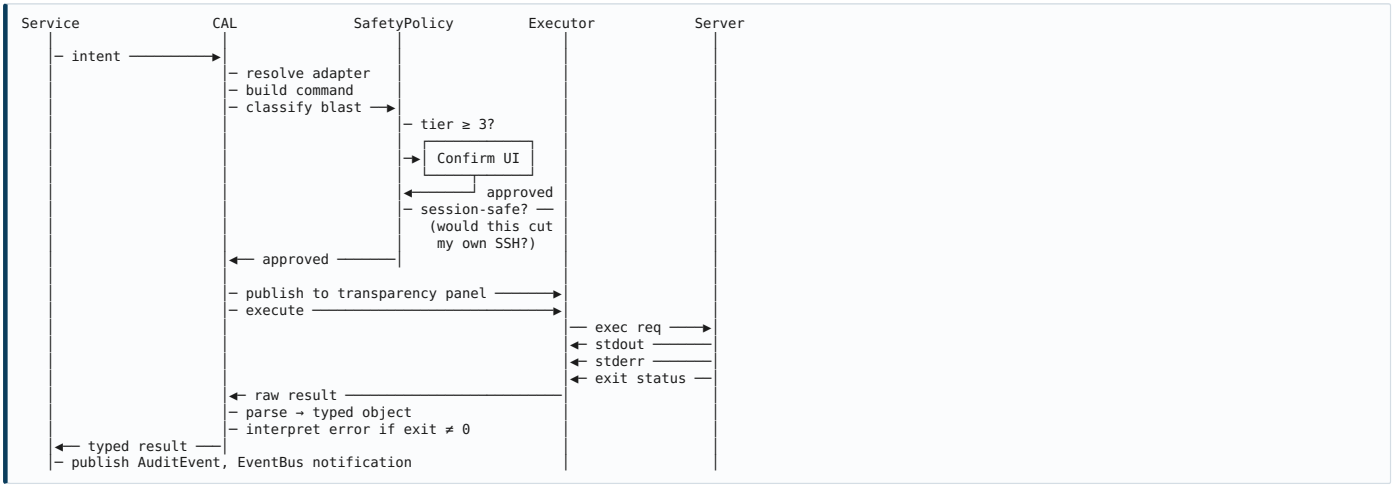
A single TCP connection carries many logical channels, which is why the product does not need one SSH connection per operation.



Why pooling matters. Opening an SSH channel is cheap relative to opening a TCP connection but not free. Pooling exec channels avoids per-command channel setup, which on a high-latency link is the difference between a process list refreshing in 300 ms and in 900 ms.

Reserved channel. One channel is always held in reserve for interactive PTY use. Without this, an operation requiring a sudo password prompt during a period of heavy transfer activity would queue behind transfers and appear to hang.

10.4 Command Execution

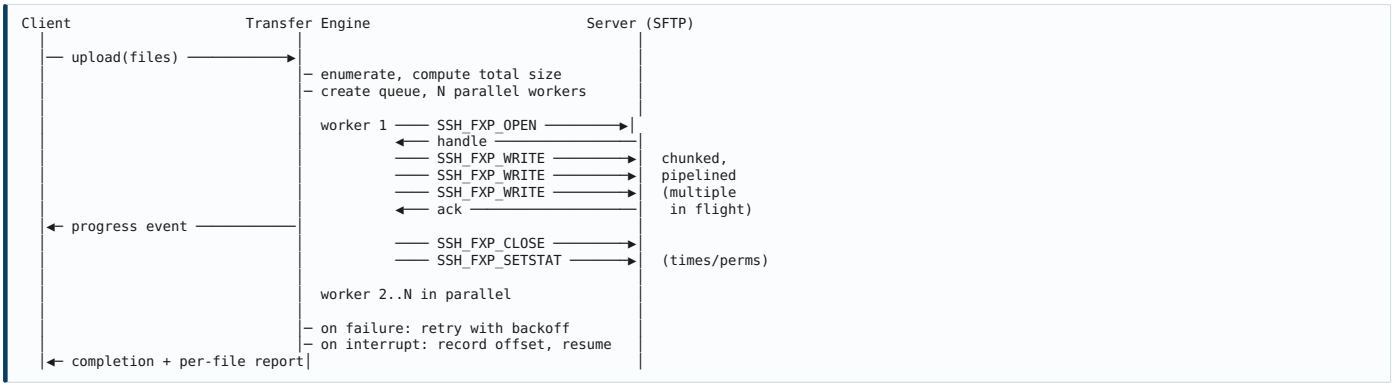


Elevation strategy.

Situation	Approach
Passwordless sudo detected	`sudo -n <cmd>` directly, no prompt
Password sudo required	Allocate PTY, `sudo -S`, write cached password to stdin, detect prompt
Password not cached	Prompt user, cache for session if permitted, clear on disconnect
Sudo unavailable	Offer `su -c` if root password is available; otherwise report the limitation clearly
Sudo denied	Report exactly which privilege is missing and what sudoers rule would grant it

Interactive prompt handling. Package operations and some configuration tools prompt mid-execution. These execute on a PTY channel with a prompt detector matching known patterns (`[Y/n]`, `[y/N]`, `Password:`, dpkg conffile prompts). On detection, execution pauses and the UI surfaces a graphical prompt; the response is written to the PTY. A timeout with automatic safe-default selection prevents indefinite hangs. **This mechanism should be prototyped early** — it is technically the trickiest part of the execution layer and it gates the entire package management module (see FR-PKG-014).

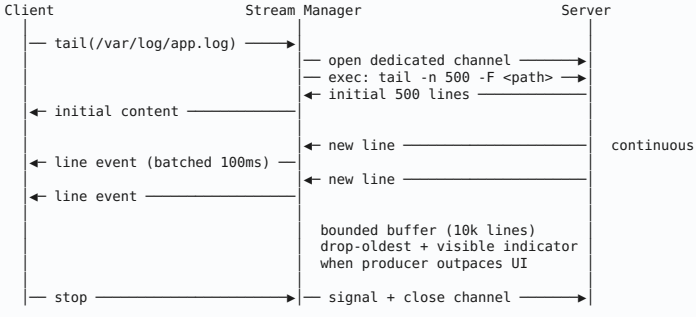
10.5 File Transfer



Throughput considerations. SFTP throughput on high-latency links is dominated by the number of outstanding write requests, not by bandwidth. A naive implementation that writes one chunk and waits achieves a small fraction of available bandwidth on a 100 ms link. The transfer engine must pipeline multiple outstanding requests per file *and* transfer multiple files in parallel. This is the primary technique for meeting NFR-PRF-015 (≥80% of raw SCP throughput).

Large directory optimization. For directories with many small files, per-file SFTP overhead dominates. Where `tar` is available on both ends, the engine may offer a streaming tar-over-SSH path — `tar cf - dir | ssh host 'tar xf - -C dest'` — which can be an order of magnitude faster for such trees. This should be automatic above a threshold, with the transparency panel showing the alternative command used.

10.6 Log Streaming



Rotation handling. `tail -F` (capital F) follows by name and survives rotation, unlike `tail -f`. For journald, `journalctl -f -o json` provides structured output that avoids fragile text parsing entirely and should be preferred wherever journald is present.

UI batching. Emitting an FX-thread event per log line would saturate the UI thread on a busy log. Lines are batched at 100 ms intervals, which bounds UI work regardless of log volume.

10.7 Metric Polling

A single batched command per cycle, executed on a pooled exec channel:

```
LC_ALL=C; echo "##CPU"; cat /proc/stat | head -n 20
echo "##MEM"; cat /proc/meminfo
echo "##LOAD"; cat /proc/loadavg
echo "##DISK"; df -PT
echo "##INODE"; df -PTi
echo "##NET"; cat /proc/net/dev
echo "##DISKIO"; cat /proc/diskstats
echo "##UPTIME"; cat /proc/uptime
```

One round trip yields every dashboard metric. CPU and I/O percentages are derived client-side by differencing consecutive samples, which is both more accurate and cheaper than invoking `top` or `iostat` remotely. The entire cycle costs a single process spawn on the server, satisfying NFR-PRF-010 (≤1% remote CPU).

10.8 Error Handling and Reconnection

Error taxonomy and response:

Class	Example	Response
Network transient	Timeout, reset	Auto-retry with backoff; auto-reconnect
Auth failure	Wrong password/key	Re-prompt; never auto-retry (account lockout risk)
Host key mismatch	Key changed	**Abort**, prominent security warning, no auto-retry
Permission denied	EACCES	Offer sudo elevation with explanation
Not found	ENOENT	Report path, refresh view, suggest verification
Disk full	ENOSPC	Report free space, offer cleanup navigation
Command not found	Missing tool	Disable feature, state which package provides it
Non-zero exit	Command failed	Show stderr, interpret if pattern known
Parse failure	Unexpected output format	Log raw output, show raw fallback, report as defect telemetry

Reconnection sequence:

```
Connection lost
    │
    ├─→ Mark session DEGRADED; UI shows reconnecting state, disables mutations
    ├─→ Preserve: open tabs, current paths, editor drafts, transfer queue state
    ├─→ Retry: 1 s, 2 s, 4 s, 8 s, 16 s, 30 s, 30 s… (cap 30 s, max 10 minutes)
    │
    ├─ SUCCESS ─→ Re-verify host key ─→ Re-authenticate ─→ Re-probe capabilities
    │                 │
    │                 ├─ Restore channels and paths
    │                 ├─ Resume interrupted transfers from recorded offsets
    │                 ├─ Restart log streams from last received position
    │                 └─ Notify: "Reconnected. 2 transfers resumed."
    │
    └─ EXHAUSTED ─→ Mark DISCONNECTED, preserve all local state,
                    offer manual reconnect, never discard unsaved editor content
```

Critical rule: in-flight *mutating* operations are never automatically retried after a reconnect. A `rm` that may or may not have executed before the connection dropped must not be re-issued blindly. The user is shown the operation's indeterminate state and asked to verify. Automatic retry of non-idempotent operations is a data-loss vector.

11. Security Design

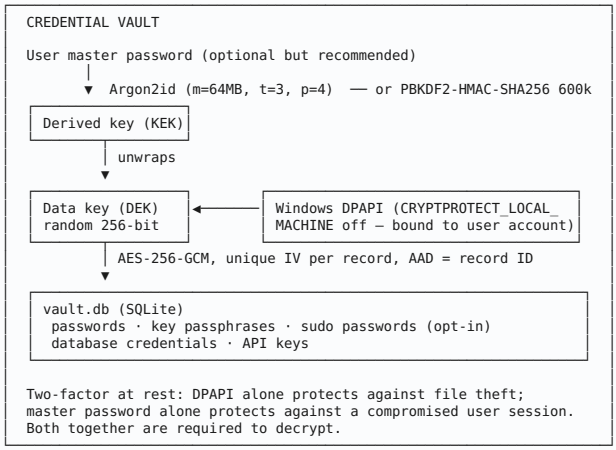
11.1 Threat Model

Assets: SSH credentials and private keys; host fingerprints; connection metadata revealing infrastructure topology; the SSH sessions themselves; the audit log; remote systems reachable through the application.

Adversaries and mitigations:

Adversary	Capability	Primary Mitigations
Malware on the operator's workstation	Read files, memory, keystrokes	DPAPI machine binding, master password, secret zeroing, no plaintext persistence
Network attacker (MITM)	Intercept, modify traffic	Mandatory host key verification, modern algorithms only, no downgrade
Malicious/compromised remote host	Return crafted output	Output treated as untrusted data; never interpolated into commands; parser hardening
Malicious plugin	Execute in-process	Capability declarations, classloader isolation, signature verification
Insider with workstation access	Physical access	Idle lock, master password, DPAPI user binding
Supply chain	Compromised dependency or update	Dependency scanning, pinned versions, signed updates, SBOM
Curious operator exceeding authority	Legitimate access, illegitimate use	Audit log, RBAC (Team tier), least-privilege guidance

11.2 Credential Storage



Rules:

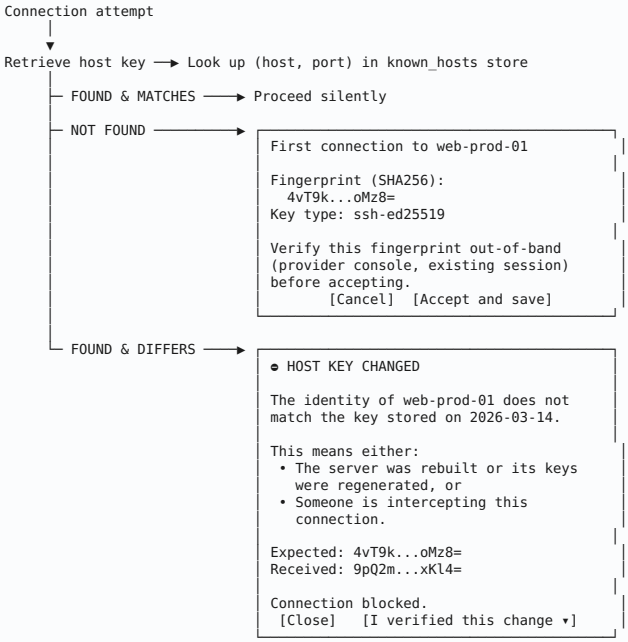
Rule	Implementation
Never store plaintext secrets on disk	All secret fields encrypted before write
Never hold secrets in `String`	`char[]` / `byte[]`, explicitly zeroed in `finally` blocks
Never log secrets	Redaction filter on all log appenders and telemetry
Never include secrets in diagnostics	Diagnostic bundle redacts by key pattern and shows a preview before export
Clear on disconnect	Session credential cache wiped on session close
Private keys stay in place	Reference the file path; never copy key material into the vault unless the user explicitly imports
Prefer external agents	Support OpenSSH agent and Pageant so key material never enters the process

11.3 SSH Key Management

Capability	Detail
Supported types	Ed25519 (recommended), ECDSA P-256/384/521, RSA ≥ 2048 (≥ 3072 recommended)
Formats	OpenSSH, PEM, PKCS#8, PuTTY `.ppk` (import)
Generation	Ed25519 default; passphrase required by default with an explicit opt-out
Deployment	Install public key to remote `authorized_keys` with correct permissions (700 `.ssh`, 600 `authorized_keys`)
Agent support	OpenSSH agent, Pageant
Forwarding	Off by default; enabling shows an explicit warning about remote-host trust
Rotation	Guided workflow: generate new, install, verify new key works, then remove old — in that order, never removing before verification
Auditing	List authorized keys per user with fingerprints; flag weak or ancient keys

Design note on rotation. The ordering is a safety property, not a convenience. Removing the old key before verifying the new one is the classic way to lock oneself out. The workflow must enforce the order and must actually test the new key with a separate authentication attempt before offering removal.

11.4 Host Verification



The mismatch dialog blocks by default. The override is deliberately secondary, requires expanding a disclosure, and is recorded in the audit log. This is one of the few places in the product where friction is intentional and correct.

11.5 Audit Logging

Record structure:

Field	Content
`id`	Monotonic sequence number
`timestamp`	ISO-8601 with timezone offset
`local_user`	Windows account
`session_id`	Session identifier
`host`	Target hostname and IP
`remote_user`	SSH username
`module`	Originating module
`intent`	Structured intent name
`command`	Exact command executed (secrets redacted)
`elevated`	Whether sudo was used
`blast_radius`	Tier 0–5
`confirmed_by`	Confirmation method, if any
`exit_code`	Result
`duration_ms`	Elapsed time
`bytes_transferred`	Where applicable
`outcome`	success / failure / cancelled / partial
`error`	stderr excerpt on failure
`prev_hash`	SHA-256 of the previous record
`hash`	SHA-256 of this record including `prev_hash`

Properties. Append-only. Hash-chained, so any deletion or modification breaks the chain and is detectable. Retention configurable with a minimum floor in Enterprise deployments. Exportable as CSV/JSON for compliance. Viewable and searchable in-application. Optional forwarding to syslog or SIEM in the Enterprise tier.

What is never logged: passwords, passphrases, key material, database query result contents, file contents, environment variable values matching secret patterns.

11.6 Permission and Privilege Model

Local (application) permissions — Team/Enterprise tiers:

Role	Capabilities
Viewer	Read-only: browse, view logs, view metrics, view configuration
Operator	Viewer + service control, process management, deployment to non-production
Administrator	Operator + user management, package management, firewall, production deployment
Owner	Administrator + profile management, RBAC assignment, audit configuration

Remote privilege principles:

1. **Never require root login.** The product must be fully usable as a non-root user with targeted sudo rules. Requiring `PermitRootLogin yes` would be a security regression and is explicitly rejected.
2. **Request elevation per operation, never per session.** No persistent root shell.
3. **Publish a minimal sudoers template.** Documentation should provide a sudoers fragment granting exactly the commands the product needs, so security-conscious organizations can grant least privilege rather than blanket sudo.
4. **Degrade visibly, not silently.** When privilege is insufficient, state which specific privilege is missing rather than showing an empty view.

11.7 Command Injection Prevention

The most significant application-specific vulnerability class. A filename, service name, or username originating from remote output or user input must never be able to alter command structure.

Control	Implementation
Never concatenate strings into commands	All commands built via `CommandBuilder` with typed, escaped parameters
Shell-quote every parameter	Single-quote wrapping with internal quote escaping; no exceptions
Validate against expected form	Service names, usernames, and package names matched against strict allowlist patterns before use
Reject control characters	Newlines, nulls, and escapes rejected in path and identifier parameters
Prefer argument arrays	Where the transport permits, avoid a shell entirely
Treat remote output as untrusted	Parsed output used as data only; never re-injected into a command without re-escaping
Path canonicalization	Resolve `..` and symlinks before permission and safety evaluation
Fuzz testing	Command builders fuzzed against adversarial filenames (`; rm -rf /` , `$(...)` , backticks, newlines, unicode homoglyphs) in CI

Illustrative failure this prevents: a directory containing a file named `foo; rm -rf ~` — legal on Linux — would, under naive string concatenation, cause a delete operation on that file to execute an arbitrary second command. This is not hypothetical; it is the single most likely path to a catastrophic defect in this class of application, and the `CommandBuilder` abstraction exists specifically to make it structurally impossible rather than merely unlikely.

12. Product Roadmap

12.1 Roadmap Overview

12.2 Phase 1 — Foundation (Months 1–6)

Objective. Prove the core thesis: a graphical, agentless interface that a developer will choose over WinSCP + PuTTY for daily work. Ship to private beta.

Deliverable	Requirements	Complexity
SSH/SFTP transport with channel pooling and reconnection	FR-CON-001–013, 080–087	**XL**
Authentication (password, key, agent, keyboard-interactive, sudo)	FR-CON-020–030	**L**
Host key verification and known-hosts management	FR-CON-040–045	M
Server profiles, groups, colour tags, production marking	FR-CON-050–060	M
Capability probe and adapter framework	FR-CON-090–095	**XL**
Command Abstraction Layer with safety policy	§8.4, FR-SEC-020	**XL**
Transparency panel	P3, FR-GEN-001	M
Audit log (hash-chained)	FR-SEC-016, §11.5	M
File browser with virtualized table	FR-FIL-001–020	**XL**
File operations	FR-FIL-030–045	L
Transfer engine with queue, parallelism, resume	FR-FIL-050–065	**XL**
Drag and drop including drag-out to Explorer	FR-FIL-110–116	**XL**
Search	FR-FIL-070–078	M
Permissions and ownership	FR-FIL-080–091	M
Compression and archives	FR-FIL-100–108	M
File editor with highlighting, find/replace, history	FR-EDT-001–050	**XL**
Config validation (nginx, sshd, sudoers, JSON, YAML)	FR-EDT-037–038	M
Service manager (systemd)	FR-SVC-001–031	L
Process manager	FR-PRC-001–024	L
Monitoring and dashboard	FR-MON-001–086	**XL**
Log viewer with streaming and windowed loading	FR-LOG-001–024	**XL**
Deployment engine with backup and rollback	FR-DEP-001–060	**XL**
Application shell, ribbon, themes, keyboard model	§7.2, §7.12–7.13	**XL**
Credential vault (DPAPI + AES-GCM)	§11.2	L
Packaging, installer, auto-update	NFR-SEC-010–011	L
CI with Testcontainers across 6 distributions	§9.3	L

Estimated effort: 42–52 engineer-months. With 4 engineers at realistic productivity, 6 months is achievable but tight. **Recommended contingency: 25%.**

Highest-risk items, to be spiked in the first six weeks:

1. Drag-out to Windows Explorer (virtual file promises) — **prototype before committing to the design**
2. PTY-based sudo and interactive prompt handling
3. Virtualized table performance at 100k rows in JavaFX
4. Transfer engine throughput on high-latency links
5. Capability probe reliability across the distribution matrix

Exit criteria for Phase 1: a beta user can perform a full deploy-verify-rollback cycle, diagnose a slow server from the dashboard, and edit and validate a config file — without opening a terminal.

12.3 Phase 2 — Operations (Months 7–12)

Objective. Complete the administrative surface. Reach 1.0 general availability with a paid tier.

Deliverable	Requirements	Complexity
Package management with PTY prompt handling	FR-PKG-001–025	**XL**
User, group, and SSH key management	FR-USR-001–026	L
Cron and systemd timer management	FR-CRN-001–016	L
Network interfaces, routing, DNS	FR-NET-001–007, 018–020	L
Firewall management with session-preservation guard	FR-NET-008–014	**XL**
SSH server configuration with validation	FR-NET-016–017	M
Backup and restore	FR-BAK-001–020	**XL**
Security posture summary	FR-SEC-012–019	L
Multi-host dashboard and batch operations	FR-MON-083, FR-CON-087	**XL**
System configuration and environment variables	FR-SYS, FR-ENV	M
Disk and storage (read-mostly)	FR-DSK-001–007	M
Command palette	NFR-USA-005	M
Localization framework and first languages	NFR-LOC-001–005	M
Accessibility conformance	NFR-USA-008–014	L
Licensing, billing, tier enforcement	—	L
Documentation and onboarding	—	L
Third-party penetration test	NFR-SEC-014	—

Estimated effort: 38–46 engineer-months.

Exit criteria: every P0 persona's Critical features from §4.9 are shipped; penetration test findings remediated; 1.0 released commercially.

12.4 Phase 3 — Platform (Months 13–18)

Objective. Extend to containers, databases, and teams. Open the platform. Add Linux desktop support.

Deliverable	Requirements	Complexity
Docker container, image, volume, network management	FR-DOC-001–037	**XL**
Docker Compose support	FR-DOC-032–035	L
Database management (MySQL, PostgreSQL, SQLite)	FR-DBM-001–023	**XL**
Certificate and SSL management with certbot	FR-SEC-004–011	L
Plugin API and loader with isolation	§8.10, NFR-EXT-005–010	**XL**
Team tier: shared profiles, RBAC, centralized audit	§11.6	**XL**
Linux desktop build	NFR-CRP-004	L
Advanced deployment (staging swap, zero-downtime)	FR-DEP-018	L
Saved operations / runbooks	—	L
Performance optimization pass	§6.1 targets	L

Estimated effort: 40–48 engineer-months.

12.5 Phase 4 — Intelligence and Enterprise (Months 19–24)

Deliverable	Requirements	Complexity
AI assistance: NL→command proposals, error explanation, log analysis	FR-AI-001–018	**XL**
Local model support for privacy-sensitive deployments	FR-AI-015	L
Enterprise: SSO/SAML/OIDC, self-hosted licence server, air-gap	—	**XL**
SIEM integration and compliance reporting	§11.5	L
macOS build	NFR-CRP-005	L
Plugin marketplace	§14.2	**XL**
Approval workflows for high-risk operations	—	L
SOC 2 Type II preparation	—	—

Estimated effort: 42–50 engineer-months.

12.6 Phase 5 — Expansion (Months 25–36)

Deliverable	Complexity
Kubernetes management (namespaces, workloads, logs, exec)	**XL**

Cloud settings sync with end-to-end encryption	**XL**
Real-time collaboration (shared session view)	**XL**
Mobile companion (monitoring and alerts, limited actions)	**XL**
Automation engine (event-triggered saved operations)	**XL**
Ansible playbook generation from action sequences	L
Cloud provider integration (AWS, Azure, GCP metadata)	L

Estimated effort: 50–65 engineer-months.

12.7 Cumulative Effort Summary

Phase	Duration	Team	Engineer-Months	Cumulative
1 — Foundation	6 mo	4	42–52	42–52
2 — Operations	6 mo	5	38–46	80–98
3 — Platform	6 mo	6	40–48	120–146
4 — Intelligence	6 mo	7	42–50	162–196
5 — Expansion	12 mo	8	50–65	212–261

Non-engineering roles required throughout: product manager (1.0 FTE from month 1), UX designer (1.0 FTE Phase 1–3, 0.5 thereafter), QA engineer (1.0 from month 3, 2.0 from month 9), technical writer (0.5 from month 4), DevOps (0.5 throughout), security consultant (engagement-based).

Indicative fully-loaded cost to 1.0 GA (end of Phase 2): approximately 80–98 engineer-months plus ~14 months of non-engineering effort. At blended rates this is a meaningful multi-million-dollar investment before first significant revenue. This figure should anchor the funding conversation.

12.8 Complexity Distribution

Complexity	Count	Representative Items
XL (> 6 weeks)	24	Transport, CAL, transfer engine, drag-out, editor, log viewer, deployment, monitoring, firewall guard, package PTY, Docker, database, plugin API, Team tier, AI, Kubernetes
L (2–6 weeks)	31	Auth, services, processes, users, cron, backup, certificates, licensing, accessibility
M (1–2 weeks)	38	Host keys, profiles, search, permissions, archives, palette, localization
S/XS (< 1 week)	60+	Individual dialogs, small features, settings

The XL concentration in Phase 1 is the schedule's principal risk. Four of the six Phase 1 XL items (transport, CAL, transfer engine, drag-out) are on the critical path and cannot be parallelized easily.

13. Risks

13.1 Risk Register

Scoring: Probability (1–5) × Impact (1–5) = Score. **Bold** indicates score ≥ 15.

Technical Risks

ID	Risk	P	I	Score	Mitigation
T1	**Distribution fragmentation makes reliable abstraction infeasible; parsers break constantly**	4	5	**20**	Machine-readable output only; `LC_ALL=C`; fixture corpus per distro/version; CI matrix; explicit supported-distro list; graceful raw-output fallback
T2	**JavaFX cannot meet performance targets for 100k-row tables**	3	5	**15**	Spike in month 1; custom virtualized control if needed; hard row cap with paging as fallback
T3	Drag-out to Explorer proves impractical	3	3	9	Spike in month 1; temp-materialization fallback accepted
T4	SSH library limitation forces migration mid-project	2	4	8	Transport wrapped behind internal interface from day one
T5	PTY-based sudo/prompt handling unreliable across environments	3	4	12	Early spike; prompt-pattern corpus; timeout with safe default; documented limitations
T6	Memory exhaustion on large logs or directory trees	3	4	12	Windowed loading mandated architecturally; bounded buffers; soak tests
T7	High-latency links make the product feel unusable	3	4	12	Command batching; channel pooling; optimistic UI with reconciliation; latency simulation in test
T8	Java desktop packaging friction (antivirus false positives, SmartScreen)	3	3	9	EV code signing; reputation building; MSI best practices
T9	Command injection defect reaches production	2	5	10	CommandBuilder abstraction; fuzz testing in CI; security review of all builders
T10	Cancellation leaves orphaned remote processes	3	3	9	Explicit signal-and-verify on cancel; orphan detection on reconnect

Business and Market Risks

ID	Risk	P	I	Score	Mitigation
B1	**Cockpit is free and improving; "good enough" for most**	4	4	**16**	Compete on agentless, multi-host, deployment, native performance; be honest where Cockpit wins; free tier for bottom-up adoption
B2	**Scope is too large; runs out of funding before differentiation ships**	4	5	**20**	Ruthless Phase 1 scoping; ship narrow and excellent; resist breadth-before-depth
B3	Willingness to pay is lower than assumed	3	4	12	Validate pricing with 30+ interviews before Phase 2; instrument free-tier conversion early
B4	Enterprise sales cycle exceeds runway	3	4	12	Fund from self-serve Pro; treat Enterprise as upside, not the plan
B5	A major vendor (Red Hat, Microsoft, a cloud provider) ships an equivalent	2	5	10	Move fast; build community; differentiate on agentless breadth
B6	Open-source clone erodes commercial viability	3	3	9	Compete on polish, support, and Team/Enterprise features rather than raw capability
B7	Support burden across the distro matrix exceeds capacity	4	3	12	Explicit supported-platform policy; self-service diagnostics; community forum

Adoption Risks

ID	Risk	P	I	Score	Mitigation
A1	**Technical community dismisses it as "a GUI for people who can't use Linux"**	4	4	**16**	Command transparency as the central answer; position as learning accelerator; escape hatch always visible; engage skeptics openly rather than defensively
A2	Users graduate to the shell and churn	3	3	9	Reframe as a feature; value shifts to multi-host, deployment, and audit which the shell does not provide
A3	Abstraction is distrusted after a single wrong command	3	5	15	Transparency panel; exhaustive parser testing; conservative defaults; never claim unverified success
A4	Free tier cannibalizes Pro	3	3	9	Careful tier design: host count and deployment as the primary gates
A5	Windows-only limits addressable market	3	3	9	Linux build Phase 3, macOS Phase 4; architecture prepared from day one

Security Risks

ID	Risk	P	I	Score	Mitigation
S1	**A credential-exposure vulnerability destroys trust permanently**	2	5	**10**	Defense in depth (§11.2); external penetration test pre-GA; bug bounty; rapid disclosure policy
S2	Supply chain compromise of a dependency	2	5	10	Dependency scanning; pinned versions; SBOM; signed builds; reproducible build investigation
S3	Malicious plugin exfiltrates credentials	3	4	12	Capability model; signature verification; curation; isolation
S4	AI feature leaks sensitive server data to a third party	3	4	12	Opt-in; explicit disclosure; redaction; local model option; enterprise policy disable
S5	Product used as an attack tool against systems the user does not own	2	3	6	Terms of use; no offensive tooling; audit log as deterrent

Performance Risks

ID	Risk	P	I	Score	Mitigation
P1	Startup time exceeds 3 s, damaging first impressions	3	3	9	jlink module stripping; lazy module init; compile-time DI; startup budget enforced in CI
P2	Memory footprint grows unacceptably with many sessions	3	4	12	Per-session budget; cache bounds; 24-hour soak tests in CI
P3	Polling burdens remote hosts, causing user complaints	3	4	12	Single batched poll; visibility-gated polling; configurable interval; measured overhead published
P4	Transfer throughput materially below WinSCP	3	4	12	Pipelined SFTP writes; parallel workers; tar-over-SSH for many small files; benchmark against WinSCP continuously

13.2 Top Five Risks — Detail

B2 (score 20) — Scope overrun. This is the most likely cause of failure. The vision specifies more than twenty functional domains. Phase 1 alone is 42–52 engineer-months and contains six XL items on the critical path. The failure mode is predictable: everything is 70% complete, nothing is excellent, funding runs out. **The mitigation is organizational, not technical** — a product owner with the authority and inclination to cut scope, and a Phase 1 definition treated as a contract rather than an aspiration. Consider cutting Docker, databases, and AI entirely from the first two years' plan if velocity disappoints.

T1 (score 20) — Distribution fragmentation. The adapter architecture addresses this structurally, but the practical burden is a parser corpus that must be maintained as distributions release new versions. A new Ubuntu LTS or RHEL major release can silently change output formats. **Mitigation requires ongoing investment, not a one-time fix:** an explicit supported-platform list, CI running against real distribution images, and a graceful raw-output fallback so an unparseable response degrades to "here is what the server said" rather than an error.

B1 (score 16) — Cockpit. A free, well-designed, vendor-backed competitor with substantial functional overlap. The honest position is that Cockpit is the better choice for some users, and saying so builds credibility. LinuxDesk's defensible ground is agentless operation, multi-host work, local↔remote deployment, and native desktop performance. **If the product cannot articulate why someone would choose it over Cockpit in one sentence, it does not have a market.**

A1 (score 16) — Community dismissal. Sysadmin culture treats shell fluency as a professional marker, and GUI tools carry stigma. A dismissive reception on Hacker News or r/sysadmin would materially damage adoption. **The command transparency panel is the direct answer** and should be the centerpiece of technical marketing: this tool shows you every command it runs. Engaging skeptics openly — including publishing the sudoers template and the exact commands each module issues — converts more of them than defensiveness will.

T2 (score 15) — JavaFX table performance. 100,000 rows at 55 fps is demanding for JavaFX's `TableView`. If the built-in control cannot meet it, a custom canvas-based virtualized control is required — a substantial, unbudgeted piece of work. **This must be spiked in month 1**, before the file module design is committed. A hard row cap with paging is an acceptable fallback but weakens a stated differentiator.

13.3 Assumptions Register

ID	Assumption	If False
AS-1	Target users will pay ~$10/month for this capability	Revenue model requires rework; consider one-time licence or higher Team focus
AS-2	Agentless operation is a genuine buying criterion, not a theoretical preference	Primary differentiator collapses; Cockpit comparison becomes unfavourable
AS-3	Command transparency defuses community skepticism	Adoption depends more heavily on paid acquisition
AS-4	JavaFX can meet the stated performance targets	Toolkit change or scope reduction required
AS-5	Six distributions cover the large majority of target hosts	Support matrix expands, increasing ongoing cost
AS-6	4–5 engineers can deliver Phase 1 in 6 months	Timeline extends or scope must be cut
AS-7	SSH-only access is sufficient for every specified capability	Some modules become infeasible; scope reduction required

AS-2 is the assumption most worth validating before significant spend. The entire positioning rests on it. Twenty structured interviews with target users — specifically asking whether they *could* install Cockpit and why they have or have not — would substantially de-risk the investment for a trivial cost.

14. Future Enhancements

14.1 Plugin Ecosystem

Beyond the Phase 3 plugin API: a plugin SDK with project templates and a local test harness; a curated official plugin set (Nginx configuration, Let's Encrypt, WordPress, Redis, Elasticsearch); community plugin discovery within the application; and a plugin development guide. The strategic value is coverage of the long tail without core engineering cost — Webmin's module ecosystem is the demonstration that this works, and its uneven quality is the demonstration that curation matters.

14.2 Marketplace

A distribution channel for plugins, themes, deployment templates, and saved operation libraries, with signature verification, ratings, and optional paid listings with revenue sharing. This becomes viable only at meaningful installed-base scale; attempting it early produces an empty store that signals failure.

14.3 Cloud Sync

End-to-end encrypted synchronization of profiles, settings, bookmarks, and deployment configurations across devices — with private keys explicitly excluded from sync by default. Termius demonstrates both the demand and the model. This is a Team-tier differentiator and must be architected so that the vendor cannot decrypt user data even under compulsion; anything less will fail enterprise review.

14.4 Collaboration

Shared session viewing (one operator acts, others observe in real time) for incident response and mentoring; annotation of shared dashboards; and handoff notes attached to hosts. The mentoring use case is particularly compelling for Persona 2, who currently cannot delegate without either granting root or watching over a shoulder.

14.5 Remote Terminal Enhancement

The escape-hatch terminal can be improved without becoming the product: full xterm-256color emulation, tmux/screen session attachment, split panes, and — most valuably — **bidirectional context integration**, where the terminal opens in the current GUI directory and GUI views refresh when terminal commands change state. A terminal that knows what the GUI is looking at, and vice versa, is a differentiator no pure terminal can match.

14.6 AI Automation

Beyond Phase 4's assistive features: anomaly detection over metric history; predictive alerts (disk-full projection, memory leak detection); root-cause hypothesis ranking correlating metrics, logs, and recent changes; and natural-language runbook generation. **All must remain proposal-only** (FR-AI-002). The most valuable near-term application is correlation — "this error started at 08:03, which is when deployment v12 completed" — which requires no model at all, only the integrated data the product already holds. **Correlation before generation** is the right sequencing.

14.7 Mobile Companion

A read-mostly iOS/Android application for monitoring, alerting, log viewing, and a small set of safe actions (restart a service, acknowledge an alert). Not a full administration client — administration from a phone is a poor idea and a security liability. The value is the 3 a.m. alert that can be triaged before deciding whether to open a laptop.

14.8 Other Candidate Directions

Enhancement	Value	Complexity
Kubernetes management	Extends to container-orchestrated fleets	**XL**
Ansible playbook generation from action sequences	Bridges interactive and declarative; strong differentiator	L
Infrastructure diagram generation from discovered topology	Documentation and onboarding value	L
Compliance scanning (CIS benchmarks)	Enterprise value	**XL**
Cost visibility for cloud hosts	Adjacent value for freelancers	M
Scheduled health reports by email	Passive value for Persona 6	M
Windows Server support via WinRM	Doubles addressable hosts; dilutes focus	**XL**
Web UI companion for the desktop app	Access from unmanaged machines	**XL**
Terraform state visualization	DevOps adjacency	L
Session recording and replay	Compliance and training	L

15. Final Recommendation

15.1 Technical Feasibility

Verdict: feasible, with three qualifications.

Nothing in this specification requires unproven technology. SSH and SFTP are stable, well-documented protocols with mature Java implementations. Ansible proves at scale that comprehensive agentless management over SSH works. Cockpit and Webmin prove that graphical Linux administration is achievable. JavaFX is a capable desktop toolkit. The individual pieces all exist.

The difficulty is **integration at breadth**, not any individual capability.

Qualification 1 — Output parsing is the dominant ongoing engineering cost. Not the hardest problem, but the one that never ends. Distributions release, tools change output, and parsers break. The architecture handles this correctly (machine-readable formats, forced locale, fixture corpus, adapter isolation), but this is a permanent maintenance obligation rather than a one-time build. Budget for it indefinitely.

Qualification 2 — Three items carry real technical uncertainty and must be spiked before design is committed:

Item	Uncertainty	Spike
JavaFX table at 100k rows, 55 fps	Whether the built-in control suffices	Month 1, 1 week
Drag-out to Explorer via virtual file promises	Whether JNA interop is workable and reliable	Month 1, 2 weeks
PTY sudo and interactive prompt handling	Reliability across distributions and prompt variants	Month 2, 2 weeks

Each has an acceptable fallback (paging; temp materialization; documented limitation with terminal handoff), so none is existential — but each would change the design if discovered late.

Qualification 3 — Some specified capabilities are poor fits for remote graphical management and should be scoped down rather than attempted. Full network reconfiguration (FR-NET-003) risks severing the connection and varies enormously across netplan, NetworkManager, systemd-networkd, and ifupdown. Partitioning and filesystem operations (FR-DSK-009, FR-DSK-012) destroy data irrecoverably when wrong. Both are correctly ranked Could-have and should stay read-only in v1. **The staged-apply-with-automatic-revert pattern (§5.14) should be treated as a prerequisite for any network mutation feature**, not an enhancement to it.

15.2 Development Effort

Milestone	Elapsed	Engineer-Months	Team
Private beta (Phase 1)	6 months	42–52	4 engineers + PM + UX
1.0 GA (Phase 2)	**12 months**	**80–98**	**5 engineers + PM + UX + QA**
2.0 (Phase 3)	18 months	120–146	6 engineers + support
3.0 (Phase 4)	24 months	162–196	7 engineers
4.0 (Phase 5)	36 months	212–261	8 engineers

Recommended contingency: 25% on Phase 1, 20% thereafter. The Phase 1 XL concentration and the three technical unknowns justify the higher figure.

Minimum viable team for Phase 1:

Role	FTE	Notes
Senior Java/JavaFX engineer	2	One must have deep JavaFX experience — this is not a commodity skill
Backend/systems engineer	1.5	SSH, Linux internals, parsing
Windows platform engineer	0.5	Native interop, packaging, drag-out
Product manager	1	Critical for scope discipline (risk B2)
UX designer	1	The abstraction *is* the product
QA engineer	1 (from month 3)	Distribution matrix testing
DevOps	0.5	CI, containerized test infrastructure

A smaller team is possible only with a materially smaller Phase 1. A viable reduced scope: connection, files, editor, deployment, and logs only — dropping services, processes, and monitoring to Phase 2. That is roughly 26–32 engineer-months and deliverable by three engineers in six months. It would still be a better product than WinSCP for the developer persona.

15.3 Does This Fill a Market Gap?

Yes — a real one, though narrower than the vision implies.

The gap is precise: **agentless, broad-surface, task-oriented, native-desktop, multi-host Linux administration.** Section 2.15 shows that quadrant occupied only by Ansible, which sits there at a very high expertise requirement. At a low expertise requirement, it is empty.

The gap is real because:

- WinSCP, PuTTY, MobaXterm, and Termius stop at files and terminals.

- Cockpit and Webmin require server-side installation, which is disqualifying for hardened hosts, port-22-only environments, and operators without install authority.
- Ansible raises rather than lowers the expertise floor and offers no interactive exploration.
- Portainer and VS Code Remote are excellent within domains far narrower than this scope.

But the gap is narrower than the vision states, for three reasons:

1. **Cockpit is genuinely good and genuinely free.** For a single host where installation is permitted, it is a strong choice. The addressable market is people for whom installation is impossible, undesirable, or insufficient — a real population, but a subset.
2. **Many target users have working habits, not just working tools.** WinSCP plus PuTTY plus a bookmarked cheat sheet is a functioning workflow. Displacing a functioning workflow requires being substantially better, not marginally better.
3. **The breadth in the vision exceeds what the market will pay a premium for.** Files, deployment, services, logs, and monitoring will drive nearly all purchase decisions. Certificates, disk partitioning, and environment variable management are completeness features, not conversion features. **Building them early would be a misallocation.**

15.4 Strongest Differentiators

In order of defensibility:

1. Agentless with full breadth. The one property no comparable-breadth competitor has. Every marketing message should lead with it: *nothing installed on the server.*

2. Command transparency. Simultaneously the trust mechanism, the learning mechanism, and the answer to community skepticism. Cheap to build, hard to retrofit into competitors whose entire design premise is hiding the command. This is the product's philosophical core and should be treated as sacred.

3. Deployment with real rollback. The most commercially compelling single feature for the highest-conversion personas. No competitor at this layer offers integrated local↔remote diff, automatic pre-deploy backup, health check, and one-click rollback.

4. Safety architecture. Blast-radius tiering, session-preservation guards, `WHERE`-clause warnings, last-sudo-user protection, and staged network changes with automatic revert. Individually small; collectively they make the GUI *safer* than the shell, which inverts the usual expectation and is a genuinely novel claim.

5. Multi-host as a first-class concept. Cockpit's weakest area and a daily need for Personas 2 and 5.

6. Contextual integration. Logs, metrics, services, and deployment history in one application means correlation is possible. "This error began when deployment v12 landed" requires no AI — only integrated data — and no single-purpose competitor can offer it.

15.5 Recommended Changes Before Development Begins

1. Cut Phase 1 scope by roughly a third. Nine major modules in six months with four engineers is not realistic. Recommended Phase 1: connection, files, editor, deployment, logs, and a basic dashboard. Move services, processes, and full monitoring to an early Phase 2. **Ship five excellent modules rather than nine adequate ones.** Risk B2 is the highest-scoring risk in the register and this is its primary mitigation.

2. Validate assumption AS-2 before committing significant spend. Twenty to thirty structured interviews with target users, asking specifically: could you install Cockpit on your hosts? Have you? Why not? If the answers indicate that installation is rarely a real barrier, the core differentiator is weaker than assumed and positioning must change. This costs perhaps three weeks and de-risks a multi-million-dollar investment.

3. Run the three technical spikes in months 1–2, before design commitment. JavaFX table performance, Explorer drag-out, and PTY prompt handling. Each has an acceptable fallback, but discovering the fallback is needed in month 5 rather than month 1 is expensive.

4. Narrow the supported distribution list explicitly. Recommended v1: Ubuntu 22.04/24.04 LTS, Debian 12, RHEL 9 and Rocky/Alma 9, Amazon Linux 2023. Publish it. Everything else is best-effort with a clear disclaimer. An unbounded support matrix is an unbounded cost, and risk T1 is scored 20 largely because of it.

5. Rename the product. "LinuxDesk" is a placeholder. The name should communicate control and safety rather than merely "Linux." Conduct a proper trademark search before any public commitment — this is inexpensive now and extremely expensive later.

6. Design the audit log, credential vault, and adapter interfaces in Phase 1 even though their consumers arrive in Phase 3–4. These are the architectural decisions that are prohibitively expensive to retrofit. Enterprise requirements shape architecture from day one (§4.8).

7. Reduce AI scope to error explanation and log correlation only. Natural-language command generation is the highest-risk, lowest-differentiation AI feature. Error interpretation and log correlation deliver most of the value with a fraction of the risk, and correlation requires no model at all. Defer generation to Phase 4 or later.

8. Publish the sudoers template and the command catalogue early. Documenting exactly what commands each module runs, and providing a least-privilege sudoers fragment, directly addresses risk A1 (community dismissal) and risk S1 (trust). It is also simply the right thing to do for a tool that executes commands on production infrastructure.

9. Treat the transparency panel as a hard requirement, not a feature. It should be impossible to ship a module that executes a command without publishing it. Enforce this architecturally — the executor should require a transparency publication before execution — rather than by convention.

10. Establish the performance budget as a CI gate in month 1. Startup time, memory footprint, and table render time should fail the build when exceeded. Performance regressions accumulate invisibly and are far cheaper to prevent than to fix at month 20.

15.6 Overall Recommendation

> ***Proceed — with a materially reduced Phase 1 scope and a validated core assumption.***

The product concept is sound. The gap it targets is real, if narrower than the vision claims. The architecture proposed here is appropriate and the technology choices are sensible. The differentiators are defensible, and one of them — command transparency — is both cheap to build and philosophically central in a way competitors would struggle to copy without abandoning their own design premises.

The primary risk is not technical. It is scope. This specification describes a product that could absorb five years and thirty engineers. The version that succeeds is the one that does five things excellently in year one and expands from a position of strength. Every additional module in Phase 1 increases the probability that nothing ships well.

The second risk is positioning. Being honest about where Cockpit is the better choice will build more credibility with the target technical audience than claiming universal superiority. A product that says "if you can install Cockpit everywhere and manage one host at a time, use Cockpit — we are for the other cases" will be trusted. One that claims to beat everything at everything will not.

The recommended immediate next steps, in order:

1. Validate AS-2 with 20–30 target-user interviews (3 weeks)
2. Run the three technical spikes in parallel (4 weeks)
3. Rewrite Phase 1 scope based on both results (1 week)
4. Confirm funding against the revised, contingency-adjusted estimate
5. Begin construction of transport, CAL, and safety policy — the three components everything else depends on

---

Appendix A — Requirement Count Summary

Module	Must	Should	Could	Won't	Total
Connection Management	31	18	8	0	57
File Management	52	24	12	0	88
File Editor	24	17	6	0	47
Deployment	34	8	4	0	46
Monitoring	42	15	12	1	70
Process Management	14	8	2	0	24
Service Management	20	10	2	0	32
Log Viewer	15	6	3	0	24
Docker	14	14	9	0	37
Database	15	6	4	0	25
Users & Permissions	17	8	1	0	26
Cron & Scheduler	8	5	3	0	16
Package Management	14	8	3	0	25
Network	10	8	2	0	20
Backup & Restore	12	6	2	0	20
Security	8	8	5	0	21
System / Env / Disk	7	15	11	1	34
AI Features	8	6	4	0	18
Cross-cutting	8	0	0	0	8
Total functional	**353**	**190**	**93**	**2**	**638**
Non-functional	78	34	9	0	121
Grand total	**431**	**224**	**102**	**2**	**759**

Appendix B — Traceability: Personas → Phases

Persona	Phase 1 Coverage	Phase 2	Phase 3	Fully Served
P1 Developer	85%	95%	100%	**Phase 1**
P2 SysAdmin	45%	90%	95%	Phase 2
P3 DevOps	40%	65%	90%	Phase 3
P4 Student	70%	85%	90%	**Phase 1**
P5 Freelancer	65%	95%	100%	Phase 2
P6 SMB	30%	70%	80%	Phase 2
P7 Enterprise	15%	35%	70%	Phase 4

Phase 1 fully serves the developer and student personas — the two most likely to generate early advocacy and word-of-mouth. This is the

correct early-adopter target and validates the recommended Phase 1 reduction in §15.5.

Appendix C — Supported Platform Policy (Recommended v1)

Distribution	Versions	Support Level
Ubuntu	22.04 LTS, 24.04 LTS	Full — CI tested every build
Debian	12	Full — CI tested every build
RHEL / Rocky / AlmaLinux	9.x	Full — CI tested every build
Amazon Linux	2023	Full — CI tested every build
RHEL / CentOS	7, 8	Best effort — degraded features documented
SUSE / openSUSE	15.x	Best effort
Alpine	3.19+	Best effort — limited (no systemd)
Arch	Rolling	Community
Others	—	Unsupported; core file and command features may work

Client requirements: Windows 10 build 1809 or later, or Windows 11; x64 or ARM64; 4 GB RAM minimum, 8 GB recommended; 500 MB disk.

End of document.